Going Beyond Worst-Case: A Study on Cache Adaptivity and Machine Learning Advice

A Dissertation Presented

by

Arghya Bhattacharya

to

The Graduate School

in Partial Fulfillment of the Requirements

for the Degree of

Doctor of Philosophy

in

Computer Science

Stony Brook University

December 2024

Stony Brook University

The Graduate School

Arghya Bhattacharya

We, the dissertation committee for the above candidate for the Doctor of Philosophy degree, hereby recommend acceptance of this dissertation.

Dr. Michael A. Bender – Dissertation Advisor
John L. Hennessy Chaired Professor, Computer Science, Stony Brook University

Dr. Rezaul A. Chowdhury – Chairperson of Defense
Associate Professor, Computer Science, Stony Brook University

Dr. Joseph S. B. Mitchell – Committee Member
Distinguished Professor and Chair, Applied Mathematics and Statistics, Stony Brook University

Dr. Helen Xu – External Committee Member
Assistant Professor, Computational Science and Engineering Georgia Tech

This dissertation is accepted by the Graduate School.

Dr. Celia Marshik
Dean of the Graduate School, Stony Brook University

Abstract of the Dissertation

Going Beyond Worst-Case: A Study on Cache Adaptivity and Machine Learning Advice

by

Arghya Bhattacharya

Doctor of Philosophy

in

Computer Science

Stony Brook University

2024

Traditional online algorithms are typically evaluated through competitive analysis, which compares their performance against an optimal offline algorithm under worst-case scenarios. However, the focus on the worst-case performance may sometimes fail to provide insights into real-world performance. Several theoretical approaches have been taken to study beyond-worst-case performance of online algorithms. This thesis investigates two techniques to look beyond-worst-case for algorithm designers.

First, we study how to write programs for multi-core, multi-threaded, shared-memory systems where the individual programs may face fluctuations of available resources. We formally refer to this as the cache adaptivity problem. We develop an empirical framework to assess whether algorithms can achieve cache adaptivity and use this framework to evaluate cache-friendly algorithms under various memory fluctuation scenarios. Our findings show that cache-obliviousness is a strong proxy for cache adaptivity in non-adversarial memory conditions. In particular, we experimentally

evaluate the theoretical worst-case performance gap between cache-adaptive and non-adaptive algorithms for matrix multiplication and external-memory sorting under a range of memory workloads that are non-adaptive in the sense that they remain oblivious to the execution of the program.

Second, we explore Machine Learning (ML)-augmented online algorithms, which can perform provably better than the traditional online algorithms when the error of the machine learning oracle is low for worst-case and average-case inputs. Even with poor ML advice, these algorithms remain robust, avoiding abrupt performance degradation. We propose a novel variant of rent-or-buy problem, ski rental problem with a discount, and introduce ML-advised algorithms to solve it with one or more pieces of advice. Furthermore, we investigate the list update problem, demonstrating that the design of ML advice is crucial for achieving performance gains. Accurate predictions alone do not guarantee improved performance.

Contents

List of Figures **vii**

Acknowledgments **x**

1 Introduction **1**
1.1 Tools to analyze algorithmic performance 1
1.2 Architecture of modern computing environments 3
1.3 Thesis contribution 5
1.3.1 Cache-adaptive algorithms design 6
1.3.2 ML-advised algorithm design 9

2 Cache Adaptivity **13**
2.1 Introduction 14
2.2 Problems and algorithms 21
2.2.1 Matrix multiplication algorithms 21
2.2.2 External-memory sorting algorithms 23
2.3 Cache-adaptive analysis 26
2.3.1 External memory models 26
2.3.2 Analytical tools for cache-adaptive analysis 30
2.3.3 Theoretical results on cache-adaptive model 31
2.4 Memory profile design 31
2.5 Evaluation on constant profiles 34
2.6 Evaluation on adaptive profiles 35
2.7 Evaluation on oblivious profiles 37
2.8 Conclusion 43

3 ML-Advised Algorithms for Ski Rental Problem **45**
3.1 Introduction 46
3.2 Preliminaries 50
3.3 Ski rental problem with a discount 53
3.4 Algorithms with a single ML adviser 54

3.5 Algorithms with two ML advisers 58
3.6 Experimental evaluation 62
3.7 Conclusion 65

4 ML-Advised List Update Problem 66
4.1 Introduction 66
4.2 List update algorithm with ML advisor 68
4.2.1 PDA algorithm 69
4.2.2 Pareto-efficiency 70
4.3 Conclusion 71

5 ML-Advised Non-Clairvoyant Scheduling 72
5.1 Introduction 73
5.2 Problem formulation 76
5.3 Preemptive non-clairvoyant scheduling 77
5.4 ML-advised non-clairvoyant scheduling 79
5.5 Empirical evaluation 84
5.5.1 Dataset preparation 84
5.5.2 Algorithm implementation 85
5.5.3 Experimental details 87
5.6 Conclusion 88

6 Conclusion 91

Bibliography 93

List of Figures

1.1 Memory hierarchy of modern computers. 4

1.2 The goal of ML-advised algorithms. Ml-advised algorithms are robust with good average-case performance and bounded worst-case performance. 8

2.1 Normalized disk I/O of the non-adaptive algorithm to the cache-adaptive algorithm as a function of input size for matrix multiplication and sorting for four memory profiles: adversarial (worst-case), benevolent, constant, and oblivious. An increase in the y-axis means that the non-adaptive algorithm performs progressively worse relative to the adaptive algorithm. 17

2.2 Dynamic share of memory of 4 concurrent program instances. The above two plots show the share of memory for the 4 concurrent matrix multiplication program instances, where each instance multiplies two matrices of width 8192 (each matrix takes 256 MiB) and an overall memory of 100 MiB is shared among all 4 concurrent instances. Similarly, the bottom two plots show the share of memory for the 4 concurrent sorting program instances, where each instance sorts an array of 16 GiB, and an overall memory of 64 MiB is shared among all 4 concurrent instances. 18

2.3 MM-INPLACE algorithm: a cache-oblivious cache-adaptive algorithm for matrix multiplication. The diagram assumes that the problem size is N, i.e., each input matrix is $\sqrt{N} \times \sqrt{N}$, memory size is M, and cache-line size is B. 22

2.4 MM-SCAN algorithm: a cache-oblivious non-adaptive algorithm for matrix multiplication. The diagram assumes that the problem size is N, i.e., each input matrix is $\sqrt{N} \times \sqrt{N}$, memory size is M, and cache-line size is B. 23

2.5 External-memory Merge Sort (EMS) algorithm: a cache-aware (non-oblivious) non-adaptive algorithm for external-memory sorting. The diagram assumes that the length of the unsorted array is N, memory size is M, and cache-line size is B. 24
2.6 Lazy Funnel Sort (LFS) algorithm: a cache-oblivious cache-adaptive algorithm for external-memory sorting. The diagram assumes that the length of the unsorted array is N, memory size is M, and cache-line size is B. 25
2.7 The Disk Access Machine (DAM) model of computation. . . . 27
2.8 The ideal cache model of computation. The memory size and block size is unknown to the algorithms. 28
2.9 Cache-adaptive model of computation. It allows the available memory to change at each timestep. 29
2.10 Sample adversarial memory profile generated from running MM-SCAN to multiply two square matrices of width **8192**. The structure of the memory profile mimics the recursive structure of MM-SCAN. 32
2.11 Different categories of memory profiles explored in this study. Primarily, we explore two types of memory profiles: adaptively constructed memory profiles and memory profiles that are oblivious to the algorithms' execution. 33
2.12 Average disk I/O of a non-adaptive program to the cache-adaptive program as a function of problem size when running up to 4 instances concurrently normalized to the same when a single instance runs concurrently. 38
2.13 This diagram shows different ways to generate oblivious memory profiles. For the matrix multiplication experiments, we use $M =$ 10 MiB and for the sorting experiments, we use $M = 256$ MiB. 39
2.14 Average normalized disk I/O of a program instance when up to four program instances run concurrently normalized to the disk I/O when a single instance runs. 40
2.15 Average normalized disk I/O of non-adaptive MM-SCAN to cache-adaptive MM-INPLACE as a function of problem size with 30 MiB memory given. The y-axis indicates that MM-SCAN performs within a constant factor of MM-INPLACE. . 41
2.16 Average normalized disk I/O of non-adaptive External-memory Merge Sort (EMS) to cache-adaptive Lazy Funnel Sort (LFS) as a function of problem size with 768 MiB memory given. An increase in the y-axis indicates that EMS performs progressively worse than LFS. 42

2.17 Average normalized disk I/O of non-adaptive matrix multiplication and sorting algorithms to cache-adaptive matrix multiplication and sorting algorithms as a function of problem size. For matrix multiplication (MM-INPLACE and MM-SCAN), we use a memory of size 30 MiB. For external-memory sorting (EMS and LFS), we use a memory of size 768 MiB. An increase in the y-axis indicates that EMS performs progressively worse than LFS. On the other hand, MM-INPLACE and MM-SCAN does not show a trend that grows with problem size. In each experiment, we run a total of 6 concurrent instances, 3 of cache-adaptive algorithms and 3 of non-adaptive algorithms, both for the experiments for matrix multiplication and sorting. 43
2.18 Normalized disk I/O of the non-adaptive to the cache-adaptive algorithm for variable problem size for oblivious memory workloads that we generated by running TPC-C concurrently with the programs. An increase in the y-axis means that the non-adaptive algorithm performs progressively worse relative to the adaptive algorithm. 44

3.1 The design and approach of ML-advised algorithms. 47
3.2 Robustness vs. Consistency tradeoff curve for the ML advised algorithm with a trust parameter. 58
3.3 Average competitive ratio of the algorithms for (i) varying training epochs and (ii) varying the standard deviation of the prediction error. 64

5.1 Empirical Competitive Ratio (E.C.R.) of the non-clairvoyant algorithms studied in our work as we vary the standard deviation (σ) of the normal distribution in the prediction error. RJF is the best-known online algorithm for non-preemptive non-clairvoyant scheduling without learned advice. The mean of the prediction error is 0 and the standard deviation is varied from 0 to 5000. Note that $\sigma = 0$ denotes that the advice has no error. PSTF gets λ fraction of the processing resource and the rest goes to RJF. 85

Acknowledgments

"Gratitude is not only the greatest of virtues, but the parent of all the others."
- Cicero

Cicero's words deeply resonate with me, as I reflect on the myriad individuals, experiences, and circumstances that have shaped this thesis. It is impossible to acknowledge everyone who has contributed to this journey, as the influences on my life and work are as vast and diverse as the threads of my own story. I am profoundly grateful for the environment in which I was raised, a part of India that cherishes education, critical thinking, reasoning, and philosophy. Growing up in a culture that values inquiry and intellectual exploration has been a privilege that shaped my outlook on life and research. I recognize that any attempt to fully express my gratitude will inevitably fall short, as I am indebted to so many.

First and foremost, I am deeply indebted to my thesis advisor, Prof. Michael A. Bender. I always feel that it is a heavy pair of shoes to fill because being part of the academic lineage that he represents is both an honor and a challenge. It is an honor to extend that academic heritage, but it also carries the weight of expectation of contributing meaningfully to research.

I am equally honored that Prof. Rezaul A. Chowdhury agreed to serve as the chair of my dissertation committee. Dr. Chowdhury gave me advice throughout my doctoral journey; whenever I needed it, he never hesitated to provide me with career advice and technical help in my research. I am fortunate to have been able to co-author a paper with him, which was a truly enriching experience.

I feel honored to have Prof. Joseph S. B. Mitchell on my dissertation committee. His brilliant insights during our algorithms reading group sessions, reading research papers he coauthored, and listening to his talks for years sharpened my skills and fostered a deeper understanding of graph processing systems–skills that proved instrumental during my internship at Berkeley Lab.

I feel extremely fortunate to have Prof. Helen Xu (from Georgia Tech) as the external member of my dissertation committee. Her mentorship, which began when she was a senior doctoral student at MIT, has been a cornerstone

of my academic development. Her invitation to collaborate during her time at Lawrence Berkeley National Lab and hosting me at Berkeley Lab in Fall 2023 introduced me to the fascinating domain of dynamic sparse graphs, a pivotal moment in my academic journey.

Prof. Rathish Das (from the University of Houston) has been a valuable mentor throughout my doctoral journey, starting from when he was a senior doctoral student and I was a newcomer to the graduate program at Stony Brook and the lab. Collaborating with him on emerging fields such as "Machine Learning Advice" and engaging in discussions on algorithmic bounds were intellectually invigorating experiences.

Throughout my life, I have had many teachers who inspired me to dive deep into various subjects. I was often motivated by the desire to explain difficult topics in a way that would be accessible to different audiences. In fact, my dissertation advisor, Dr. Bender, played a key role in that. When I first joined the Ph.D. program, I took a graduate course on algorithms and data structures. Dr. Bender has a knack for explaining advanced concepts in an accessible way that students can easily internalize and leverage the knowledge in research and interview settings. I have adopted his approach to explaining complex algorithms in my own presentations, which has helped me communicate my research more effectively. One of the most rewarding experiences during my Ph.D. was participating in the High School Women in Science and Engineering (HS-WISE) program, where I taught high school seniors in Long Island. I introduced them to concepts of probability and the fundamentals of computer science. At the end of my time there, I received a wonderful recommendation letter from the WISE program director, Jacquelyn Harper.

I am forever indebted to many friends I found at Stony Brook. I found a great friend in Dr. Alëna Aksënova who helped me a lot throughout thick and thin, despite being extremely busy herself at her job at Google. She is the one who introduced me to Google foobar and gave me an amazing referral to Google. She even connected me with several of her professional contacts, providing glowing introductions. I also met some amazing people like Shayan Sarkar, Amartya Pal, Anushka Ojha, and Souvik Paul, all of whom are soon-to-be doctors. I could not have survived the journey and reached the finish line unless they would have opened their doors (literally) for me whenever I needed it the most. Dr. Pratik Dutta and Saheli Datta have been encouraging me constantly and relentlessly that gives me enough confidence in what I do. My first housemates in the United States, Rohit Aich, Jay Gite, and Rishikesh Gokhale, welcomed me warmly, making my transition to a new country smoother. I had great fun by taking part in events like Hackathons, where I got to do some cool piece of engineering in a couple of days. I won

some of them; thanks to my partner-in-crime Dr. Anushka Banerjee. I am thankful to Dr. Nisha Verma and Dr. Atreyo Mukherjee for supporting me throughout my job search phase.

The most important teacher life has gifted me is my mother, Manashi Bhattacharya. She made me who I am, and I owe much of my values and outlook on life to her. Her early teachings in mathematics laid the foundation for my academic pursuits and remain the bedrock of my achievements. Her influence is one of the main reasons I decided to pursue a Ph.D. and return to academia. For her unconditional love and unwavering belief in me, I dedicate this thesis to my mother. Additionally, I have been gifted with two great in-laws, Sugopa Mukherjee and Rabi Sankar Mukherjee, who somehow crossed the barrier of law and became my parents. They supported me through all ups and downs; I can always talk about deeper understanding of life with them and borrow wise words to find motivation from the sea of literature they ingurgitated.

Finally, I could not have completed this Ph.D. without the continuous support of my amazing wife, Upasana Mukherjee, who is also a soon-to-be doctor and whose resilience and support sustained me through the most challenging phases of my Ph.D., including the years of the COVID-19 pandemic. Her talent, dedication, and hard work in her own field motivated me throughout this challenging journey. I am constantly inspired by her passion for uncovering answers to the unanswered questions in her field.

P.S. As I defended my thesis and took a position at Google already at the time I am submitting this dissertation, I cannot but mention that I have found a home in a team of outstanding individuals. My manager, Carl Williams, has had a remarkable career with diverse experience in technology. My team members, James Mouradian, Jacob Wells, Tucker Wilde, and Dr. Jinghao Zhao, are fantastic engineers who are incredibly supportive and fun to work with.

Hence, let the journey begin. As the Katha Upanishad and the Bhagavad Gita says, "na hi jnanena sadrusham pavitram iha vidyate." The potential translation is as follows: "indeed, there is nothing as purifying as knowledge in this world."

Chapter 1

Introduction

This thesis addresses the development of efficient software solutions for modern computing systems. It provides software application developers with a comprehensive understanding of software performance in contemporary resource-shared computing environments. Additionally, this thesis advocates for software design principles that foster empirically robust performance. Furthermore, it elucidates the distinctions between traditional computing algorithm design principles and machine learning algorithms, aiming to bridge the gap between advancements in both fields by leveraging their combined strengths.

1.1 Tools to analyze algorithmic performance

In this section, we describe some theoretical and empirical tools to analyze algorithmic performance. The thesis will focus on a subset of these tools; we will discuss them in the later sections on how some of these tools come short of providing a holistic view of algorithmic performance and how software developers can circumnavigate the above challenge.

Traditional competitive analysis and its disadvantages. Many computing problems we face in real life can be solved with a class of solutions named ***online algorithms*** [1, 2], i.e., an algorithm that processes its input piece-by-piece in a sequential manner, without having access to future input elements. One of the traditional ways to study the performance of an online algorithm is classical ***competitive analysis*** [1–4], also known as ***worst-case analysis***. Worst-case analysis of an online algorithm evaluates the performance of the algorithm under the most unfavorable conditions over all possible inputs. This approach measures how the algorithm performs relative to the optimal offline algorithm, which has complete knowledge of future events.

The metric used is the ***competitive ratio***, which is the worst-case ratio of the performance of the online algorithm to the performance of the optimal offline algorithm.

Competitive analysis can sometimes result in unrealistically high competitive ratios for popular online algorithms [5, 6]. This analysis typically focuses on the worst-case scenarios, which may not accurately reflect the algorithm's performance in more typical, real-world situations. As a result, online algorithms that are optimized for worst-case inputs can perform poorly on "benevolent" inputs, where the conditions are more favorable. A prime example of this issue is the Least Recently Used (LRU) algorithm for competitive caching [1, 2]. Despite its theoretical shortcomings, it is the gold standard in practice that LRU performs well in practice for many common access patterns [7, 8], but competitive analysis might not explain this efficiency adequately. The gap between theoretical worst-case performance and empirical average-case performance often leaves algorithm designers unable to justify why some popular algorithms, like LRU, work so well in practice [7, 9]. Moreover, the pessimistic perspective inherent in worst-case performance analysis can lead to unnecessarily complex algorithm designs [1] (e.g., randomized marker algorithm for competitive caching [10, 11]). These designs aim to achieve a better competitive ratio, addressing the worst-case scenarios but potentially sacrificing simplicity and practical efficiency.

Algorithm designers found several ways that help them see beyond the worst-case performance bounds of an algorithm. For example, beyond-worst-case analysis is a family of techniques to study algorithmic performance beyond the worst-case inputs providing software developers with tools to gain a more nuanced understanding. Furthermore, software developers often rely on empirical studies to assess the performance of a system or software in practical scenarios. In the following sections, we will briefly discuss both beyond-worst-case analysis and empirical performance evaluation.

Beyond-worst-case analysis. Beyond-worst-case analysis [5, 8, 12, 13] of online algorithms seeks to provide a more nuanced understanding of algorithm performance by considering scenarios that are not captured by traditional worst-case analysis. This approach looks at performance under more realistic conditions, such as average-case scenarios, specific distributions of inputs, or certain structural properties of the input sequence.

For example, average-case analysis [5, 8] examines the expected performance of an algorithm under a probabilistic model of input. Instead of considering the worst possible input, it assumes inputs are drawn from a known distribution. On the other hand, smoothed analysis [12–14] blends worst-case

and average-case analysis by considering slightly perturbed inputs. In the case of smoothed analysis, the algorithm's performance is studied under all possible inputs, however, an adversary called *Nature* can perturb the input, which mimics the arbitrary noise in a realistic system environment where the algorithm will run in reality. This helps to understand how small changes in the input can affect performance.

Empirical performance analysis. The empirical performance study of an algorithm involves evaluating its behavior and effectiveness through practical experiments and observations rather than purely theoretical analysis. This approach typically includes running the algorithm on a variety of test cases and input datasets, measuring key performance metrics such as execution time, memory usage, accuracy, and robustness. The goal is to understand how the algorithm performs in real-world scenarios and identify any practical limitations or advantages. For example, to study software solutions for sorting and searching, Bentley and McIlroy [15] performed a thorough empirical analysis. Another important example of the competitive paging problem, theoretical worst-case analysis showed that Least Recently Used (LRU) and First-In-First-Out (FIFO) are expected to have similar performance [4]. However, empirical analyses showed (and practitioners follow this as a gold standard) that LRU performs empirically better than FIFO [7, 9].

1.2 Architecture of modern computing environments

In this section, we will discuss another important aspect that software developers require to keep in mind in order to write efficient software solutions for large computing problems: fundamentals of the architecture of the modern computing environments (such as memory and where data resides) where the application is supposed to be used. Additionally, we will discuss another commonly used technique in modern-day programming, parallelism, which is crucial for optimizing performance in high-performance computing environments.

Hierarchical memory in modern computers. Most modern computers, be it a desktop, laptop, or a supercomputer, has a hierarchical memory structure. [1] It consists of multiple levels of memory—registers, followed by L1,

[1] Some specialized hardware have mostly flat memory; they do not have a traditional hierarchical structure of memory. For example, some Field-Programmable Gate Arrays (FP-

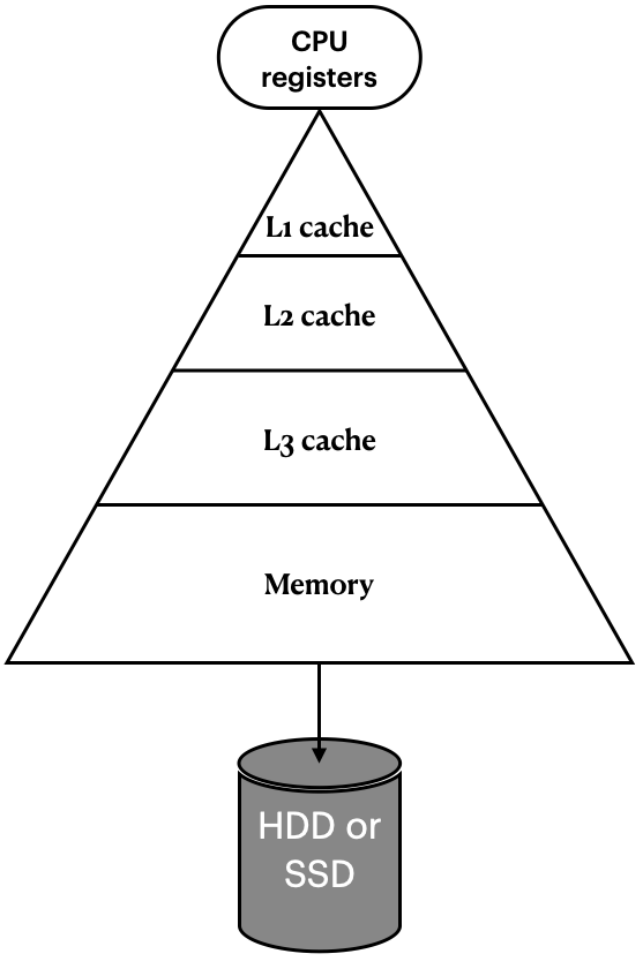


Figure 1.1: Memory hierarchy of modern computers.

L2, and L3 caches, and then RAM and disk or solid-state drives (SSD). They all vary in latency and size. The idea is to put the memory with the lowest latency closer to the CPU to feed the CPU at a faster rate. As the memory becomes further than the processor (in the hierarchy), the size of the memory increases, e.g., the disk is much larger than the RAM. Figure 1.1 shows the memory hierarchy of a typical modern computer.

Small and fast memory (RAM) is usually a scarce resource in a computer, and it is shared among many processes that run concurrently on a computing platform. Efficiently managing it is crucial for achieving high performance for

GAs), Digital Signal Processors (DSPs), Application-Specific Integrated Circuits (ASICs), and some embedded systems have mostly flat memory and they are used for custom computing tasks.

individual performance of the concurrent processes and also for overall high performance. Data is served to the CPU from the small, fast memory. Since the small memory often cannot fit all the data required by a processor, data is transferred between the small and the large memories. Often the time spent on data transfers between the memory levels dominates the running time of programs [16]. Hence, optimizing data transfers and efficiently managing the fast, small memory play a vital role in getting high performance in most large applications.

Parallel computing. In recent times, there has been a surge in the use of multiprocessor architectures and ***parallel computing*** refers to the process of breaking down larger problems into smaller parts that can be executed simultaneously by multiple processors [17]. The reason to move towards a multiprocessor architecture is primarily attributed to the fact that we have reached a certain point in terms of hardware design, so that we have almost achieved the highest clock speed possible for a single processor, as suggested by the Moore's law [18, 19]. Hence, to maintain steady growth in processing power, hardware vendors have taken a strategy–using multiple processors instead of a single processor to share the same workload. This new technique has placed parallel computation at the forefront of high-performance computing. At the same time, multiprocessor systems bring new challenges that are not seen when memory is used by a single processor only. For example, sharing the small and fast memory among multiple processors presents many challenges. One of these challenges for sharing the small and fast memory among multiple processors is that the available resources to each individual process that runs concurrently in computing environments may fluctuate heavily with time [20]. Bender et al. [21, 22] proposed a theoretical model to study the problem of resource (such as memory or cache) formally as the ***cache-adaptive model***. This thesis studies the cache adaptivity problem and finds practical solutions to this prevalent problem in computing.

1.3 Thesis contribution

This thesis presents a comprehensive study of two critical areas in algorithm design. First, we show how to design efficient software solutions for multi-threaded, multi-process, resource-shared computing environments. We show how to structure programs so that they can share memory (or cache) efficiently. In order to discuss the above problem, we design an empirical framework that studies the brittle nature of the previously found worst-case performance bounds of a large class of algorithms and bridges the gap between

theory and practice. Thereafter, we study another algorithm design framework, machine learning advice. We will show how this framework enables software developers to study algorithmic performance beyond the worst case. Through this study, this thesis will show that worst-case performance metrics and bounds may not be enough for an application developer to get a holistic view of the performance of an algorithm. This will also advocate two potential ways for algorithm designers, application developers, and system engineers to use theoretical performance bounds and go beyond them by having additional performance insights.

1.3.1 Cache-adaptive algorithms design

Fluctuations of available resources that are shared among concurrent processes and threads can be a performance bottleneck in a computing environment. The ability to adapt to the fluctuations of available resources such as cache and RAM is an extremely sought-after quality for modern multi-threaded, multi-core, time-sharing computing environments.

Systems engineers and database researchers came up with empirical solutions for cache adaptivity, and came up with sorting and joining solutions for database queries [23–26]. Researchers have also explored environment-level modifications to adapt to dynamic memory fluctuations [27–29]. The empirical approach lacked performance guarantees for adversarial memory fluctuations. On the other hand, Barve and Vitter [30, 31] designed algorithms that have provable performance guarantees even when the memory size changes. More recently, Bender et al. [21, 22] developed a theoretical framework to analyze algorithms under memory fluctuations, i.e. the ***cache-adaptive model***. The theoretical research showed when a class of algorithms may be optimally cache adaptive under any memory fluctuations. However, the theoretical finding remained inconclusive because the theory also shows there might be an asymptotic performance gap even within the same class of algorithms under an adversarial memory fluctuation, where the cache-adaptive algorithms belong.

For the cache adaptivity problem, we show that the theoretical insights and bounds of cache-adaptive algorithm design may not be sufficient to provide a holistic view of the developers. As today's application developers write programs and applications that are likely to run on a shared-resource environments, adapting to fluctuating resources (such as memory) is a highly valued property in an application. We highlight the need for a structured approach for an extensive experimental study in the form of an empirical framework, the advantage of building one, and how the observations made from experiments can help provide application developers with a practical recipe for algorithm

design.

Theoretical findings on cache adaptivity. Applications running on multi-threaded and multi-core systems often experience fluctuations in the amount of physical memory available. These memory fluctuations can cause an application's performance to suffer [27, 28]. Past theoretical work shows that optimal cache-oblivious algorithms with a specific recursive structure (also known as ***(a, b, c)-regular algorithms***) have a good potential to be cache-adaptive, however, they can perform poorly on adversarially constructed memory fluctuations [21, 22]. Bender et al.'s work [21, 22] showed a specific memory fluctuation under which some (a, b, c)-regular algorithms perform suboptimally, whereas some can adapt to those fluctuations. The above theoretical result invited a holistic evaluation of the cache adaptivity of algorithms with an empirical framework.

Empirical analyses of cache adaptivity. In Section 2.4, we develop an empirical framework to study cache adaptivity for algorithms that solve fundamental computing problems such as matrix multiplication and sorting. Using this framework, we show that (a, b, c)-regular algorithms achieve the practical benefits of cache adaptivity. We also show that M/B-way external-memory merge sort, a cache-aware algorithm, performs suboptimally under a varied range of memory fluctuations. In order to generate fine-grained memory fluctuations, we designed a ***memory profile generator*** that can simulate any adversarial memory fluctuation even by looking into a program's execution runtime.

In Section 2.5, Section 2.6, and Section 2.7, we show an evaluation of matrix multiplication and external-memory sorting algorithms. MM-SCAN and MM-INPLACE are both theoretically optimal algorithms for cubic-time matrix multiplication when the memory does not fluctuate; empirically, Figure 2.1a shows that MM-INPLACE performs roughly 1.8× better than MM-SCAN across all problem sizes under a ***constant memory profile*** (fixed memory size). We used the memory profile generator to create profiles coupled with algorithm execution. Under the adversarial memory workload, MM-SCAN performs 1.6 – 3.8× more disk I/Os than MM-INPLACE. Critically, the performance gap grows with the input size. We also created a ***benevolent memory profile*** that is tightly synchronized with the algorithm's execution but non-adversarial in nature. Under the benevolent memory profile, MM-SCAN and MM-INPLACE perform roughly within 1.5× of each other for all problem sizes. Under ***oblivious memory profiles***, i.e., profiles that are not tightly coupled with the algorithms' execution, we found that MM-SCAN incurs about

1.8× more I/Os than MM-INPLACE for all problem sizes. These results suggest that MM-SCAN possesses almost all the benefits of cache-adaptivity.

Also, in Section 2.5, Section 2.6, and Section 2.7, we evaluate the cache-oblivious Lazy Funnel Sort (LFS) [32, 33] and the non-oblivious External-memory Merge Sort (EMS) [16] (both are I/O-optimal when the memory is fixed), and we find that even though LFS has a computational overhead over EMS stemming from its cache-obliviousness; it outperforms EMS on a wide range of fluctuating memory profiles. Figure 2.1b suggests that External-memory Merge Sort has better I/O performance than Lazy Funnel Sort for all problem sizes under the constant memory profile (roughly incurring 0.9× disk I/Os on average). On the other hand, LFS adapts when the memory fluctuates, whereas EMS fails to adapt. EMS performs progressively worse than LFS as the input size increases under all types of fluctuating memory workloads (adversarial, benevolent, and oblivious), incurring 1.3 − 7× more disk I/Os. Despite the computational overhead of cache obliviousness, LFS performs significantly better than EMS under a range of fluctuating memory profiles.

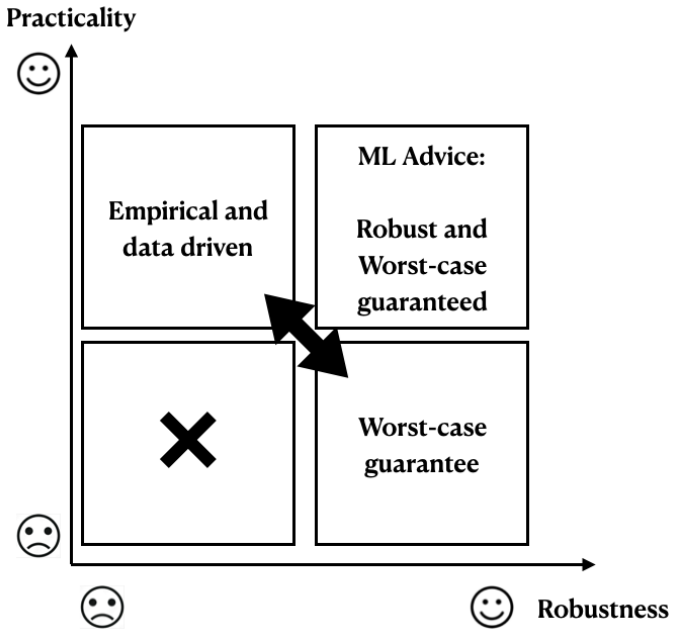


Figure 1.2: The goal of ML-advised algorithms. Ml-advised algorithms are robust with good average-case performance and bounded worst-case performance.

1.3.2 ML-advised algorithm design

This thesis discusses another approach to look beyond the worst-case performance bounds for software application developers, namely the ML advice framework. The ***ML-augmented algorithms***, also known as ***ML-advised algorithms***, ***learning-augmented algorithms***, or ***algorithms with predictions***, follow an algorithm design principle that aims to bridge the gap between traditional online algorithms and recent advances in machine learning.

Traditional online algorithms [1, 2] are designed to optimize the worst-case performance ratio by comparing their performance with that of the offline optimal algorithm. This focus on worst-case performance ensures that the algorithm performs reasonably well even under the most unfavorable conditions. However, this emphasis on worst-case scenarios can sometimes lead to suboptimal performance in average-case situations. Machine learning algorithms, on the other hand, are typically designed to perform well on average-case inputs by learning patterns and making predictions based on data. While this can lead to excellent performance in typical scenarios, ML algorithms often lack performance guarantees in worst-case scenarios. This is because they are trained on specific datasets and might not generalize well to all possible inputs, particularly adversarial ones. ML-augmented algorithms [34, 35] offer a promising middle ground. These algorithms combine the strengths of traditional online algorithms and machine learning techniques. By integrating ML components into the decision-making process, ML-augmented algorithms can achieve good average-case performance while still maintaining bounded worst-case performance guarantees. This hybrid approach leverages the predictive power of machine learning to enhance performance in typical scenarios, without sacrificing the robustness provided by worst-case analysis.

Machine learning advice. Machine Learning (ML) advice [34–37] has emerged as a promising framework for enhancing the performance of traditional online algorithms. Over the past six years, significant research efforts have been dedicated to integrating ML techniques with classical online algorithm strategies to achieve better performance.

Traditional online algorithms operate under the assumption that they have no prior knowledge of the input sequence. These algorithms are designed to minimize their competitive ratio, which measures their performance relative to an optimal offline algorithm that has complete foresight. By focusing on worst-case scenarios, traditional online algorithms often adopt overly cautious strategies to handle potentially "bad" inputs [6], where their performance could significantly lag behind the optimal offline solution. This caution, however, leads to a higher competitive ratio for "easy" inputs. Consequently, in

simulations with realistic input sequences, traditional online algorithms might perform worse on average compared to ML algorithms that can leverage historical data and patterns. ML algorithms, in contrast, excel at learning from past data and predicting future patterns. They are particularly effective in handling average-case inputs where the data follows discernible trends. Despite their strength in prediction, ML algorithms lack performance guarantees in worst-case scenarios, as their training may not encompass all possible input variations, particularly adversarial ones.

To harness the strengths of both traditional online algorithms and ML algorithms, researchers [34, 36] used ML advice that augments the traditional online algorithms; the ML-advised algorithms [34–39] achieve better average-case performance while retaining bounded worst-case performance. This class of algorithms has three desirable properties. The algorithms need to be ***consistent***: when the predictions are of good quality, i.e., the predicted metric is close to (or equal to) the actual metric, the ML-advised algorithms should perform close to the optimal offline algorithm. The second desired property is ***robustness***: when the prediction quality is arbitrarily poor, the ML-advised algorithm should not have an unbounded performance ratio relative to the optimal offline algorithm. Finally, ML-advised algorithms should have a ***smoothness*** property, i.e., as the prediction quality degrades gradually, the performance should degrade gracefully relative to the optimal offline algorithm.

Source of predictions. Do the learning-augmented algorithms need to be augmented with an advisor that is trained using machine learning techniques? The straightforward answer is no. Technically, predictions can originate from various sources; the source of predictions does not have to be a trained machine learning oracle. For instance, predictors or advisors might utilize simpler, heuristics-based approaches. This is why ML-augmented algorithms are also referred to as algorithms with predictions [6].

Computational overhead from ML training. ML-advised algorithms utilize predictions from ML models to enhance average-case performance while maintaining robust worst-case guarantees [34, 35, 37]. However, the computational overhead of ML training, which is significant for generating high-quality advice, must be considered. The advisor's complexity should be constant time, $O(1)$, for theoretical performance bounds to hold. If training and generating advice exceed this complexity, the practical benefits and theoretical bounds may be compromised.

This thesis explores the integration of machine learning (ML) advice into

online algorithms as a tool to achieve a significant advancement in algorithm design. By combining predictive insights from ML with the robust framework of traditional online algorithms, ML-advised algorithms can achieve a balanced performance profile, excelling in both average-case and worst-case scenarios. This hybrid approach has the potential to significantly improve the practical utility and efficiency of online algorithms in real-world applications.

Bounds for ML-advised online algorithm. In Section 3.4 and Section 3.5, we extend the conventional online algorithms for rent-or-buy applications by designing algorithms with ML advice to solve an online problem, the ski rental problem with a discount. This problem serves as a relaxed version of the traditional ski rental problem, a well-known example of the rent-or-buy dilemma.

In the problem formulation of the ski rental problem with a discount, there is a purchasing price for the ski and an actual rental price of the ski. However, there is an effective rental price that is strictly less than the actual rental price, moreover, it can fluctuate arbitrarily. The practical motivation is explained in Section 3.3. The skier must make an irrevocable decision to either rent or buy the skis at the start of each day, without knowing the total number of skiing days x until the season ends. The objective is to minimize the total cost incurred by making the optimal decision to purchase the skis in an online manner.

We first design an ML-advised algorithm to solve the ski rental problem with a discount. The algorithm does not blindly follow the ML advice; instead, it has a tuning hyperparameter to adjust the trust in the ML advice and achieve a better performance bound. First, we show in Lemma 1 that if the ML-advised algorithm blindly follows the ML oracle, the competitive ratio of the algorithm is unbounded. However, instead of following the oracle blindly, if the algorithm has a trust parameter $\lambda \in (0, 1]$, we show in Lemma 2 that the algorithm has an improved competitive ratio of $(1 + 1/\lambda)$ for any given prediction error. It reaches a competitive ratio $(1 + \lambda)$, when there is no prediction error.

Finally, we extend the problem to another interesting direction where more than one predictors are present in the ecosystem and the ML-augmented algorithm has to decide which advice to rely on. In particular, we study the situation where two independent ML advisers are present. If one of the advisers is correct, we show in Lemma 3 that the algorithm is ϕ-competitive, where ϕ is the golden ratio (see Section 3.5). Lemma 4 shows that otherwise, the competitive ratio of the algorithm is $\max\{\phi + 1, 1/(\phi - 1 - r\eta/p)\}$; η is the prediction error, r is the actual rental price, and p is the purchasing price.

The above two approaches can help application developers go beyond what theoretical studies and bounds may offer, providing more practical and effective algorithm design solutions.

Design of the advice. In Chapter 4, we show how a particular futuristic advice is not enough to find any performance advantage even after designing an ML-advised algorithm. For the used case, we choose a popular problem in computing: the list update problem.

For the list update problem, the input is an online sequence of access requests to items in a linked list. To serve a request to an item at position i, an online algorithm makes a linear scan of the list until it finds the requested item. The above linear scan has an access cost of i. After accessing the item, the algorithm is allowed to move the item to a position closer to the front using a free exchange without incurring any cost. Furthermore, the list update algorithm is allowed to make any number of paid exchanges, i.e., swapping two consecutive items at a unit cost.

We show that even correct predictions about the next request for each request cannot ensure optimality for an ML-advised algorithm. In other words, the advice is functionally useless for the purpose of the ML-advised algorithm. Next, we show that having a piece of advice with the next two requests for each request might be useful for designing an ML-advised list update algorithm. We propose an algorithm in Section 4.2, namely the Penalty Distance Algorithm (PDA), which has access to predictions concerning the next two occurrences of the requested item in the request sequence. We introduce a conjecture that the performance of the algorithm can be very close to the best-known online algorithm for the list update problem.

Chapter 2

Cache Adaptivity

This chapter presents a study on cache adaptivity, a prevalent problem in modern software solutions and a highly necessary quality in software solutions written for contemporary resource-shared computing environments.

This work was conducted in collaboration with Dr. Helen Xu, Dr. Rathish Das, Abiyaz Chowdhury, Dr. Rezaul A. Chowdhury, Dr. Rob Johnson, Dr. Rishab Nithyanand, and Dr. Michael A. Bender [40].

Abstract. Memory fluctuations are the norm in most modern computing systems, significantly affecting program performance. While many database algorithms perform well empirically when the amount of available memory changes dynamically, they often lack formal performance guarantees. Theoretical research demonstrates the design of cache-adaptive algorithms that passively adapt to all types of memory fluctuations without requiring any change in the algorithm based on the environment parameters, and hence cache-optimal when memory size fluctuates. Bender et al. highlighted that many—though not all—are optimally cache adaptive, suggesting that cache-obliviousness is a good place to start when designing cache-adaptive algorithms. They compared two cubic-time cache-oblivious algorithms for matrix multiplication, which we refer to as MM-SCAN and MM-INPLACE, both are cache-optimal in the disk-access machine model. The algorithms diverge in the face of memory fluctuations: MM-INPLACE is cache adaptive under all types of memory fluctuations, while MM-SCAN is not cache adaptive and exhibits a suboptimal performance under certain adversarially generated memory fluctuations. This chapter investigates whether this performance disparity persists under non-adversarial memory fluctuations.

In this chapter, we empirically evaluate cache-friendly algorithms under various memory fluctuation scenarios, demonstrating that cache-obliviousness

serves as a strong proxy for cache adaptivity under non-adversarially-generated memory conditions. In particular, we experimentally evaluate the performance gap between cache-adaptive and non-adaptive algorithms for matrix multiplication and external-memory sorting under memory workloads that are non-adaptive in the sense that they remain oblivious to the execution of the program. We generate such memory workloads by running each algorithm concurrently with other programs. Our findings reveal that the performance gap between MM-SCAN and MM-INPLACE does not scale with input size; MM-SCAN incurs up to twice the disk I/Os of MM-INPLACE under oblivious memory workloads, indicating that the worst-case theoretical performance gap is not universally observed.

Additionally, we compare External-memory Merge Sort (EMS), which is neither cache-adaptive nor cache-oblivious, with Lazy Funnel Sort (LFS), which possesses both cache-adaptivity and cache-obliviousness properties, under oblivious memory workloads. The relative performance of EMS degrades as the problem size increases, incurring up to 6× the disk I/Os compared to LFS on large inputs. However, LFS incurs no significant computational overheads for having cache-obliviousness when the memory profile is not erratic: EMS incurs within 1.7× number of disk I/Os of LFS when the available memory does not fluctuate. These results suggest that cache-oblivious algorithms are likely to capture almost all of the benefits of cache-adaptive algorithms under a variety of memory fluctuations that do not adapt to the execution of the algorithm.

2.1 Introduction

Applications running on multi-threaded and multi-core systems often experience fluctuations in the amount of physical memory available. For example, when concurrently running programs share RAM, the amount of RAM allocated to any particular program can change as other programs start or finish. Interestingly, even when identical copies of the same program run concurrently, these copies may acquire an unequal fraction of the RAM [20], resulting in further unpredictability.

These memory fluctuations can cause an application's performance to suffer [27, 28]. If the available memory becomes scarce, an application may ***thrash***, which means that the application pages excessively, crippling performance [41–43]. If memory becomes abundant, the program may not take advantage of the larger available RAM and thus perform extraneous I/Os.

In this paper, we empirically show that some optimal cache-oblivious algorithms [33, 44, 45]—specifically matrix multiplication and sorting—adapt

gracefully to most memory fluctuations, except for some adversarially constructed worst-case instances. Optimal ***cache-oblivious*** algorithms are asymptotically optimal for all fixed memory sizes without the knowledge of the memory size. Past theoretical work shows that optimal cache-oblivious algorithms can perform poorly on adversarially constructed memory fluctuations [21, 22].

Experimental approaches to adaptivity. Past experimental work [23–26] focused on developing improved algorithms that adapt to a variety of memory profiles, where a ***memory profile*** is a curve representing the memory size as a function of time. For example, Pang et al. [23] introduced a memory-adaptive merge sort that dynamically adjusts the size of each subproblem at each level of the recursion as memory fluctuates. A similar technique is used by Pang et al. [24] to modify the GRACE hash join algorithm [46] so that it can adapt to changes in memory size. Zhang and Larson [25, 26] introduced techniques that balance the memory usage among many sorting programs running concurrently on a single machine sharing its memory.

Researchers have also explored environment-level modifications to adapt to dynamic memory profiles. For example, Brown et al. [47] introduced memory-management techniques for a DBMS that accommodate multiple concurrently running database workloads, where each workload may have its own individual memory requirements. Mills et al. [27–29] proposed a user interface that enables a user to request sufficient memory for their application.

Theoretical approaches to adaptivity. Barve and Vitter [30, 31] designed algorithms that have provable performance guarantees even when the memory size changes. They gave memory-adaptive algorithms for sorting, matrix multiplication, FFT, LU decomposition, and permutation. To prove optimality, they extended the traditional ***external-memory model*** [16]—also called the ***disk-access machine (DAM)***—to allow for changes in the memory size over time.

More recently, Bender et al. [21] showed that many cache-oblivious algorithms could adapt well to memory fluctuations. A ***cache-oblivious algorithm*** is a platform-independent algorithm that is not parameterized by properties of the memory hierarchy such as RAM or cache-line size [33, 44, 45]. An optimal cache-oblivious algorithm is universal in the sense that the algorithm is optimal for any value of the cache parameters, provided that these values do not change over time. (Past theoretical work on cache-oblivious algorithms use the terms memory, cache, and RAM interchangeably) [48, 49]. On the other hand, a cache-adaptive algorithm remains cache-optimal even when memory size changes over time. One possible way to deal with memory fluctuations is

to start with an algorithm already known to be cache-oblivious and hope it also happens to be cache adaptive.

Cache-obliviousness and cache-adaptivity. Bender et al. [21] showed that cache-oblivious algorithms are often (but not always) cache adaptive. For example, they showed that Lazy Funnel Sort (LFS) [32, 33], a cache-oblivious sorting algorithm, is optimally cache-adaptive. Follow-up work [22] provided an analytical framework for determining whether cache-oblivious algorithms having a certain recursive form ("(a, b, c)-regular") are also cache adaptive. Algorithms with (a, b, c)-regular recursive structure have a common form of divide-and-conquer cache-oblivious design [22, 50]; see Section 2.3 for more details. Depending on the settings of the parameters a, b, and c, these cache-oblivious algorithms are either asymptotically optimal or a logarithmic factor away from optimal. For example, cache-oblivious algorithms exist for matrix multiplication [33, 44], some of which are optimally cache-adaptive (MM-INPLACE) and some are a log factor away from optimally cache-adaptive (MM-SCAN) [21, 22]. MM-SCAN ($a = 8$, $b = 4$, and $c = 1$) performs an out-of-place matrix addition at the end of each recursive call and is suboptimal in the cache-adaptive model, while MM-INPLACE ($a = 8$, $b = 4$, and $c < 1$) performs the additions in-place and is optimally cache-adaptive; see Section 2.4.

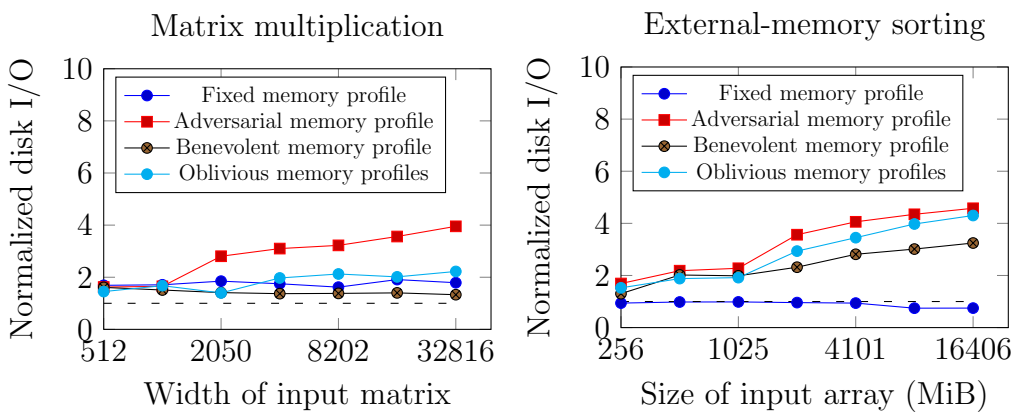
Connections between experimental and theoretical approaches. Existing theoretical approaches to cache adaptivity provide worst-case guarantees but leave open the question of how algorithms perform under arbitrary memory workloads. Specifically, past work showed that cache-oblivious (a, b, c)-regular algorithms are always at most a log factor away from being optimally cache-adaptive. However, this log factor is based on a worst-case memory workload that seems brittle and unlikely to appear in practice. Furthermore, this worst-case analysis does not capture how cache-oblivious algorithms actually perform under practical memory workloads.

In contrast, empirical solutions [23–29, 46, 47] are designed to improve performance under practical memory workloads. However, they lack worst-case analysis and therefore do not have worst-case performance guarantees.

Recent work [50] has gone beyond worst-case analysis. The authors apply smoothing techniques [13] on memory profiles and explore how well cache-oblivious algorithms adapt to smoothed memory profiles. They observe that the I/O-performance gap between cache-obliviousness and cache-adaptivity disappears given sufficient smoothing. Their results leave open the question of whether the performance gap between cache-oblivious and cache-adaptive

algorithms persists across a larger space of memory profiles. If the performance gap turns out to be only a theoretical artifact rarely seen in practice, cache-oblivious algorithms could be an exciting way to solve a wide range of problems with no known cache-adaptive solutions. Examples of such problems include Gaussian elimination [51], triangle counting [52], min-weight cycle [53], negative triangle detection and counting [53], replacement paths problem [53], etc.

Do these worst-case memory workloads arise naturally when no adversary controls the available memory? How feasible is it to construct such worst-case memory profiles?



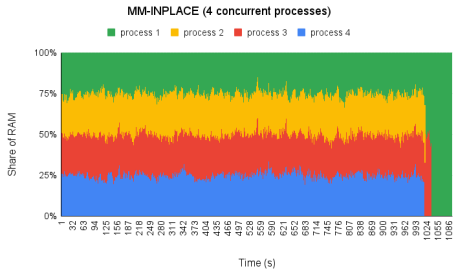
(a) Normalized disk I/O of MM-SCAN to MM-INPLACE

(b) Normalized disk I/O of External-memory Merge Sort to Lazy Funnel Sort

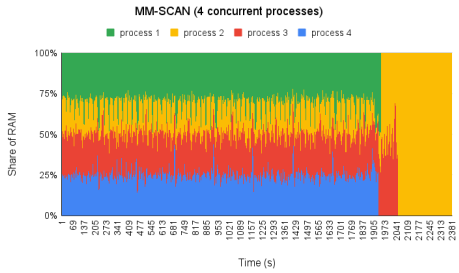
Figure 2.1: Normalized disk I/O of the non-adaptive algorithm to the cache-adaptive algorithm as a function of input size for matrix multiplication and sorting for four memory profiles: adversarial (worst-case), benevolent, constant, and oblivious. An increase in the y-axis means that the non-adaptive algorithm performs progressively worse relative to the adaptive algorithm.

Results

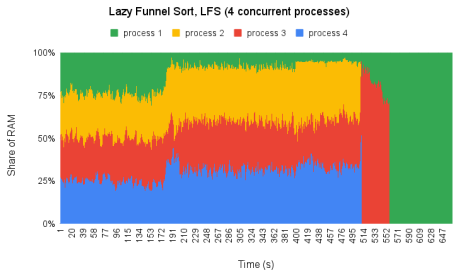
We empirically study how cubic-time matrix multiplication (MM) algorithms and external-memory sorting algorithms adapt to a range of different memory profiles. MM is a particularly interesting test case because not only is it the canonical example of an (a, b, c)-regular algorithm but also we can find a pair of MM algorithms, both with the same I/O-complexity in the cache-oblivious model, where one is optimally cache-adaptive and the other is a logarithmic factor away from optimal. For the case of MM, we evaluate MM-INPLACE



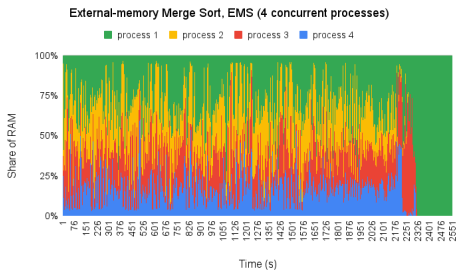
(a) 4 concurrent instances of MM-INPLACE



(b) 4 concurrent instances of MM-SCAN



(c) 4 concurrent instances of Lazy Funnel Sort (LFS)



(d) 4 concurrent instances of External-memory Merge Sort

Figure 2.2: Dynamic share of memory of 4 concurrent program instances. The above two plots show the share of memory for the 4 concurrent matrix multiplication program instances, where each instance multiplies two matrices of width 8192 (each matrix takes 256 MiB) and an overall memory of 100 MiB is shared among all 4 concurrent instances. Similarly, the bottom two plots show the share of memory for the 4 concurrent sorting program instances, where each instance sorts an array of 16 GiB, and an overall memory of 64 MiB is shared among all 4 concurrent instances.

Algorithm	*Cache-oblivious*	*Cache-adaptive*
MM-SCAN [33, 44]	✓	✗
MM-INPLACE [33, 44]	✓	✓
EM Merge Sort [16]	✗	✗
Lazy Funnel Sort [32]	✓	✓

Table 2.1: Properties of the algorithms for cubic-time matrix multiplication and external-memory sorting studied in this paper.

and MM-SCAN and find that both can adapt to a wide range of memory profiles even though MM-SCAN is not provably cache-adaptive. In contrast, for the case of sorting, we evaluate LFS [32, 33] and External-memory Merge Sort (EMS) [16], a non-oblivious and non-adaptive algorithm, and find that EMS does not adapt well to a wide range of memory fluctuations. Table 2.1 illustrates some of the key properties of the tested algorithms.

Since the worst-case profile is tightly coupled with an algorithm's structure, we designed a ***memory profile generator*** that can simulate an adversarial by looking into a program's execution. It can generate the worst-case ***adversarial memory profile*** by increasing the available memory when the program cannot benefit from the extra memory and decreasing the memory when the program would benefit. It can also generate non-adversarial profiles that follow a program's execution.

First, we empirically study the MM-SCAN and MM-INPLACE algorithms for matrix multiplication, and we find that even though MM-SCAN is not optimally cache-adaptive, it adapts well to a wide range of memory fluctuations except the most adversarially constructed profiles. We measure how well an algorithm adapts to the memory fluctuations by measuring the disk I/Os it incurs during its execution. MM-SCAN and MM-INPLACE are both theoretically optimal when the memory does not fluctuate; empirically, Figure 2.1a shows that MM-INPLACE performs roughly 1.8× better than MM-SCAN across all problem sizes under a ***constant memory profile*** (fixed memory size). We used the memory profile generator to create profiles coupled with algorithm execution. Under the adversarial memory workload, MM-SCAN performs 1.6 – 3.8× more disk I/Os than MM-INPLACE. Critically, the performance gap grows with the input size. We also created a ***benevolent memory profile*** that is tightly synchronized with the algorithm's execution but non-adversarial in nature. Under the benevolent memory profile, MM-SCAN and MM-INPLACE perform roughly within 1.5× of each other for all problem sizes. Under ***oblivious memory profiles***, i.e., profiles that are not tightly coupled with the algorithms' execution, we found that MM-SCAN incurs about

1.8× more I/Os than MM-INPLACE for all problem sizes. These results suggest that MM-SCAN possesses almost all the benefits of cache-adaptivity.

Next, we evaluate the cache-oblivious Lazy Funnel Sort (LFS) [32, 33] and the non-oblivious External-memory Merge Sort (EMS) [16] (both are I/O-optimal when the memory is fixed), and we find that even though LFS has a computational overhead over EMS stemming from its cache-obliviousness; it outperforms EMS on a wide range of fluctuating memory profiles. Figure 2.1b suggests that External-memory Merge Sort has better I/O-performance than Lazy Funnel Sort for all problem sizes under the constant memory profile (roughly incurring 0.9× disk I/Os on average). On the other hand, LFS adapts when the memory fluctuates, whereas EMS fails to adapt. EMS performs progressively worse than LFS as the input size increases under all types of fluctuating memory workloads (adversarial, benevolent, and oblivious), incurring $1.3 - 7\times$ more disk I/Os. Despite the computational overhead of cache-obliviousness, LFS performs significantly better than EMS under a range of fluctuating memory profiles.

We conducted an experiment to analyze the dynamic memory sharing among four concurrent program instances, specifically focusing on matrix multiplication and sorting tasks. In the first set of experiments of Figure 2.2, each of the four concurrent matrix multiplication instances multiplied two matrices of width 8192, with each matrix consuming 256 MiB of memory. An overall memory of 100 MiB was shared among all four instances. The top two plots in the figure represent the memory share dynamics for these matrix multiplication instances. In the second set of experiments, each of the four concurrent sorting instances sorted an array of 16 GiB, with a total of 64 GiB of memory shared among the four instances. The bottom two plots in the figure depict the memory sharing for these sorting instances. The experimental results indicate that the MM-INPLACE method and, to a great extent, the MM-SCAN method can effectively share memory among the concurrent instances. Specifically, these methods demonstrate a graceful degradation in memory allocation, allowing efficient memory utilization even under concurrent execution. Similarly, the LFS method shows effective memory sharing for the four sorting instances. In contrast, the EMS method fails to achieve efficient memory sharing, leading to suboptimal performance and indicating a lack of cache adaptivity. This discrepancy underscores the importance of memory sharing mechanisms in achieving efficient concurrent execution.

The above empirical results suggest that for the problems of matrix multiplication and sorting, cache-oblivious but non-adaptive algorithm MM-SCAN adapts to a large range of memory fluctuations well, while non-oblivious algorithm EMS does not. The tested cache-oblivious algorithms adapt well because

they are agnostic to the cache size, while the cache-aware algorithms are optimized for specific cache sizes. We conjecture that cache-oblivious algorithms have most of the empirical benefits of cache adaptivity, and our results are consistent with the conjecture.

Chapter overview. The rest of this section is organized as follows. Section 2.2 reviews the problems and algorithms that we explore in this study. Section 2.3 reviews reviews preliminaries about the cache-adaptive model and analysis necessary to understand the experimental design in this paper. Section 2.4 explains the memory profiles that we tested on. Section 2.5 describes the experimental setup and the algorithms' performance with fixed memory. Section 2.6 describes the memory-profile generator and the algorithms' performance under adaptively constructed memory profiles. Section 2.7 explores the algorithms' performance under obliviously constructed memory profiles. Section 2.8 provides concluding remarks and future directions.

2.2 Problems and algorithms

We study two well-studied problems that are extremely popular and are kernel to many other algorithmic solutions: matrix multiplication and external-memory sorting.

2.2.1 Matrix multiplication algorithms

This section describes the two cache-oblivious matrix multiplication (MM) algorithms we used. We analyze two variants of the cache-oblivious matrix multiplication algorithm, as described by Frigo et al. [33], which we refer to as MM-INPLACE Figure 2.3 and MM-SCAN Figure 2.4. Both algorithms divide each input matrix into four submatrices and perform eight recursive calls to compute submatrix products. To solve a matrix multiplication problem with both input matrices of size $\sqrt{N} \times \sqrt{N}$, both MM-SCAN and MM-INPLACE perform $O(N^{3/2}/B\sqrt{M})$ disk I/Os in the DAM model, which is optimal. Here, M refers to the size of memory and B refers to the cache-line size.

Remarkably, only MM-INPLACE is optimally cache adaptive; MM-SCAN is a $\Theta(\log(N/B^2))$ factor away from being optimal in the cache-adaptive model, when solving problem instances of size N [22]. This shows that cache-oblivious algorithms are not always cache adaptive. The two algorithms differ in how they combine the eight matrix sub-products. MM-SCAN adds the eight matrix sub-products in one final linear scan, yielding a recurrence for disk I/O as

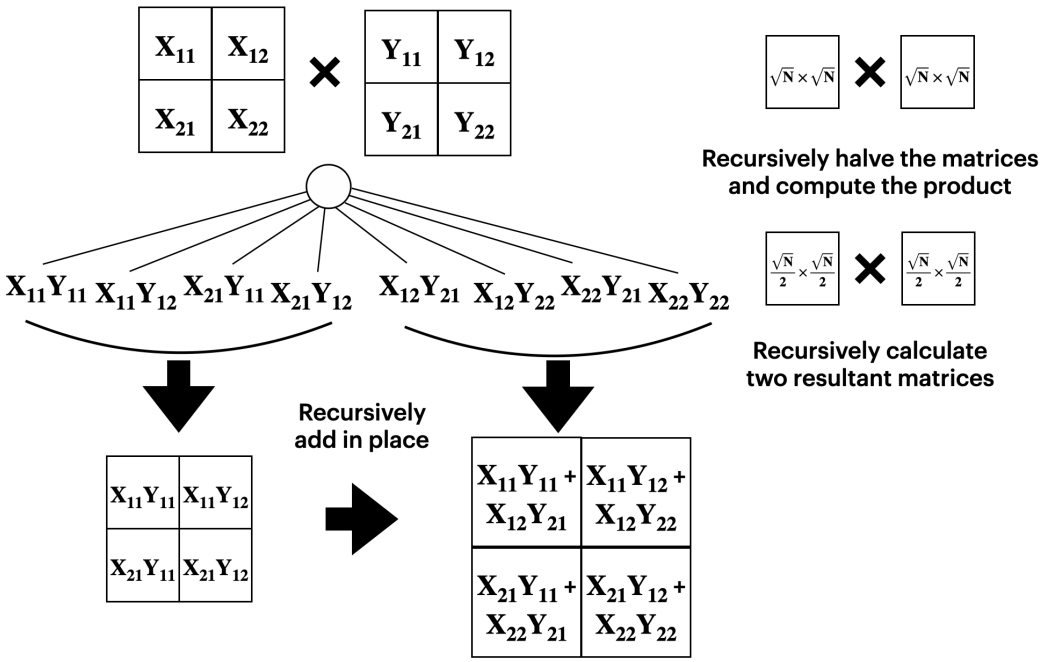


Figure 2.3: MM-INPLACE algorithm: a cache-oblivious cache-adaptive algorithm for matrix multiplication. The diagram assumes that the problem size is N, i.e., each input matrix is $\sqrt{N} \times \sqrt{N}$, memory size is M, and cache-line size is B.

follows, as we analyze under the DAM model.

$$Q(N) = 8Q(N/4) + \Theta(1 + N/B).$$

MM-INPLACE computes the eight matrix sub-products "in place," adding the results of elementary multiplications into the output matrix as soon as it computes them, and yielding a recurrence for disk I/O as follows when we analyze the algorithm under the DAM model.

$$Q(N) = 8Q(N/4) + O(1).$$

Both algorithms require a tall cache of $\Omega(B^2)$ and both recurrences have the same solution in the DAM model.

However, the linear scans of MM-SCAN are wasteful if they execute when memory is plentiful. For each input size N, we can construct a profile that causes MM-SCAN to run inefficiently by giving the algorithm lots of memory during its scans, when it can't use it, and very little memory at other times. As a result, the memory profile will have a recursive structure that mimics

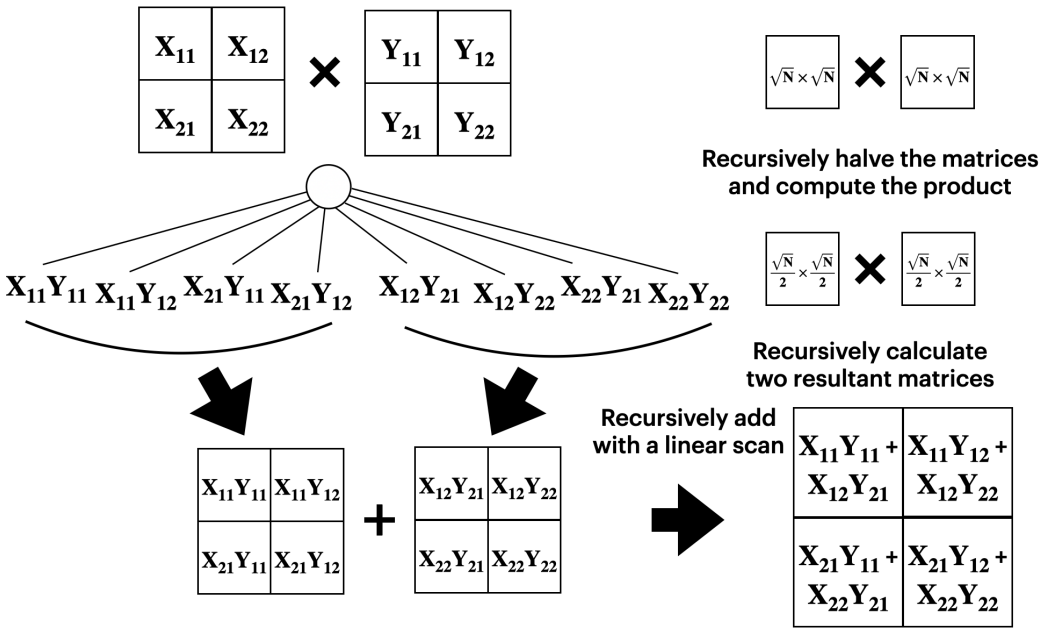


Figure 2.4: MM-SCAN algorithm: a cache-oblivious non-adaptive algorithm for matrix multiplication. The diagram assumes that the problem size is N, i.e., each input matrix is $\sqrt{N} \times \sqrt{N}$, memory size is M, and cache-line size is B.

that of MM-SCAN: where MM-SCAN performs a linear scan of size $\Theta(N)$, the memory profile will contain a square of size $\Theta(N)$.

2.2.2 External-memory sorting algorithms

External-memory sorting algorithms are crucial for efficiently sorting in large datasets that do not fit into the main memory (RAM). The first external-memory sorting algorithm we explore is the M/B-way merge sort, which is specifically designed to optimize Input/Output (I/O) operations between fast internal memory and slower external storage. This algorithm extends the classical merge sort to handle the constraints and characteristics of external memory hierarchies, achieving significant improvements in I/O efficiency. The objective is to minimize I/O: an I/O operation typically involves transferring a block of data between the internal memory and the external storage. The key idea is to divide the data into smaller blocks, sort each block in internal memory, and then merge these blocks in an I/O-efficient manner. It is schematically explained in Figure 2.5.

The M/B-way merge sort uses a recursive approach to achieve its I/O

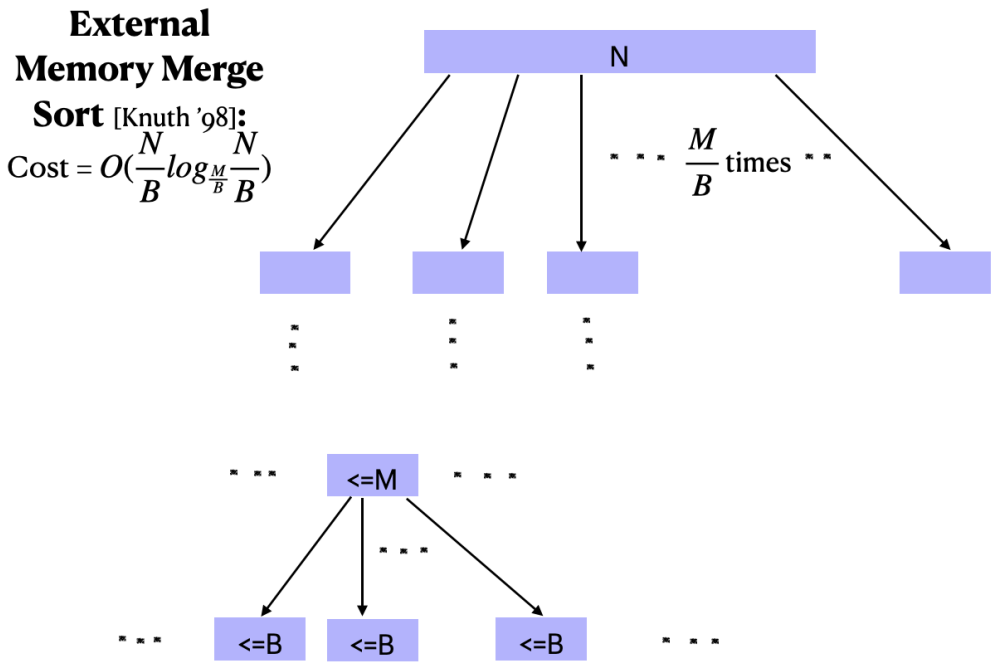


Figure 2.5: External-memory Merge Sort (EMS) algorithm: a cache-aware (non-oblivious) non-adaptive algorithm for external-memory sorting. The diagram assumes that the length of the unsorted array is N, memory size is M, and cache-line size is B.

efficiency. Initially, the algorithm divides the input data into N/B chunks, each of size B. Each chunk is sorted using an in-memory sorting algorithm, which involves reading the chunk into memory, sorting it entirely in memory, and writing it back to the external storage. This initial sorting phase requires $O(N/B)$ I/Os. The sorted chunks are then merged recursively. In each merge step, the algorithm merges M/B sorted chunks into a single larger sorted chunk. The merge operation reads M/B blocks into memory, merges them, and writes the merged blocks back to external storage. This merge process is repeated until the entire dataset is merged into a single sorted sequence. The depth of the merge tree is $\log_{M/B}(N/B)$ and each level of the tree requires $O(N/B)$ I/Os. External-memory merge sort is I/O-optimal when the memory size is fixed throughout the execution achieving an asymptotic disk I/O of $O(N/B \log_{M/B}(N/B))$. The above bound matches the lower bound for disk I/O for external-memory sorting.

The second algorithm we study in our experiments is Lazy Funnel Sort

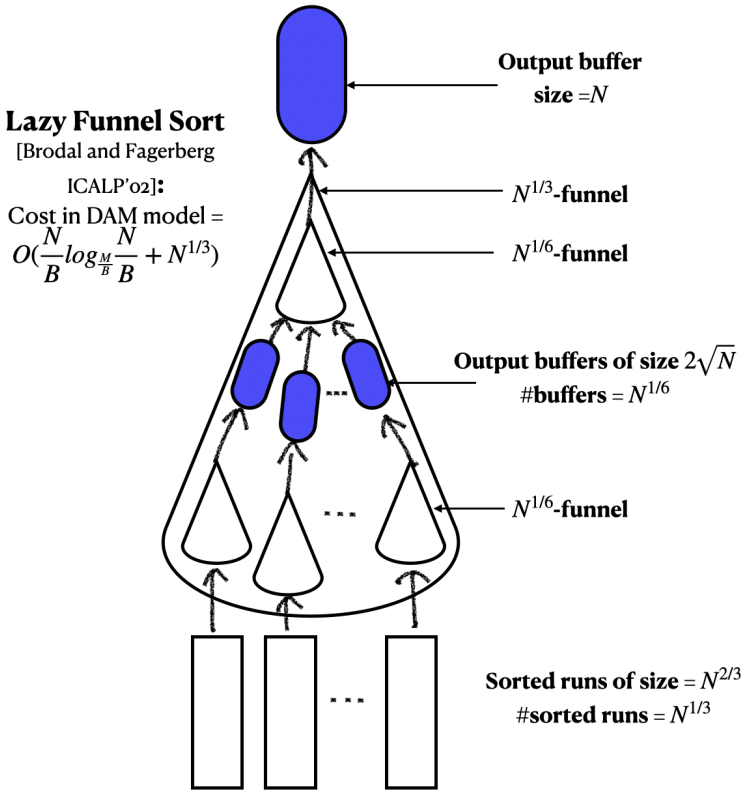


Figure 2.6: Lazy Funnel Sort (LFS) algorithm: a cache-oblivious cache-adaptive algorithm for external-memory sorting. The diagram assumes that the length of the unsorted array is N, memory size is M, and cache-line size is B.

(LFS) introduced by Brodal and Fagerberg [32], which is a simpler variation of the Funnel Sort algorithm [33]. We start with a brief exposition of the LFS algorithm [32]. At the core, LFS employs a concept known as k-merger, a perfectly balanced binary tree with k leaves and a binary merger at each internal node. Each leaf has a sorted input stream, and the root has an output stream with capacity k^d, where $d \geq 2$ is a tuning parameter. The size of buffers between internal nodes is defined recursively: Consider a horizontal cut in the tree at half its full height: $D_0 = \lceil \log k/2 \rceil$. The buffers between nodes of depth D_0 and D_0+1 have size $\lceil k^d/2 \rceil$. The subtree above depth D_0 is referred to as the top tree and all the subtrees rooted below are called bottom trees. The buffer sizes in the top and bottom trees are defined recursively.

Upon each invocation, a k-merger merges k^d elements into its output buffer. We call a complete invocation of a k-merger, producing k^d elements in the output buffer, a round of execution of that k-merger. A k-merger together

with its internal buffer is linearized in a recursive Van Emde Boas layout. First, the top subtree is laid out in a contiguous array, followed by all the bottom subtrees in their respective contiguous arrays. The workflow of a k-merger is based on recursive calls to the underlying binary merger tree. A call is made to the root of a k-merger to fill its output stream by merging its two input buffers. When one of the buffers is exhausted, a recursive call is made to the child node to fill it. Figure 2.6 provides a schematic representation of the funnel in the LFS algorithm. For a complete description and analysis, we refer the interested reader to Brodal and Fagerberg's paper [32].

LFS cannot use a single N-merger to sort the input array, since an N-merger would have a superlinear size. Instead, LFS calls itself recursively to produce $N^{1/d}$ sorted streams of size $N^{1-1/d}$ and then merges these using an $N^{1/d}$-merger. This approach allows LFS to maintain its cache-oblivious property while efficiently handling large datasets. The recursive structure and careful management of buffers ensure that the algorithm remains I/O efficient, making it well-suited for systems with complex memory hierarchies. The total I/O complexity of LFS is the same as External-memory M/B-way Merge Sort (EMS), i.e., $O(N/B \log_{M/B}(N/B))$.

2.3 Cache-adaptive analysis

This section reviews the fundamentals of cache-adaptive and cache-oblivious analysis. It also explains how (a, b, c)-regular algorithms play a key role in the analysis.

2.3.1 External memory models

First, we explain a number of theoretical models to analyze the performance of external memory algorithms, which run on data that does not fit into memory. These memory models formalize the theoretical analysis of algorithms on modern computers with hierarchical memory.

Disk Access Machine (DAM) model. In the DAM model [16] of computation, the memory is modeled to have a two-level hierarchy, containing a fast memory of limited size M and an external disk of unlimited size. It does not take any time to compute with the data that resides in the memory. Data, required for the computation, that is not residing in the memory, has to be read from the external disk. Similarly, when the memory is full and data is to be read from an external disk, some data has to be written from memory to an external disk to make room for the new data. Data is transferred between

the memory and external disk in chunks of fixed size B. Figure 2.7 gives a schematic view of the DAM model. When we study the performance of an algorithm under the DAM model, the performance is measured by the number of I/Os (i.e., block transfer). This measure serves as a proxy for running time. In each step, one block is transferred between the memory and the disk. The internal memory is fully associative.

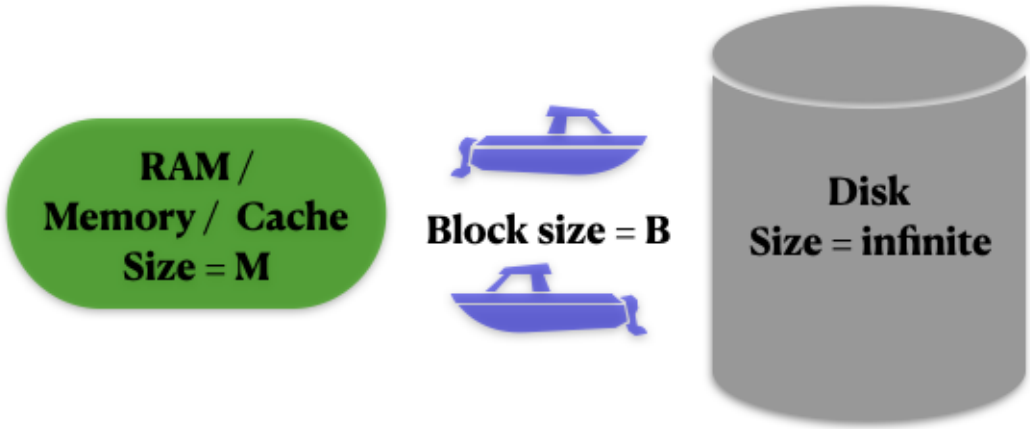


Figure 2.7: The Disk Access Machine (DAM) model of computation.

Ideal-cache model. Cache-oblivious algorithms [33] are analyzed using the ideal cache model [33, 45]. The ideal-cache model is an extension of the traditional DAM model. It includes an automatic and optimal page replacement policy. Additionally, the ideal-cache model includes ***tall-cache assumption***: it posits that the cache is large enough to hold a significant fraction of the problem's working set, but not so large that it can store the entire data set. Formally, if the cache size is M and the block size is B, the tall-cache assumption requires that $M = \Omega(B^2)$. This ensures that the cache is sufficiently large to exploit spatial locality effectively, allowing algorithms to achieve near-optimal performance by minimizing the number of cache misses. The assumption helps in deriving theoretical bounds and guiding practical optimizations for algorithms in hierarchical memory systems.

Under the ideal-cache model, the cache-oblivious algorithms are not parameterized by the value of M and B. On the contrary, the performance of the algorithms is analyzed in terms of M and B. Figure 2.8 shows a schematic of the ideal cache model.

Automatic page replacement in the ideal cache model is necessary because the algorithm has no knowledge of M and B. The optimal page replacement assumption is justified since the Least Recently Used (LRU) page replace-

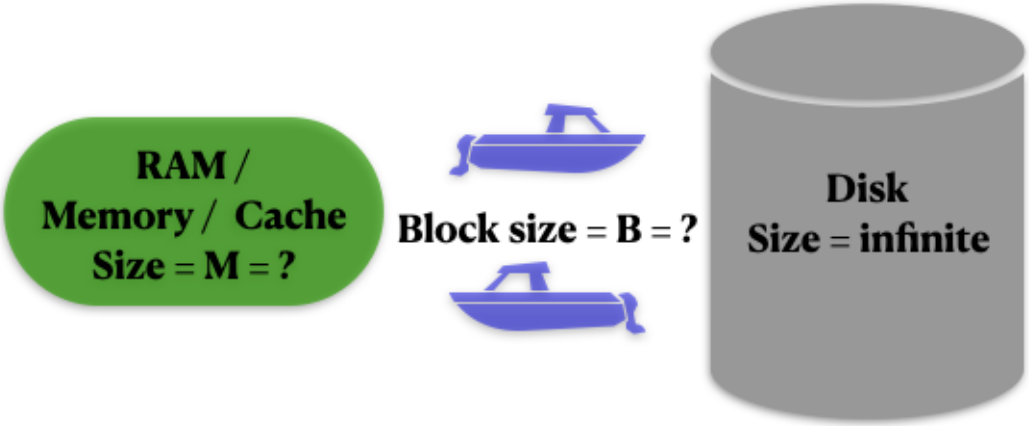


Figure 2.8: The ideal cache model of computation. The memory size and block size is unknown to the algorithms.

ment policy is constant-competitive when given a constant memory augmentation [4]. Although, LRU is $O(k)$-competitive when the memory size is k and constant-competitive with resource augmentation.

Cache-adaptive model. The cache-adaptive model is an extension of the disk-access machine (DAM) model [16], where the size of memory available to an algorithm can change during an algorithm's execution. In the DAM model, the machine has a two-level memory hierarchy comprising a cache/memory of size M and a disk of unbounded size. Data is transferred between disk and memory in blocks of size B (B is also called the cache-line size), called I/Os. The cache-adaptive model extends the traditional DAM model by allowing the memory size to be a function of time. Figure 2.9 shows the schematic of the cache-adaptive model of computation.

Each I/O takes one time step and computation is modeled as free and instantaneous. We are using I/O as a proxy of time because we are interested in studying I/O-bound algorithms. At time t, the memory available to a program is $M(t)$, which can change after each time step. We refer to $M(t)$ as a memory profile.

When the size of memory decreases, a large number of blocks may need to be written back to disk, when there are dirty pages and blocks. The cache-adaptive model assumes that an algorithm can query the current size of memory, $M(t)$, at each time step t. Note that, the concept of the cache-oblivious algorithm does not change under the cache-adaptive model from the ideal-cache model, i.e., cache-oblivious algorithms still do not query the size of memory and stay oblivious to any dynamic changes to the memory size. As

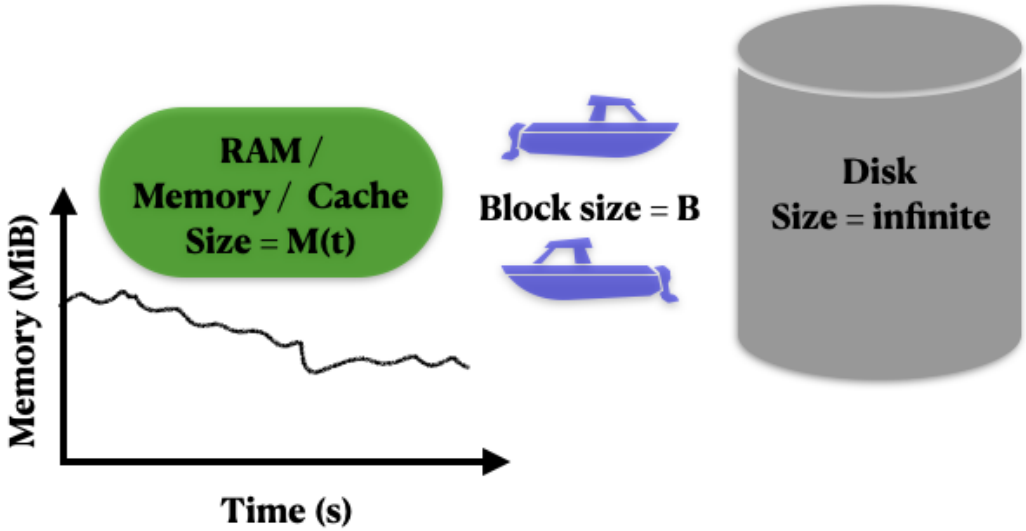


Figure 2.9: Cache-adaptive model of computation. It allows the available memory to change at each timestep.

in the ideal-cache model, when we consider a cache-oblivious algorithm in the CA model, we assume that the system manages the content of the memory automatically. However, non-oblivious algorithms can manage the content of memory on their own.

Cache-oblivious algorithms and (a, b, c)-regularity. An algorithm is cache-oblivious [33, 44, 45] if it is not parameterized by M and B. Interestingly, an optimal cache-oblivious algorithm is universal, in the sense that it runs optimally under the DAM model for all possible (fixed) values of M and B.

Cache-oblivious algorithms with a particular kind of divide-and-conquer structure, are said to be ***(a, b, c)-regular***. An algorithm is (a, b, c)-regular for constants $a \geq 1$, $b > 1$, and $0 \leq c \leq 1$, if, for problem size N, its I/O complexity satisfies the following recurrence:

$$Q(N) = a \cdot Q(N/b) + O(1 + N^c/B).$$

Specifically, the algorithm has a recursive calls on sub-problems of size N/b and $\Theta(1)$ ***linear scans*** before, between, or after the recursive calls, where the size of the largest scan can be up to the size $\Theta(N^c)$. A linear scan is defined as the work done by the cache-efficient recursive algorithm while not making recursive calls; it is an algorithm that makes $\Theta(l)$ memory references with an I/O complexity of $\Theta(1 + l/B)$ [22]. Note that a linear scan might be more

general than a traditional ***sequential scan***, i.e., accessing a set of contiguous elements sequentially, although the sequential scan qualifies as a linear scan. Note that some linear scans and algorithms are I/O efficient only under a tall-cache assumption, i.e., the memory size has to be $\Omega(B^2)$ (e.g., cache-oblivious sorting or matrix transpose) [33]. Thus, the definition of a linear scan also depends implicitly on the memory profile.

2.3.2 Analytical tools for cache-adaptive analysis

In this section, we explain the analytical tools proposed in the theoretical cache adaptive analysis framework. Primarily, we shall talk about how to model memory fluctuations and compare performance of two programs under a specific set of memory fluctuations.

Memory profile. The cache-adaptive model [21] is an extension of the DAM model [16] except that memory may change size after each I/O (i.e., after each cache miss). Thus, the size of memory is not a constant, but rather a function of time $m(t)$ giving the size of memory (in blocks) after the tth I/O. $m(t)$ is called a ***memory profile***.

Speed augmentation. In the cache-adaptive model, it might be improper to compare two algorithms' performance directly under a given memory profile $m(t)$. To see why, suppose B is a workload that performs one dummy I/O and then runs A. By any reasonable definition, B's running time should be asymptotically no worse than A's. However, consider a memory profile $m(t)$ that drops to very little memory just after the number of I/Os A should make on the memory profile $m(t)$. Now B may finish arbitrarily later than A.

Under cache-adaptive analysis, optimality is formalized by comparing algorithms using speed augmentation. Rather than granting an algorithm extra time, the model allows it to perform multiple I/Os per time step in the analysis. Speed augmentation is a theoretical tool to ensure that algorithms are compared on profiles that provide the same overall resources up to a constant factor. Thus, rather than saying one algorithm is (say) twice the speed of another, we say that it performs equally well on hardware with half the latency. This gives the same intuition as classic asymptotic analysis while being meaningful in the cache-adaptive model.

2.3.3 Theoretical results on cache-adaptive model

The worst-case performance gap between obliviousness and adaptivity. DAM-optimal (a, b, c)-regular algorithms can be up to an $O(\log N)$-factor away from being optimally cache-adaptive [22]. Specifically, they are suboptimal in the cache-adaptive model when $a \geq b$ and $c \geq 1$ [22]. Intuitively, (a, b, c)-regular algorithms are not optimal when they contain large sequential reads through memory at each level of the recursion.

The worst-case memory profile. The logarithmic gap in I/O-performance of some (a, b, c)-regular algorithms from being optimally cache-adaptive is based on a worst-case memory workload that adaptively mimics the recursive structure of the algorithm and is tightly synchronized with the execution of the algorithm. Specifically, the worst-case profile provides extra memory to the non-adaptive algorithm during the linear scans (when it cannot benefit from the extra memory) and takes away the extra memory at the end of the linear scans (when it would be able to use the extra memory). The worst-case memory profile is constructed adaptively by following the execution of the non-adaptive algorithm, MM-SCAN in case of matrix multiplication, and EMS in case of sorting.

2.4 Memory profile design

This section presents the high-level design of our evaluation of cache-adaptive and non-adaptive algorithms on a variety of memory profiles for matrix multiplication and external-memory sorting. We test on adaptively constructed memory profiles (e.g. the worst-case profile) as well as "oblivious" memory profiles that are erratic but not adaptively constructed. To perform this evaluation, this study uses a memory profile that does not fluctuate with respect to time. We refer to this as the ***constant memory profile*** and we use it as a baseline in our experiments.

It is theoretically shown that under ***adaptively constructed*** worst-case memory workload the performance of two DAM-optimal algorithms diverges. Between two (a, b, c)-regular algorithms, MM-INPLACE is cache adaptive and MM-SCAN is non-adaptive as it is log-factor away from being optimally cache adaptive under this worst-case memory profile. We construct this worst-case ***adversarial memory profile*** by adapting to the execution of the non-adaptive algorithm and allowing it enough memory to store each recursive sub-problem during its linear scan, and low memory when the algorithm is

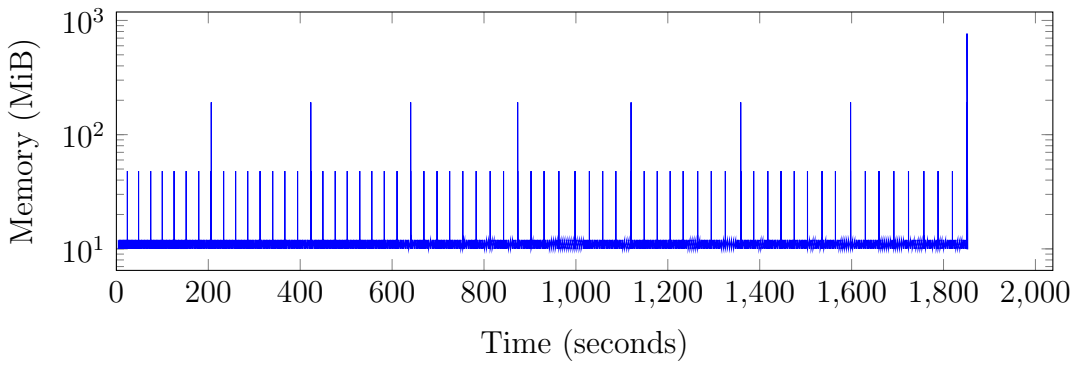


Figure 2.10: Sample adversarial memory profile generated from running MM-SCAN to multiply two square matrices of width **8192**. The structure of the memory profile mimics the recursive structure of MM-SCAN.

not performing a linear scan. Since linear scans do not employ any locality of reference, the algorithm only requires $O(1)$ memory; any memory given more than that is not utilized during linear scan. For example, MM-SCAN contains a linear scan at the end of each recursion; in each linear scan of size $N \times N$, memory is increased to $5N^2$ to hold the input and output matrices as well as the intermediate results. However, the cache-adaptive algorithm opportunistically benefits from these memory increases, causing its I/O performance to improve considerably relative to that of the non-adaptive algorithm. Figure 2.10 shows a sample adversarial memory profile for matrix multiplication.

Similarly, we construct another category of adaptive memory profiles, i.e., the ***benevolent memory profile***, but these are designed to be benevolent to the non-adaptive algorithm. The non-adaptive algorithms are unable to use any extra memory that adversarial memory provides during the recursive sequential scans. In the case of the benevolent memory profile, during the sequential scans, instead of increasing the memory we adaptively decrease memory. When the linear scan ends, the memory is again increased to the initial value.

Both the adversarial and benevolent memory profiles are tightly synchronized with the execution of the non-adaptive algorithm. In fact, the adversarial memory profile mimics the recursive structure of the non-adaptive algorithm to bring out the worst performance relative to the adaptive algorithm. In contrast, the benevolent profile aims to minimize the advantage of the cache-adaptive algorithm over the non-adaptive algorithm. However, adversarial and benevolent memory profiles might be too pessimistic or too optimistic in terms of the relative performance of the non-adaptive algorithm and rarely found in real-world applications. Beyond-worst-case analyses [50] show that

the performance gap between MM-SCAN and MM-INPLACE disappears with "sufficient" smoothing on the worst-case profile, otherwise, the performance gap remains even under smoothed memory profiles. Hence the question remains, how do the non-adaptive algorithms perform under a range of memory profiles that are not generated in an adaptive manner (i.e. the profiles which do not adapt to the execution of the algorithm)?

To gain a more complete view of algorithm performance under memory fluctuations, we run algorithms under ***oblivious memory profiles*** that are generated non-synthetically and non-adaptively. These profiles are created independently of a given algorithm's execution. A sample oblivious profile can be obtained by running several programs concurrently in shared memory without limiting any of their memory usage [54, 55]. An evaluation under such oblivious workloads accounts for the fact that in practice, often a running process is forced to share available RAM with other concurrent memory-intensive programs. Hence, these oblivious profiles enable us to observe a more complete view of an algorithm's performance under memory fluctuations in a multi-program environment. Figure 2.11 showcases the different classes of memory profiles we develop in the framework.

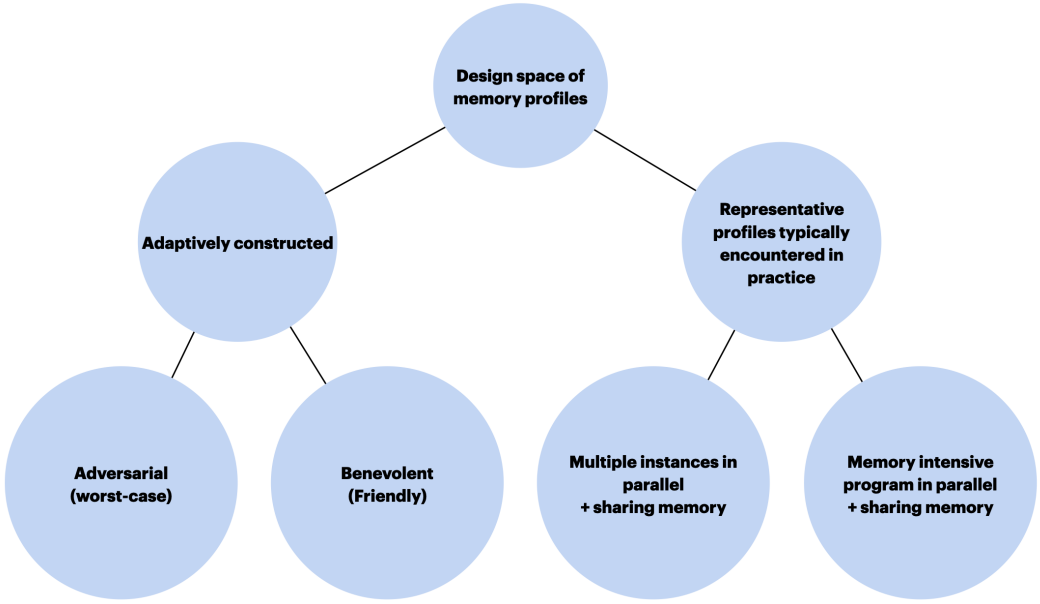


Figure 2.11: Different categories of memory profiles explored in this study. Primarily, we explore two types of memory profiles: adaptively constructed memory profiles and memory profiles that are oblivious to the algorithms' execution.

2.5 Evaluation on constant profiles

This section presents details of the experimental setup and studies the performance of the matrix multiplication and external-memory sorting algorithms described in Section 2.4 on constant memory profiles. All of the tested algorithms are theoretically optimal when the memory does not fluctuate. The empirical results confirm the theory: the performance gap between the cache-adaptive and non-adaptive algorithms does not grow with the problem size when the memory size is fixed.

Experimental setup. All experiments were conducted on a Dell Precision 5820 with an Intel®Core™i9-9900X 3.50 GHz processor, two Samsung 16 GiB M378A2K43DB1-CTD Dual Rank Memory Modules, and Seagate Barracuda ST2000DM001 Desktop SATA Hard Drive. The operating system used is Linux Mint 19.1, Tessa, with kernel version 4.15.0-88. We used C++14 to implement the algorithms, g++ 9.3.0-17ubuntu1 20.04 as the compiler, and bash scripts to run the experiments. We measure the number of disk I/Os incurred by a program using */proc*. At the start of each experiment, we use the `sync` system call to free page-cache and slab objects. We use Linux control groups (*cgroups*) to limit the memory available to a program. We run all the experiments with 25 trials and average the measured I/Os.

Algorithm implementation descriptions. We implemented the two algorithms for MM-SCAN and MM-INPLACE directly from their description in past work on cache-adaptivity [22]. Section 2.3 provides a detailed description of the divide-and-conquer recursion of these two algorithms.

We explored two algorithms for sorting: External-memory Merge Sort (EMS) and Lazy Funnel Sort (LFS). We implemented an (M/B)-way external-memory merge sort that is compatible with the memory profile generator and chose $M = 256$ MiB and $B = 8$ MiB. Previous work [56] on engineering external sorting algorithms demonstrate that carefully engineered cache-aware sorting algorithms can perform up to $2\times$ better than cache-oblivious ones when the memory does not fluctuate. We expect our results regarding relative algorithm performance in the face of memory fluctuations to hold for other implementations of EMS and LFS because of the difference in the algorithm structure between the two algorithms. For LFS, we used an implementation from Olsen and Skov [57].

Problem sizes. We evaluate the relative performance of MM-SCAN to MM-INPLACE on the constant memory workloads as a function of input size. For each matrix multiplication experiment, we multiply two square matrices. We

increase the input matrix width from 512 to 32768. For all the experiments, we provide a fixed memory of 10 MiB to the programs.

Similarly, we study the relative performance of EMS to LFS on the constant memory workloads. For each sorting experiment, we sort an integer array. We increase the input array length from 67 million to 1 billion. We provide a fixed memory of 256 MiB to the programs.

Results. Figure 2.1a shows that when memory size is fixed, cache-adaptive MM-INPLACE performs roughly 1.8× better than non-adaptive MM-SCAN for all input sizes, i.e., their performance gap does not grow with problem size. On the other hand, Figure 2.1b shows that cache-aware non-adaptive EMS incurs less disk I/Os (roughly 0.9× on average) than cache-adaptive LFS. This result shows the cache-oblivious sorting algorithm does not bear a major computational overhead when the memory does not fluctuate.

2.6 Evaluation on adaptive profiles

This section presents details of how we generate the memory fluctuations and studies the performance of the matrix multiplication and sorting algorithms described in Section 2.4 on adaptively generated memory workloads. Specifically, it first introduces the memory profile generator used to construct the adaptive profiles. Using this profile generator, we explore two types of adaptive memory profiles: the adversarial profile, and the benevolent profile. Under adversarial workloads, the cache-adaptive algorithms perform up to 4.5× better than the non-adaptive algorithms with increasing problem size. On the other hand, the benevolent memory profile demonstrates that cache-oblivious algorithms for matrix multiplication and sorting that we experiment with adapt well to non-adversarial memory fluctuations while non-oblivious algorithms suffer. MM-INPLACE is up to 1.5× better than MM-SCAN, but the gap does not grow with the problem size. In contrast, LFS performs increasingly better (up to 3.2×) than EMS as the problem size grows.

Memory profile generator. We designed a ***memory profile generator*** that runs a program under a particular memory workload. To fluctuate the memory available to a program, we first used *cgroups*, however, *cgroups* does not allow us to decrease the memory available to a program (that the program already claimed) efficiently while the program is running. To address this, the memory profile generator uses *cgroups* to set an initial memory limit available to a program and runs a *balloon* program concurrently with the program within the *cgroups*. The balloon program uses the memory complement to what we

intend the program to use. For example, if we want a program to use memory M until time t and thereafter memory $M+\delta$, and the *cgroups* has memory C, the balloon program uses memory $C-M$ until time t and thereafter it uses memory $C-M-\delta$. The initial memory given to a program remains the same as for the constant memory workload.

Using this memory profile generator, we generate two types of profiles, adversarial and benevolent; see Section 2.4. We measure algorithm performance using the same problem sizes as in Section 2.5.

Results for the adversarial memory profile

First, let us turn our attention to the adversarial memory workloads. Figure 2.1 illustrates that under the adversarial memory workloads, the cache-adaptive algorithms perform better than the non-adaptive algorithms, and the gap increases as the problem sizes grow. MM-INPLACE performs roughly up to 4× better than MM-SCAN and LFS performs roughly up to 4.5× better than EMS.

Time spent in linear scan. The performance gap between the cache-adaptive and non-adaptive algorithms grows with the problem size because the relative time spent in scans by the non-adaptive algorithms (MM-SCAN and EMS) grows with the problem size. Table 2.2 shows that the relative time spent in scans grows with problem size for both MM-SCAN (up to 13%) and EMS (up to 76%). Hence, the performance gap also increases between the adaptive and non-adaptive algorithms with problem size. Cache-adaptive algorithms do not perform any such linear scans and take advantage of the extra memory in the adversarial profile.

Algorithm	*Input size*	*% of runtime*
MM-SCAN	1024×1024	0.1
MM-SCAN	2048×2048	4.7
MM-SCAN	4096×4096	13.2
EM Merge Sort	512 MiB	43.6
EM Merge Sort	1 GiB	57.9
EM Merge Sort	2 GiB	76.8

Table 2.2: Percentage of running time that the non-adaptive algorithms spend on linear scans.

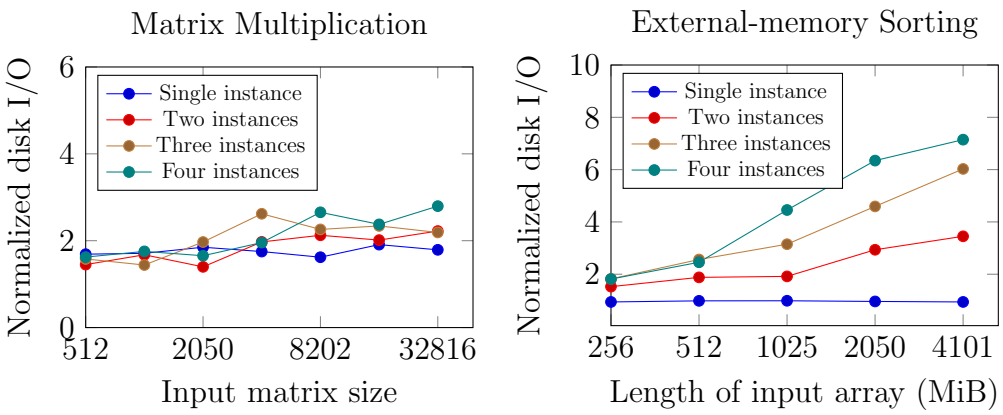
Results for the benevolent memory profile

Next, let us turn our attention to the algorithms' performance under the benevolent memory workloads. Figure 2.1a shows that MM-SCAN performs very close to MM-INPLACE (between $1.3\times - 1.6\times$) on the benevolent profiles since MM-SCAN is not affected by the memory reductions that occurred during the linear scans whereas MM-INPLACE is affected. On the other hand, Figure 2.1b shows that the gap between EMS and LFS grows with the problem size — EMS incurs up to $3.2\times$ more I/Os than LFS on the largest input. EMS is unable to adapt to memory reductions during linear scans while LFS adapts to all changes in memory.

2.7 Evaluation on oblivious profiles

This section explains the setup for the oblivious memory workloads that are not synchronized with the recursive execution of the algorithm and shows that the I/O-performance gap between the two cache-oblivious MM algorithms disappears, but remains between the cache-oblivious and non-oblivious algorithms for sorting. Figure 2.1a illustrates that MM-SCAN incurs roughly $2\times$ more disk I/Os on average than MM-INPLACE for oblivious memory profiles as we increase the input size. Figure 2.1b, on the other hand, shows that EMS incurs increasingly worse than LFS with problem size incurring up to $4.3\times$ I/Os on average under oblivious memory profiles. In fact, Figure 2.12b shows EMS may perform up to $7\times$ more I/Os under oblivious memory workloads.

Experimental setup. This study runs multiple memory-intensive programs concurrently, all sharing the same RAM, to ensure that each program instance experiences memory fluctuations that do not follow the recursive execution of the algorithm. We create these oblivious memory fluctuations in two ways: by creating a ***uniform*** environment where multiple identically-sized copies of the same program run concurrently, and by creating a ***nonuniform*** environment by creating nonuniformity in the concurrent programs. Such nonuniform concurrent programs may include program instances of the same program of different problem sizes, or different programs of the same problem sizes. The concurrent programs share a fixed memory that is given at the start of the experiment. For all the MM experiments, it is 10 MiB, and for all the sorting experiments, it is 256 MiB, if not stated otherwise. We keep the rest of the experimental setup the same as in the case of constant memory profiles (see Section 2.5).



(a) Normalized disk I/O of MM-SCAN to MM-INPLACE, instances share a memory of 10 MiB.

(b) Normalized disk I/O of External Merge Sort to LFS, instances share a memory of 256 MiB.

Figure 2.12: Average disk I/O of a non-adaptive program to the cache-adaptive program as a function of problem size when running up to 4 instances concurrently normalized to the same when a single instance runs concurrently.

Running uniform instances concurrently

First, let us turn our attention to the oblivious memory workloads that are generated by running multiple concurrent instances of the same program that share a memory [54, 55]. Figure 2.12a illustrates that cache-oblivious non-adaptive algorithm MM-SCAN performs close to cache-adaptive MM-INPLACE and incurs roughly up to 2× disk I/Os across all problem sizes. On the other hand, Figure 2.12b shows that the I/O-performance for the cache-aware algorithm EMS rapidly declines relative to the cache-adaptive LFS with increasing problem size (1.4 – 7.1×). Dice et al. [20] showed that multiple threads in shared memory may exhibit a "winner-takes-all" phenomenon where a thread may take memory from the others. However, Figure 2.12 shows that even if uniformly-sized program instances unevenly share the cache, cache-oblivious algorithms adapt well to memory fluctuations.

To analyze the dynamic memory sharing among four concurrent program instances, we conducted an experiment focusing on matrix multiplication and sorting tasks. The memory usage was dynamically captured using the resident set size (RSS) from Linux as the programs were executed concurrently. To ensure controlled memory usage, we employed Linux control groups (cgroups) to upper bound the total memory available to the concurrent programs. In the first set of experiments, depicted in Figure 2.2, each of the four concurrent matrix multiplication instances multiplied two matrices of width 8192, with

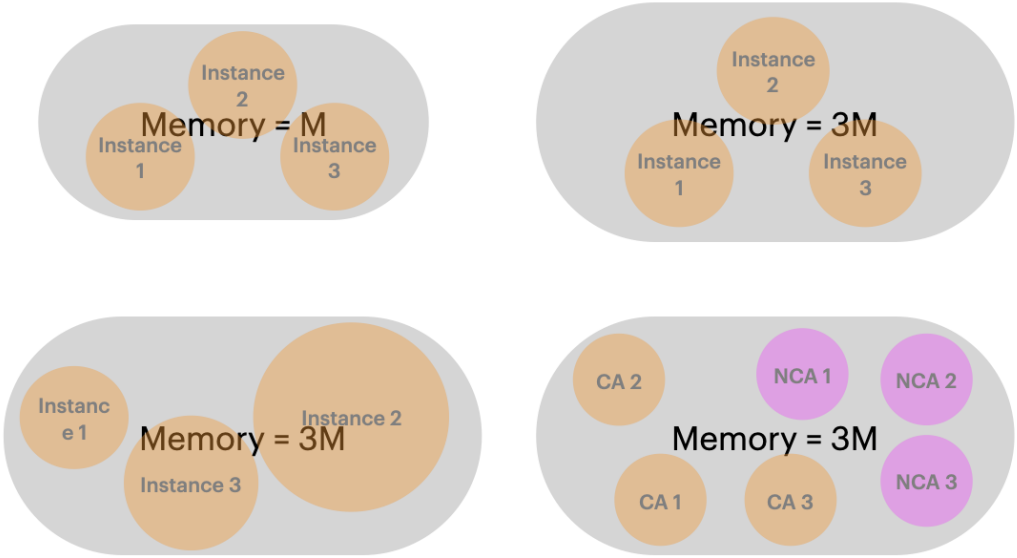


Figure 2.13: This diagram shows different ways to generate oblivious memory profiles. For the matrix multiplication experiments, we use $M = 10$ MiB and for the sorting experiments, we use $M = 256$ MiB.

each matrix consuming 256 MiB of memory. An overall memory of 100 MiB was shared among all four instances. The top two plots in the figure illustrate the memory share dynamics for these matrix multiplication instances. In the second set of experiments, each of the four concurrent sorting instances sorted an array of 16 GiB, with a total of 64 GiB of memory shared among the four instances. The bottom two plots in the figure depict the memory sharing for these sorting instances. The experimental results indicate that the MM-INPLACE method and, to a significant extent, the MM-SCAN method can effectively share memory among the concurrent instances. These methods demonstrate a graceful degradation in memory allocation, allowing for efficient memory utilization even under concurrent execution. Similarly, the LFS method shows effective memory sharing for the four sorting instances. In contrast, the EMS method fails to achieve efficient memory sharing, leading to suboptimal performance and indicating a lack of cache adaptivity. This discrepancy highlights the critical importance of effective memory sharing mechanisms in achieving efficient concurrent execution.

Setup. We evaluate the average I/O performance of the cache-adaptive and non-adaptive algorithms by running concurrent uniform instances, i.e., multiple identically-sized copies of the same program. This multi-program

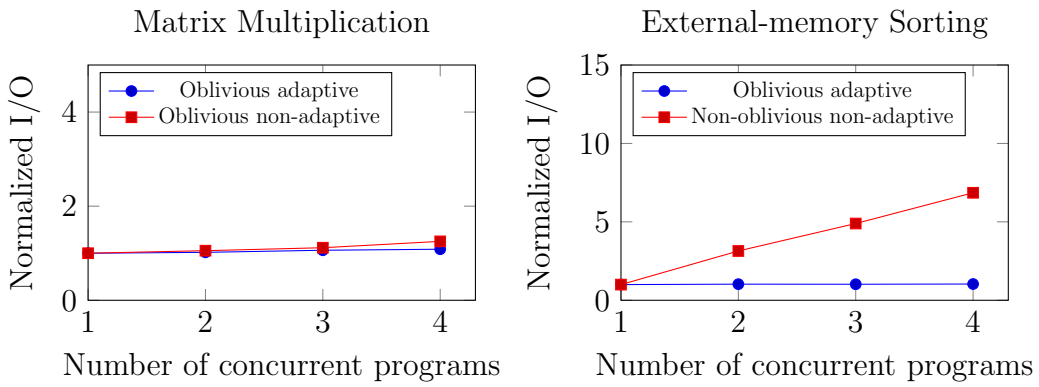


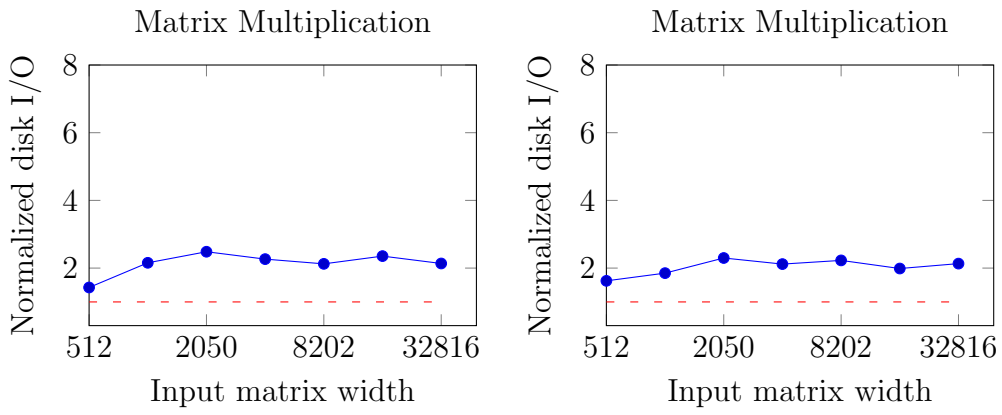
Figure 2.14: Average normalized disk I/O of a program instance when up to four program instances run concurrently normalized to the disk I/O when a single instance runs.

environment setup ensures that each program instance runs under a memory profile that is erratic due to the uncertainty stemming from the concurrent programs' memory requirement but independent of the recursive structure of the particular program under consideration. We perform the study on uniform instances in two ways. First, we vary the input problem sizes given a number of concurrent program instances. Next, we vary the number of concurrent instances in the range of $1 - 4$ for a given problem size and observe ***slowdown***, the degradation of the average I/O performance of a program instance.

Slowdown. Figure 2.14 illustrates that cache-oblivious algorithms incur relatively small slowdowns even as the number of concurrent instances increases. When we run up to 4 MM-SCAN instances concurrently, the average disk I/Os stays within 1.2× than the disk I/Os incurred by MM-SCAN when a single concurrent instance runs. The MM-INPLACE instances also face almost no slowdown in a similar environment. In contrast, concurrent EMS instances face up to 6× more disk I/Os when compared to an EMS program that does not share memory with other concurrent programs. However, LFS faces a negligible slowdown in the same scenario.

Running nonuniform instances concurrently

Next, let us turn our attention to how the algorithms perform when nonuniform program instances run concurrently and share a fixed memory. The nonuniform programs have different memory requirements at every time step and therefore are more likely to cause more dramatic memory fluctuations compared to the uniform case. We observe that along with the cache-adaptive



(a) 3 concurrent instances of matrix multiplication of the same size.

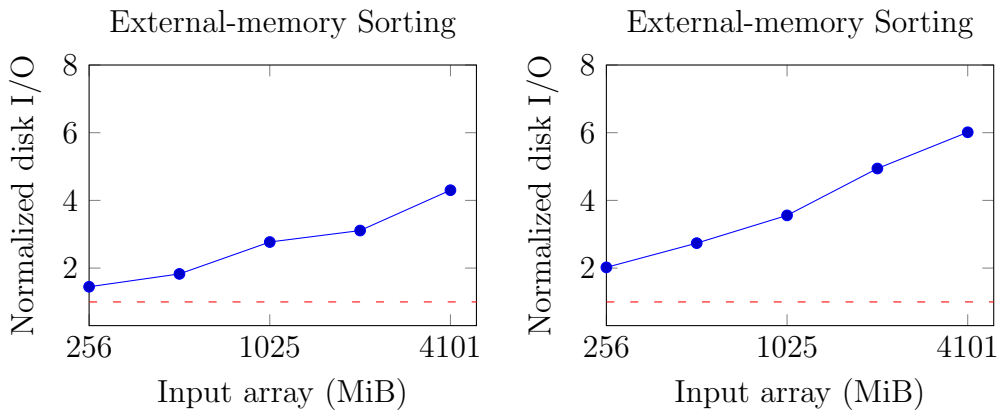
(b) 3 concurrent instances of matrix multiplication of different sizes.

Figure 2.15: Average normalized disk I/O of non-adaptive MM-SCAN to cache-adaptive MM-INPLACE as a function of problem size with 30 MiB memory given. The y-axis indicates that MM-SCAN performs within a constant factor of MM-INPLACE.

algorithms, cache-oblivious MM-SCAN also adapts to the fluctuations caused by nonuniform instances. MM-SCAN incurs up to 2.5× more disk I/Os than MM-INPLACE even when the memory fluctuations are more dramatic in the nonuniform case than in the uniform case, as shown in Figure 2.15. On the other hand, EMS incurs up to 6× more disk I/Os than LFS, which is more than the gap in the uniform case as demonstrated in Figure 2.16. Since the memory fluctuations in the nonuniform environment are likely to be more dramatic compared to the uniform environment, the performance gap between EMS and LFS is likely to be more than in the uniform environment because EMS is non-oblivious. Indeed, EMS performs increasingly worse than LFS under the oblivious memory workloads generated in nonuniform environments.

Setup. We create nonuniformity among the program instances in two different manners. In all experiments in Figures 2.15 and 2.16, we set the memory size to 30 MiB for MM and to 768 MiB for sorting and retain the problem sizes as mentioned in Section 2.5.

First, we concurrently run 3 instances of the same algorithm on different input sizes. For each experiment, we select 3 input sizes from the available set of sizes uniformly at random. The algorithm in each instance runs sequentially, but the instances run concurrently with each other. For each experiment, since the smaller instances may be completed earlier than the larger ones, we repeat any instances that have finished until all instances have finished. To measure



(a) 3 concurrent instances of sorting program of the same size.

(b) 3 concurrent instances of sorting program of different sizes.

Figure 2.16: Average normalized disk I/O of non-adaptive External-memory Merge Sort (EMS) to cache-adaptive Lazy Funnel Sort (LFS) as a function of problem size with 768 MiB memory given. An increase in the y-axis indicates that EMS performs progressively worse than LFS.

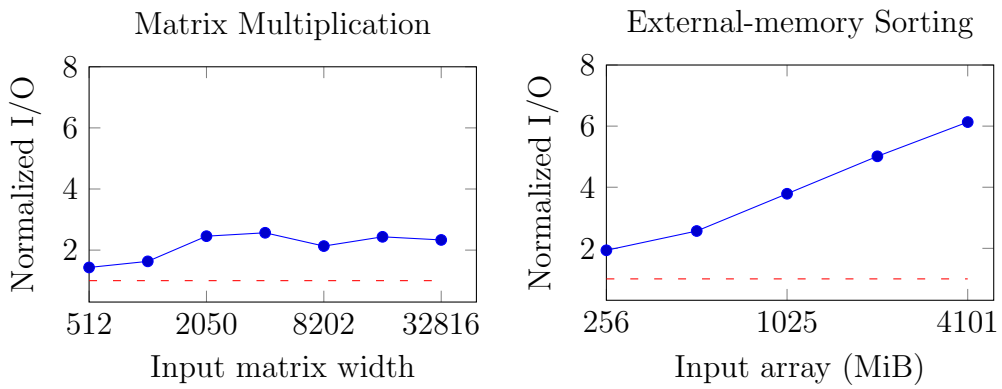
I/O, we report only the first run of each instance per experiment. As a baseline, we also run 3 uniform program instances to evaluate the effect of nonuniform input sizes.

Second, we concurrently run different algorithms for the same problem on the same input size. Specifically, we run 3 instances of the cache-adaptive algorithm (MM-INPLACE or LFS) and 3 instances of the non-adaptive algorithm (MM-SCAN or EMS) concurrently.

Running a database program concurrently

Finally, we turn our attention to how the MM and sorting algorithms perform when run under oblivious memory workloads that we simulated by running a memory-intensive database program concurrently. The database program is not adversarial and therefore not tied to any particular algorithm structure, so we expect the cache-oblivious algorithms to adapt better to the resulting memory fluctuations. Figure 2.18 confirms this hypothesis — MM-SCAN performs very close to MM-INPLACE (roughly incurring 2.1× disk I/Os), while EMS performs increasingly worse (up to 6×) than LFS as the problem size grows.

Setup. As the oblivious memory profiles are generated in a non-synthetic manner (see Section 2.4), and the memory fluctuations are oblivious to what



(a) 6 concurrent instances, 3 of MM-SCAN and 3 of MM-INPLACE.

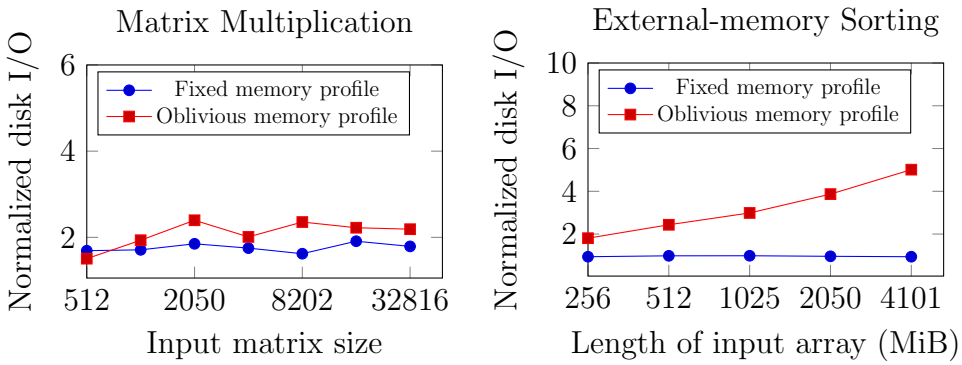
(b) 6 concurrent instances, 3 of EMS and 3 of LFS.

Figure 2.17: Average normalized disk I/O of non-adaptive matrix multiplication and sorting algorithms to cache-adaptive matrix multiplication and sorting algorithms as a function of problem size. For matrix multiplication (MM-INPLACE and MM-SCAN), we use a memory of size 30 MiB. For external-memory sorting (EMS and LFS), we use a memory of size 768 MiB. An increase in the y-axis indicates that EMS performs progressively worse than LFS. On the other hand, MM-INPLACE and MM-SCAN does not show a trend that grows with problem size. In each experiment, we run a total of 6 concurrent instances, 3 of cache-adaptive algorithms and 3 of non-adaptive algorithms, both for the experiments for matrix multiplication and sorting.

the algorithm does, we choose to run a concurrent memory-intensive database program with the desired algorithm to evaluate the algorithm's performance. We generated such an oblivious memory workload by running the TPC-C [58] workload concurrently with either a cache-adaptive or a non-adaptive program and presenting the normalized disk I/Os of the non-adaptive algorithms to the adaptive algorithms. TPC-C is a popular online transaction processing (OLTP) benchmark used to measure database management systems. Both the MM and sorting algorithms share the memory with the memory-intensive TPC-C program.

2.8 Conclusion

We investigated how some cache-oblivious and cache-adaptive (and neither) algorithms perform under a wide range of memory profiles. Specifically, we focused on matrix-multiplication algorithms as representatives of (a, b, c)-regular algorithms and sorting algorithms as representatives of cache-optimized but



(a) Normalized disk I/O of MM-SCAN to MM-INPLACE

(b) Normalized disk I/O of EMS to LFS

Figure 2.18: Normalized disk I/O of the non-adaptive to the cache-adaptive algorithm for variable problem size for oblivious memory workloads that we generated by running TPC-C concurrently with the programs. An increase in the y-axis means that the non-adaptive algorithm performs progressively worse relative to the adaptive algorithm.

not (a, b, c)-regular algorithms. We experimentally exhibited the gap in I/O-performance between the cache-adaptive and non-adaptive algorithms for MM and sorting under adversarially-generated memory workloads. To do so, we needed to design a profile generator that can dynamically change the amount of available memory depending on a program's execution. On the other hand, under our oblivious memory workloads, cache-oblivious MM algorithms performed close to each other, whereas the non-oblivious sorting algorithm still performed worse than the cache-oblivious (and adaptive) algorithm. We conjecture that what we have seen in our experiments applies more generally. That is, we conjecture that cache-oblivious programming is a powerful way of empirically achieving cache-adaptivity. This could be important because as mentioned in Section 2.1, there are many more known cache-oblivious algorithms than known cache-adaptive algorithms. Our results provide hope for the large body of work on cache-oblivious algorithms to empirically perform well even when the cache size fluctuates.

Chapter 3

ML-Advised Algorithms for Ski Rental Problem

This chapter presents an ML-advised algorithm design recipe for an online rent-or-buy problem, namely the ski rental problem with a discount, which we are going to formally define in this chapter.

This work was conducted in collaboration with Dr. Rathish Das [59]. An earlier version of this work was also published as a conference paper [60].

Abstract. Traditional online algorithms are designed to make decisions online in the face of uncertainty to perform well compared to the optimal offline algorithm for the worst-case inputs. On the other hand, machine learning algorithms try to extrapolate the pattern from past inputs to predict the future and make decisions online based on the predictions to perform well for the average-case inputs. Recent studies have augmented traditional online algorithms with machine learning oracles to get better performance for all the possible inputs. The machine learning augmented online algorithms perform provably better than the traditional online algorithms when the error of the machine learning oracle is low for the worst-case inputs and all other average-case inputs. Even when the advice from the machine learning oracle is poor, the machine learning advised algorithms are robust and, hence do not degrade abruptly with worst-case inputs.

In this article, we integrate the advantages of traditional online algorithms and machine learning algorithms in the context of a novel variant of the ski rental problem. Firstly, we propose the ski rental problem with a discount: in this problem, the rent of the ski, instead of being fixed over time, varies as a function of time. Secondly, we discuss the design and performance evaluation of the online algorithms with machine learning advice to solve the ski

rental problem with a discount. Finally, we extend this study to the situation where multiple independent machine learning advice is available. This algorithm design framework motivates redesigning several online algorithms by augmenting them with one or more machine learning oracles to improve their performance.

3.1 Introduction

Conventional online algorithms try to find a sound decision-making strategy for all situations. Hence, such an algorithm aims to provide a worst-case guarantee for every possible input. The usual way to understand the effectiveness of an online algorithm is to compare its performance with the optimal offline algorithm that is visionary and hence can see the future. Traditionally we define ***competitive ratio (CR)*** of an online algorithm as the ratio of the performance of the online algorithm to the same of the optimal offline algorithm for the worst possible input [1, 3, 4, 61].

On the contrary, machine learning (ML) systems are fundamentally different and serve distinct purposes altogether [62, 63]. ML systems learn from past situations and extrapolate patterns encountered in the past to predict the future. They come with guarantees of ***expected generalization error***: expected error in the prediction of the ML model for input data that the model has not seen yet. Hence, ML systems are "trained" to be good on the average-case inputs minimizing the expected loss. The interest lies in optimizing the prediction for most of the inputs at the expense of a few outliers. However, there is no theoretical guarantee on how "bad" the predictions are for those few outliers. The machine learning approach has another shortcoming: the expected generalization error guarantees are valid only if the training and testing data belong to the same probability distribution.

While machine learning approaches try to perform well for the average-case inputs, the conventional online algorithms compete with the optimal offline algorithm for the worst-case inputs. Online algorithms do not assume anything about the input sequence and stay oblivious of future inputs. While designing the online algorithms, the objective is to retain the competitive ratio as small as possible. Hence, the online algorithms become overly cautious about the "bad" inputs where the algorithms would perform much worse than the optimal offline algorithm. This strategy leads to a higher cost ratio for the online algorithm with respect to the optimal offline algorithm for supposedly "easy" inputs—hence in a simulation of a satisfactorily reasonable real-world input sequence, unsurprisingly, the conventional algorithms are likely to perform worse on average than the ML algorithms. Average-case competitive analysis

has been used to evaluate and capture this performance drop of traditional online algorithms [64, 65].

Machine learning augmentation. Another recent line of work tends to improve the competitive ratio by augmenting traditional online algorithms with supportive information from machine-learned oracles. These algorithms found applications in several problems such as competitive paging [34, 39, 66–70], job scheduling problem [71–75], linear search [75, 76], bin packing [77, 78], and data structures [35, 37, 79]. In 2017, Medina and Vassilvitskii [36] initiated a study on machine-learned online algorithms and used them to find the expected price of bids for revenue optimization. Lykouris and Vassilvitskii [34] extended the framework of designing online algorithms with ML advice models and formulated two critical functional aspects of such systems: ***consistency*** and ***robustness***. A consistent system performs well when the ML predictions are fairly correct, and a robust system refrains from performing exceedingly poorly as the predictions degrade. They designed a competitive caching algorithm based on the ML model's prediction for an element's subsequent arrival time as the elements come online from the input sequence. Figure 3.1 explains the high-level design and approach of ML-advised algorithms.

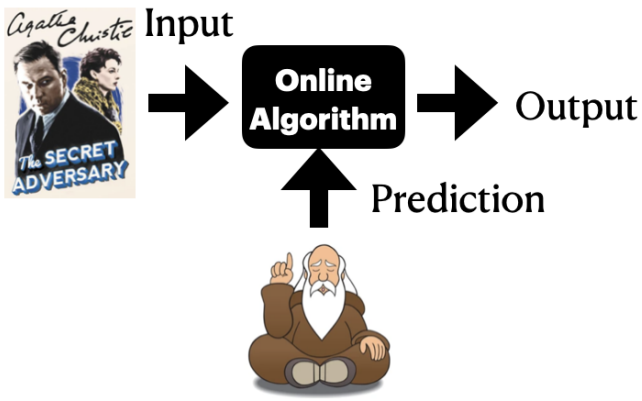


Figure 3.1: The design and approach of ML-advised algorithms.

The ML-augmented algorithms found two immediate directions of application just after the initial success with the competitive caching problem: to build better indexes for data structures [35, 37] and to design better job schedulers [71–73]. For example, Kraska et al. [37] worked on designing better index structures using ML models. Mitzenmacher [35] extended this idea and applied it formally to develop ***learned bloom filter***, an optimized data structure to answer set membership queries. In another line of research, Kumar et al. [71] used the ML augmentation to improve the classical non-clairvoyant scheduler,

where the source of uncertainty comes from the fact that the scheduler is not aware of the length of the job until it finishes. Mitzenmacher [73] extended this as a framework to formulate the price of errors in the prediction from the ML model for several queuing strategies. Lattanzi et al. [72] lead the research in another direction by proposing a robust quantity to be predicted by the ML oracle, which improves the performance of the scheduler. These approaches redesigned several online algorithms by augmenting them with ML oracles to improve their performance in the face of uncertainty.

Ski rental problem. The classical ***ski rental problem*** [1], a canonical example of the large class of online ***rent-or-buy problem***, has been of interest in this new avenue of research [38, 71, 78, 80]. The rent-or-buy problem represents a scenario where an online decision maker faces a situation where it has to optimize between two choices: one is to *rent*, with a small recurrent cost, and another is to *buy*, by paying up a comparatively large cost upfront with no expenditure thereafter. The uncertainty stems from the fact that the span of usage is unknown in advance. For instance, in the case of the ski rental problem, the skier is unaware of the length of the ski session until it ends. Her objective is to optimize between two choices: she either rents a ski at a unit price every day r buys the ski for a higher price to ski for free from then on. Intuitively, for a long span of usage, purchasing makes sense, but for the short term, renting is more optimized.

Kumar et al. [71] introduced ML-augmented online algorithms for the classical ski rental problem instance, where the ML oracle predicts the number of days to ski. Gollapudi and Panigrahi [38] extended this notion to multiple experts, where multiple ML oracles augment the traditional online algorithm and predict the future. Anand et al. [80] worked to answer another complimentary question in this context: whether we can redesign the ML algorithms to serve the online algorithms well in the context of an online rent-or-buy problem. Angelopoulos et al. [78] studied the impact of the advice that can be generated from a malicious or adversarial ML oracle.

The ski rental problem is well-studied in economics and computer science. Researchers studied several generalizations to the problem to imbibe real-life scenarios, such as deciding whether to buy or rent a house, making corporate decisions on whether to invest in a new office, data center, or rent utilities. Irani and Ramanathan [81] studied a variant of this problem where the rental price remains fixed, but the purchasing price fluctuates. Furthermore, Meyerson [82] proposed the ***parking permit problem*** that introduces multiple purchasing options with different time intervals; Khanafer et al. [83] extended this notion to rent optimization for cloud servers. Lotker et al. [84, 85] pro-

posed ***multi-slope ski rental problem***: the skier has to optimize among several choices for skis of separate rental and purchasing price. Zhang et al. [86] explored a generalization of the parking permit problem with multiple discount options available for the skis. However, in all these extensions, the price of any individual rental options is considered to be fixed.

Ski rental problem with a discount. In this paper, we design and evaluate algorithms for the ***ski rental problem with a discount***: the purchasing price is constant, but as the skier rents for a longer period, she gets a discount on the rental price.

El-Yaniv et al. [87] took the first step in this direction by introducing a fixed nominal interest rate on the rental price, where the purchasing price remains fixed. The interest effectively introduces a discount on the rental price where the discount is a fraction of the rent on the first day. The discount remains fixed with respect to time, i.e., the rental price decreases linearly. They performed the traditional competitive analysis [2, 4] of the problem. We further generalize the problem by considering the discount rate to fluctuate arbitrarily throughout the ski session rather than always being fixed. A preliminary version of this problem and the solutions were presented in the conference version of this paper in WALCOM 2022 [60], where the rental price could be presented as a non-increasing function. This article generalizes the problem further so that the rental price can have any arbitrary temporal fluctuations.

Results. Our goal is to improve the conventional online algorithms by designing algorithms with ML advice to solve the ski rental problem with a discount. We show in Lemma 1 that if the ML-advised algorithm blindly follows the ML oracle, the competitive ratio of the algorithm is unbounded. However, instead of following the oracle blindly, if the algorithm has a trust parameter $\lambda \in (0, 1]$, we show in Lemma 2 that the algorithm has an improved competitive ratio of $(1 + 1/\lambda)$ for any given prediction error. It reaches a competitive ratio $(1 + \lambda)$ when there is no prediction error. Finally, we extend the problem to the situation where two independent ML advisers are present. If one of the advisers is correct, we show in Lemma 3 that the algorithm is ϕ-competitive, where ϕ is the golden ratio (see section 3.5). Lemma 4 shows that otherwise, the competitive ratio of the algorithm is $\max\{\phi+1, 1/(\phi-1-r\eta/p)\}$; η is the prediction error, r is the actual rental price, and p is the purchasing price.

Chapter overview. The rest of the chapter is organized as follows. Section 3.2 explains the traditional ski rental problem, classical competitive analy-

sis, and the framework to analyze the classical online algorithms with machine learning augmentation. Section 3.3 introduces the ski rental problem with a discount, explains the classical algorithms to solve the problem. Section 3.4 designs a framework of the online algorithms with a single ML adviser. Section 3.5 explains the augmented algorithms with multiple ML advice. Finally, section 3.7 concludes by stating a few potential future applications of this work.

3.2 Preliminaries

In order to discuss the online problems and the existing and potential solutions, it is imperative to study the basics of machine learning (ML) and classical competitive analysis, and define a framework to analyze algorithms augmented by an ML oracle.

Classical competitive analysis. The classical competitive analysis [1, 2] is a tool to analyze the performance of an online algorithm. There is an adversary that decides the input sequence for an online problem. A conventional online algorithm makes decisions without any futuristic knowledge as each element from the input sequence arrives. We compare the performance of the online algorithm with the same of the optimal offline algorithm, which has the advantage of hindsight for all the sequences possible and finds a worst-case guarantee for the online algorithm [88]. For an optimization problem Π and a set of instances Σ_Π, τ is an input sequence where $\tau \in \Sigma_\Pi$, $OPT(\tau)$ is the cost incurred by the optimal offline algorithm, $ALG(\tau)$ is the cost incurred by an online algorithm, the online algorithm has a competitive ratio of α if for all possible input sequence τ, $ALG(\tau) \leq \alpha \cdot OPT(\tau)$.

Ski rental problem. In the ski rental problem [1], a skier's goal is to optimize by either renting a ski at a unit price every day or buying the ski for a higher price p units and ski for free from then on. The skier does not know the length of the ski season in advance and only knows it when the season ends. Intuitively, if the ski session lasts for x days and the skier rents the ski for the whole period, she would spend a total of $x \times 1 = x$ units. If $x > p$, the skier should buy the ski on the first day; otherwise, she should keep renting till the last day of usage. The optimal offline algorithm being a visionary, will do the same, thus incurring a cost of $\min\{x, p\}$.

The ski rental problem and the potential strategies for the skier are widely studied in the last two decades [1, 83, 84]. The well-known best deterministic strategy for the skier, the ***break-even algorithm***, is to rent the ski until the

cumulative rent does not exceed $p-1$ unit price and to buy the ski on the next day. This means as the ski is rented at a unit price each day, the algorithm will rent the ski for $(p-1)/1 = p-1$ days and purchase the ski on the pth day. The break-even algorithm is 2-competitive, and no other deterministic algorithm can do better than that.

Furthermore, Karlin et al. [1, 89] designed a randomized algorithm that is $(\frac{e}{e-1})$-competitive (roughly 1.58-competitive). The randomized algorithm finds an optimal value y so that the skier should rent for the first $(y-1)$ days and buy the ski on the day y, if needed. We have already observed that there is no advantage of buying the ski after day p. Hence, y shall be a value within the range of $\{1, 2, \ldots, p-1\}$ and the associated probabilities for y to have one of these values will be $Pr(y = i) = p_i$ where $i \in \{1, 2, \ldots, p-1\}$. To the best of our knowledge, it is the best online strategy for the skier; hence the algorithm is optimal among all the possible online algorithms.

Machine learning basics. This section explains the basics of a machine learning approach to solve a problem. In the ML problems, we have a ***feature space*** Σ, which says the salient attributes associated with each item and a set of corresponding ***labels***, Δ. An example is a tuple (σ, δ), where $\sigma \in \Sigma$ describes the specific attributes of the sample item, and $\delta \in \Delta$ provides the corresponding label [34, 35]. A ***hypothesis*** is a mapping (either deterministic or probabilistic) $h : \Sigma \to \Delta$, where $h \in H$, i.e. h belongs to a class of predictors or mappings, H. Depending on that, the predictor predicts δ ($\delta \in \Delta$) as the label of σ ($\sigma \in \Sigma$) with a deterministic or probabilistic strategy. The ***loss function*** is defined to quantify the error in the prediction of the machine learning hypothesis by $l : \Delta \times \Delta \to \mathcal{R}^{\geq 0}$. As the labels lie in the same metric space, we measure the absolute loss as $l_1(\delta, \hat{\delta}) = |\delta - \hat{\delta}|$ and squared loss as $l_2(\delta, \hat{\delta}) = (\delta - \hat{\delta})^2$, where the true label is $\hat{\delta}$.

Augmentation by machine learning oracle. This section revisits the framework to analyze an algorithm with ML augmentation developed by Lykouris and Vassilvitskii [34, 39]. As mentioned earlier, we assume a feature space Σ and a label space Δ and a predictor $h : \Sigma \to \Delta$ where $h \in H$ where H is a class of predictors. For an optimization problem Π, a given input sequence is $\tau = \{\tau_1, \tau_2, \ldots \tau_{|\tau|}\}$, where the length of the sequence is $|\tau|$, τ_i has feature $\sigma_i \in \Sigma$ and a label $\delta_i \in \Delta$, whereas the predicted label is $h(\sigma_i)$. The error in prediction or the loss function for the input sequence τ, $\eta_l(h, \tau)$, is defined as the difference between the predicted label and the true label.

$$\eta_l(h, \tau) = \sum_i l(\delta_i, h(\sigma_i)).$$

We define $H_l(\varepsilon)$ to be the class of ε-accurate predictors for loss function l if and only if h is ε-accurate $\forall\, h \in H_l(\varepsilon)$, i.e. $\eta_l(h,\tau) \leq \varepsilon \times OPT_{\Pi}(\tau)$ $\forall$ input τ, where the cost of the optimal offline algorithm is $OPT_{\Pi}(\tau)$. Algorithm ALG is ε-assisted if it has access to an ε-accurate predictor h. The competitive ratio of an ε-assisted algorithm ALG, $\alpha_{ALG,l}(\varepsilon)$, can be mathematically expressed as the maxima of all the cost ratios of the learning augmented online algorithm with ε-assisted predictor h compared to the optimal offline algorithm,$\alpha_{ALG,h}(\tau)$, for all ε-assisted predictors h, $\forall h \in H_l(\varepsilon)$ and all possible inputs τ.

$$\alpha_{ALG,l}(\varepsilon) = \max_{\tau, h \in H_l(\varepsilon)} \alpha_{ALG,h}(\tau).$$

The goal of the ML oracle is twofold: the consistency goal where it improves the performance of the online algorithm when the predictions by the ML oracles are fairly accurate and the robustness goal where the performance of the online algorithm does not degrade significantly as the predictions degrade [38, 71]. We define an algorithm ALG to be ***β-consistent*** if the algorithm works no worse than β times of the optimal offline algorithm when the best prediction has no error, i.e. $\alpha_{ALG,l}(0) = \beta$. On the other hand where the best case prediction has an error ε, ALG is ***γ-robust*** if $\alpha_{ALG,l}(\varepsilon) = O(\gamma(\varepsilon))$. These goals have to be met by the online algorithm without the knowledge of the quality of predictions. For the worst-case prediction, we define an algorithm ALG to be ***α-competitive*** if $\alpha_{ALG,l}(\varepsilon) \leq \alpha\ \forall\ \varepsilon$.

Machine learned binary search algorithm. A motivating example of ML augmentation is ML advised binary search. Given a sorted array $A : |A| = n$, and a query element q, the goal is to either find the index of q, $t(q)$, or state that $q \notin A$. The traditional binary search algorithm works as follows: compare the value of q to that of the middle element of A, and recurse on the correct half of A. After $O(\log n)$ probes, the method either returns $t(q)$ or ϕ.

The ML augmented binary search [34] predicts a position $h(q)$, immediately after one probe it is known if $h(q)$ is smaller or larger than $t(q)$. For instance, if $h(q) < t(q)$, we probe $h(q)+2, h(q)+4, h(q)+8, ...$, when we find an element which is greater than q, run a binary search on the last interval. If we find q, we return the position; otherwise, we return ϕ. For error in prediction is $\varepsilon = |h(q) - t(q)|$, the expected number of probes $O(E[\log(\varepsilon)]) \leq \log E[\varepsilon]$. For example, if the average prediction error is $O(\log n)$, the ML augmented binary search performs asymptotically better than the traditional binary search in expectation by achieving an improvement from $O(\log n)$ to $O(\log \log n)$.

3.3 Ski rental problem with a discount

This section introduces the ski rental problem with a discount in a mathematical setting and explains the optimal offline algorithm as well as the optimal deterministic online algorithm without any ML advice.

In the mathematical model of the ski rental problem with a discount, the ***purchasing price*** of the ski is p and ***actual rental price*** of the ski for each day is r. However, the rent has a daily discount that fluctuates with time: the ***effective rental price*** is less than the actual rental price. We simulate the effective rental price as a function $rent(.)$, where $rent(t)$ denotes the effective rental price for the tth day. The effective rental price can fluctuate arbitrarily with respect to time t and $rent(t) \in [0, r] \, \forall \, t$. The skier makes a non-revocable decision, either to rent or to buy, at the beginning of each day. The actual number of days of snow in the ski season is x, and it is unknown to the skier until it does not snow anymore. The goal of the skier is to minimize the total amount paid throughout the ski session by making the right decision when to buy the ski in an online manner.

Note that this is a generalization of the traditional ski rental problem. Due to many economic constraints, the rent may fluctuate. As long as the fluctuation of the rent is upper-bounded by a predefined rent r, all the results of this paper are valid.

We will refer to the effective rental price, $rent(.)$, as ***rent*** for the rest of the paper. The way to calculate the total rent up to the tth day is to add the rents from the first day till the tth day; we refer to this as $rent(1 \dots t)$.

$$rent(1 \dots t) = \sum_{i=1}^{t} rent(i).$$

For any given price m units, we define t_m such that $rent(1 \cdots t_m - 1) < m$ and $rent(1 \cdots t_m) \geq m$. Hence, if the skier rents the ski instead of purchasing, on the t_p^{th} day, the total rent paid exceeds the purchasing price. The highest possible rental price is r.

Optimal offline algorithm. The optimal offline algorithm OFFLINE knows ski session length x. If $x < t_p$, OFFLINE keeps renting the ski for the whole ski session, otherwise, OFFLINE purchases the ski on the first day of the session. Hence, the cost of OFFLINE can be written as $OPT = \min\{rent(1 \dots x), p\}$.

Break-even algorithm. The best deterministic strategy, the break-even algorithm, works the same way as it does in the case of the conventional ski rental problem. It rents the ski till $(t_p - 1)^{th}$ day, and purchases the ski on the

t_p^{th} day. Note that the worst-case appears for $x = t_p$. In this case, $OPT = p$ and $ALG = rent(1 \dots t_p - 1) + p \leq 2p$. This shows that the break-even algorithm has a competitive ratio of 2 for the ski rental problem with a

3.4 Algorithms with a single ML adviser

This section introduces ML-augmented online algorithms to solve the ski rental problem with a discount. First, we propose a naive ML-advised algorithm. Next, we redesign the algorithm with a "trust" parameter and evaluate the improved robustness.

Naive ML-advised algorithm. First, we present Algorithm 1 based on an ML advice [71]. The ML-based predictor estimates the number of days it is going to snow as y. Such a prediction can be made reasonably well, for instance, by building models based on weather forecasts and the past behavior of other skiers. The error in the prediction can be quantified as $\eta = |y - x|$. Algorithm 1 is 1-consistent when $\eta = 0$, i.e. the algorithm performs optimally when the prediction is correct. However, the algorithm is not robust; hence the competitive ratio is unbounded and can be arbitrarily large for poor predictions. The performance of Algorithm 1 is analyzed by Lemma 1.

Algorithm 1 Naive ML Augmented Algorithm

Require: Sequence of past input $X = \{x_i\}$.
1: **function** NaiveML(X, current input x)
2: Train the ML Oracle using X.
3: ML Oracle predicts y.
4: **if** $y \geq t_p$ **then**
5: Purchase the ski on the first day.
6: **else if** $y < t_p$ **then**
7: Keep renting the ski.
8: **end if**
9: Train the loss.
10: **end function**

Lemma 1. *Let ALG be the cost of the naive ML-advised algorithm, OPT be the cost of the optimal offline algorithm, η be the error in the prediction of the ML model, and r be the actual rental price of the ski. For all the possible inputs, $ALG \leq OPT + r\eta$.*

Proof. In case of $y \geq t_p$, and $x \geq t_p$, $ALG = p = OPT$. On the other hand, when $y < t_p$ and $x < t_p$, $ALG = OPT = rent(1 \ldots x)$. That is ML augmented algorithm performs as well as the optimal offline algorithm in both of these cases. Next, we explain the other two possibilities of x and y concerning t_p. In the case $y \geq t_p$ and $x < t_p$, $ALG = p$ and $OPT = rent(1 \ldots x)$. In case of, $x = t_p - \varepsilon$, $OPT = rent(1 \ldots t_p - \varepsilon)$ and $\eta = y - x \geq \varepsilon$.

$$ALG = p = rent(1 \cdots t_p) = rent(1 \cdots x) + rent(x + 1 \cdots t_p).$$

$$\implies ALG = OPT + rent(x + 1 \ldots t_p) \leq OPT + r \cdot \varepsilon \leq OPT + r \cdot \eta.$$

Where $y < t_p$ and $x \geq t_p$, $ALG = rent(1 \cdots x)$ and $OPT = p$. In case of $x = t_p + \varepsilon$, the error in prediction, η is $\eta = |x - y| = |t_p + \varepsilon - y| \geq \varepsilon$ (as $y < t_p$). Hence, $ALG - OPT = rent(1 \cdots x) - p \leq rent(t_p + 1 \cdots t_p + \varepsilon) \leq r\varepsilon \leq r\eta \implies ALG/OPT = 1 + r\eta/OPT$.

□

Corollary 1. *Let ALG and OPT be the cost of the naive ML-advised algorithm and the optimal offline algorithm respectively, and the error in the ML prediction model be η where $\eta = k.OPT$. The cost of the naive algorithm, ALG, is unbounded.*

Proof. For $x \geq t_p$ and $y < t_p$, if $\eta = x - y = k \cdot OPT = k \cdot b$ where $k \geq 1$, ALG becomes unbounded as k grows.

$$ALG < k \cdot OPT^2 + OPT < k \cdot b^2 + b, \text{ and } k \text{ is unbounded.}$$

□

Handling the unbounded error. As Algorithm 1 may incur an unbounded competitive ratio, we may be allured to solve the situation in the following manner. For $y < t_p$, instead of continuing to rent the ski, we can "fall back" to the traditional algorithm by purchasing the ski on the t_{2p}^{th} day. Following the definition of t_x, $rent(1 \ldots t_{2p} - 1) < 2 \cdot p$ and $rent(1 \ldots t_{2p}) \geq 2 \cdot p$. Interestingly, when $x \geq t_{2p}$, this strategy helps the algorithm to achieve a 2-robustness — the performance does not degrade in an unbounded manner anymore. However, the disadvantage in doing so is that for any $x > t_{2p}$ and $x < t_{3p}$. In this case, for $y < t_p$, Algorithm 1 incurs a cost ALG where $2 \cdot p \leq ALG \leq 3 \cdot p$. On the other hand, our modified algorithm purchases the ski on the t_{2p}^{th} day when $y < t_p$, incurring a cost $3 \cdot p$.

ML-advised algorithm with trust parameter. The shortcoming of the naive ML-advised algorithm is that it puts too much trust in the ML

oracle. Hence, as the ML oracle fails and incurs a large prediction error, the performance degrades harshly leading to a high competitive ratio in the worst-case input. As per our definition of robustness in section 3.2, the naive ML-advised algorithm is not robust; hence we aim to design a more robust algorithm.

Algorithm 2 ML advised algorithm with trust parameter

Require: Sequence of past input $X = \{x_i\}$.
  function MLTrust(X, current input x)
    Train the ML Oracle using X.
    Trust factor: $\lambda \in (0, 1]$.
    if $y \geq t_p$ **then**
      Purchase on the $t_{\lambda p}^{th}$ day.
    else if $y < t_p$ **then**
      Purchase on the $t_{p/\lambda}^{th}$ day.
    end if
    Train the loss.
  end function

Algorithm 2 improves by acquiring more robustness over Algorithm 1 while not losing much in terms of consistency. It introduces a hyperparameter, *trust factor* $\lambda \in (0, 1]$; this parameter introduces a tradeoff between the traditional algorithm and the ML advice. This helps the algorithm to acquire a tradeoff between consistency and robustness. Note that $\lambda \to 0$ refers to greater trust in the ML predictor. It leads to better consistency, i.e. the algorithm performs better than the naive ML augmented algorithm when the prediction is good. On the other hand, $\lambda \to 1$ refers to the case when the algorithm has lesser trust in the ML predictor. Algorithm 2 retains a good consistency compared to the naive algorithm while achieving more robustness. That is, the algorithm performs better when the quality of prediction is not so good. The performance of Algorithm 2 is analyzed by Lemma 2.

Lemma 2. *Let ALG be the cost of the ML advised algorithm with the trust parameter $\lambda \in (0, 1]$, OPT be the cost of the optimal offline algorithm, η be the error in the prediction of the ML adviser, and r be the actual rental price of the ski for each day. Then, $ALG \leq OPT \cdot (1 + \frac{1}{\lambda})$.*

Proof. In case where $y > t_p$ and $x < t_{\lambda p}$, $ALG = rent(1 \ldots x) = OPT$. In both the cases where $x < t_{\lambda p}$ and $t_{\lambda p} \leq x < t_p$ when $y < t_p$, $ALG = rent(1 \ldots x) = OPT$. This means Algorithm 2 pays the same cost as the optimal offline algorithm in these two cases. In the case where $t_{\lambda p} \leq x < t_p$

and $y \geq t_p$, $ALG = rent(1 \dots t_{\lambda p} - 1) + p$ and $OPT = rent(1 \dots x)$. The worst-case appears at $x = t_{\lambda p}$.

$$ALG - OPT = p - rent(t_{\lambda p}) \leq p \leq \frac{1}{\lambda}(\lambda p) \leq \frac{1}{\lambda} rent(1 \dots t_{\lambda p}) \leq \frac{OPT}{\lambda}.$$

This case is very useful to explain that if the algorithm has high trust on the ML oracle, it buys the ski on the first day instead of $t_{\lambda p}$. In the case where $t_p \leq x \leq t_{p/\lambda}$ and $y < t_p$, $ALG = rent(1 \dots x - 1) + p$ and $OPT = p$. The worst-case appears at $x = t_{p/\lambda}$.

$$ALG - OPT \leq rent(1 \dots t_{p/\lambda} - 1) \leq p/\lambda \implies ALG = OPT(1 + \frac{1}{\lambda}).$$

For $t_p \leq x \leq t_{p/\lambda}$ and $y \geq t_p$, $ALG = rent(1 \dots t_{\lambda p} - 1) + p$, and $OPT = p$.

$$ALG - OPT = rent(1 \dots t_{\lambda p}) \leq \lambda.p \leq \lambda.OPT \implies \frac{ALG}{OPT} \leq 1 + \lambda < 1 + \frac{1}{\lambda}.$$

For $x > t_{p/\lambda}$ and $y < t_p$, $ALG = p + rent(1 \dots t_{p/\lambda} - 1)$, and $OPT = p$.

$$ALG \leq OPT + rent(1 \dots t_{p/\lambda}) \leq OPT + \frac{p}{\lambda} \leq OPT(1 + \frac{1}{\lambda}).$$

In the case $x > t_{p/\lambda}$ and $y \geq t_p$, $ALG = p + rent(1 \dots t_{\lambda p} - 1)$, and $OPT = p$. Hence, $ALG \leq OPT(1 + \lambda) \leq OPT(1 + \frac{1}{\lambda})$ as $\lambda \leq 1$. □

Corollary 2. *Let ALG be the cost of the ML-advised algorithm with the trust parameter $\lambda \in (0, 1]$, OPT be the cost of the optimal offline algorithm, the prediction of the oracle is equal to the actual number of days of snow, and r be the actual daily rent. Then, $ALG \leq OPT \cdot (1 + \lambda)$.*

Proof. In case the prediction is correct, i.e., $x = y$, Algorithm 2 performs as well as the optimal offline algorithm except in one case. For $y = t_{p/\lambda}$, $ALG \leq p + \lambda \cdot p = (1 + \lambda)p$ and $OPT = p$. Hence, $ALG \leq OPT(1 + \lambda)$, i.e., Algorithm 2 is $(1 + \lambda)$-consistent. □

Hence, from Algorithm 2 and Corollary 2 we observe that there is a tradeoff between the consistency and robustness of the ML-advised algorithm with respect to the trust parameter λ. By setting the trust parameter, either we can expect more consistency from the algorithm, or we can have more robustness. Figure 3.2 shows how the corresponding values of consistency and robustness react to the change of each other for Algorithm 2. The performance of the algorithm is understood through the next two examples by setting the trust

parameter, λ, with two specific values. In the first case, we choose $\lambda = 0.5$. In the second example, we select the trust parameter λ based on the golden ratio. We solve the equation $b/z = b + z/b$, and each side is equal to the golden ratio, $\phi = (\sqrt{5} + 1)/2$ (roughly equals 1.618). We choose $\lambda = \phi - 1$.

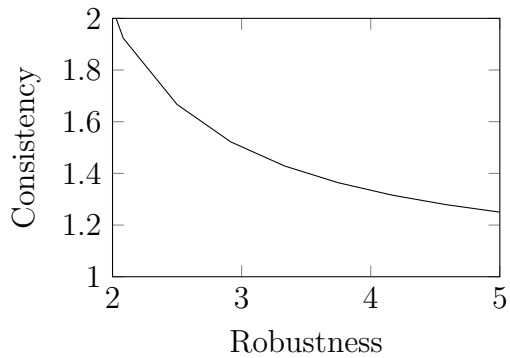


Figure 3.2: Robustness vs. Consistency tradeoff curve for the ML advised algorithm with a trust parameter.

Example 1. *Let, Algorithm 2 have the trust parameter* $\lambda = 0.5$. *Following Lemma 2 and Corollary 2, Algorithm 2 is* 1.5*-consistent and* 3*-robust when compared with the optimal offline algorithm.*

Example 2. *Let, Algorithm 2 have the trust parameter* $\lambda = \phi - 1$, *where* ϕ *is the golden ratio. Following Lemma 2 and Corollary 2, Algorithm 2 is* ϕ*-consistent and* $(\phi+1)$*-robust when compared with the optimal offline algorithm.*

The performance of Algorithm 2 can be improved by choosing λ dynamically, i.e., varying λ based on the algorithm's performance with the historical input sequence. As the predictor trains more, the performance will likely improve with time. Hence, Algorithm 2 can put more trust in the prediction. Although this article focused on choosing λ deterministically and keeping it fixed throughout, choosing λ dynamically and adaptively based on the prediction quality is one of the exciting future directions.

3.5 Algorithms with two ML advisers

This section explains the variant of the problem where multiple independent ML advisers are present. Gollapudi and Panigrahi [38] showed that the presence of more advisers helps to improve the performance as the chance of good

predictions increases. They also observed that as the number of ML advisers increases, the competitive ratio gets better, but the analysis remains the same. In this paper, we will restrict ourselves to the analysis where two advisers are present simultaneously, leading to two different predictions. ALG has access to 2 predictors with the predictions y_1 and y_2. Let's say, the prediction y_1 is closer to the actual number of days of snow, i.e., $|y_1 - x| < |y_2 - x|$. We define η as the minimum of the two absolute errors in the predictions by the two predictors, i.e., $\eta = \min\{|y_1 - x|, |y_2 - x|\}$. ALG is γ-competitive if and only if $ALG \leq \gamma \cdot OPT$.

One of the predictions is correct. The first scenario we study is where one of the two predictions is correct. The challenge here is to choose the correct one from one of these two predictions without the knowledge of its identity. Let us consider, the two predictions are y_1 and y_2, where $y_1 \geq y_2$, and the naive strategy is to blindly follow one of them. In case of $y_1 \geq y_2 \geq t_p$, the algorithm buys the ski on the first day, i.e., $ALG = OPT$. On the other hand, when $y_2 \leq y_1 < t_p$, the algorithm keeps renting the ski forever, resulting in $ALG = OPT$ again. In case of the other two possibilities, $y_1 \geq t_p$ and $y_2 < t_p$. If y_1 is correct and the algorithm blindly follows y_2, it keeps renting and the competitive ratio in this case becomes $\gamma = \frac{rent(1 \cdots y_1)}{p}$. Otherwise, if y_2 is correct and the algorithm trusts y_1, it buys the ski on the first day, leading to the competitive ratio, $\gamma = \frac{p}{rent(1 \ldots y_2)}$. In both the latter cases, γ is unbounded. We approach to modify the strategy so that when y_1 is correct and we trust y_2, instead of keeping renting forever, we rent till y_2 and then buy.

$$\text{Competitive ratio, } \gamma = \frac{rent(1 \ldots y_2) + p}{p} \leq 2.$$

Hence, although we get rid of the unbounded competitive ratio, we make no improvement from the traditional online algorithm. Necessarily, the constraint here is that the algorithm should make the decision to buy before y_2; in fact, it is the best to buy on the first day itself.

The solution lies in striking a balance between the two predictions and finding a tradeoff between them. Gollapudi and Panigrahi [38] developed such a strategy based on the golden ratio, ϕ. We define t_ϕ such that $rent(1 \ldots t_\phi - 1) < (\phi - 1)p$ and $rent(1 \ldots t_\phi) \geq (\phi - 1)p$. The algorithm decides to buy based on the value of t_ϕ as shown in Algorithm 3.

Lemma 3. *Let the cost of the augmented algorithm with two ML advisers be ALG and one of the advisers is correct in the prediction, OPT be the cost of*

Algorithm 3 ML advised algorithm with two advisers

Require: Sequence of past input $X = \{x_i\}$.
  function TwoAdviser(X, current input x)
    Train the ML oracle using X
    Predictions are y_1 and y_2 where $y_1 > y_2$.
    if $y_1, y_2 \geq t_p$ **then**
      Purchase on the first day.
    else if $y_1, y_2 < t_p$ **then**
      Keep renting.
    else if $t_\phi \leq y_2 \leq t_p$ and $y_1 > t_p$ **then**
      Purchase on the first day.
    else if $y_2 < t_\phi$ and $y_1 > t_p$ **then**
      Rent till $y_2 - 1$, buy on the y_2^{th} day.
    end if
    Train the loss.
  end function

the optimal offline algorithm, and ϕ be the golden ratio. Then,

$$ALG < \phi \cdot OPT.$$

Proof. The worst-case may have two possibilities. Firstly, when $y_2 = t_\phi$ and y_2 is correct, we get the following bound.

$$\gamma = \frac{p}{rent(1 \ldots t_\phi)} \leq \frac{p}{(\phi - 1)p} = \phi.$$

Finally, for $y_2 < t_\phi$ and y_1 being correct, Algorithm 3 holds ϕ-competitiveness.

$$\gamma = \frac{rent(1 \ldots y_2) + p}{p} \leq \frac{(\phi - 1)p + p}{p} = \phi.$$

□

Both predictions have non-zero errors. This section generalizes the problem with two ML advisers to the case where both the predictors have non-zero prediction errors. This represents a more realistic situation where we assume that the best prediction has an error η. We modify Algorithm 3 and design Algorithm 4 to avoid unbounded error for the case $y_1, y_2 < t_p$. Because, in this case Algorithm 3 keeps renting the ski forever and the optimal offline algorithm purchases the ski if $x \geq t_p$. Next, we evaluate the performance of

Algorithm 4 and observe the performance bounds. The predictions of the two experts are y_1 and y_2 where $y_2 \leq y_1$.

Algorithm 4 ML advised algorithm with two erroneous advisers

Require: Sequence of past input $X = \{x_i\}$.
  function TwoErroneousAdvisers(X, current input x)
    Train the ML oracle using X.
    Predictions are y_1 and y_2 where $y_1 > y_2$.
    if $y_2 \geq t_\phi$ **then**
      Purchase on the first day.
    else if $y_2 < t_\phi$ **then**
      Rent till $y_2 - 1$ and buy on the y_2^{th} day.
    end if
    Train the loss.
  end function

Lemma 4. *Let ALG be the cost of the augmented algorithm with two ML advisers where both the advisers have prediction errors. The best predictor has an error η, the actual rental price of the ski is r, and the purchasing price of the ski p. OPT is the cost of the optimal offline algorithm and ϕ is the golden ratio. Then,*

$$ALG \leq \max\{\frac{OPT}{\phi - 1 - r\eta/p}, (\phi + 1)OPT\}.$$

Proof. For $y_1, y_2 < t_p$, the worst-case appears when $x = y_1 + \eta \geq t_p$. This means the optimal offline algorithm buys the ski on the first day, whereas Algorithm 4 rents till $(t_\phi - 1)$ days and buys on the next day.

$$\frac{ALG}{OPT} = \frac{rent(1 \ldots t_\phi - 1) + p}{p} = \frac{(\phi - 1)p + p}{p} = \phi.$$

For $y_1, y_2 \geq t_p$, $y_2 = t_p$ and $x = y_2 - \eta < t_p$, optimal algorithm keeps renting the ski and Algorithm 4 buys the ski on the first day.

$$\frac{ALG}{OPT} = \frac{p}{rent(1 \ldots y_2 - \eta)} < \frac{p}{p - rent(t_p - \eta \ldots t_p)} < \frac{p}{p - r\eta} = \frac{1}{1 - r\eta/p}.$$

The other possibilities are where $y_1 > t_p$, $y_2 \geq t_\phi$ and the actual number of days $x = t_\phi - \varepsilon$. Hence, $\eta \geq \varepsilon$.

$$\frac{ALG}{OPT} = \frac{p}{rent(1 \ldots t_\phi - \varepsilon)} \leq \frac{p}{rent(1 \ldots t_\phi) - r\eta} \leq \frac{1}{\phi - 1 - r\eta/p}.$$

In case of $y_1 > t_p$, $y_2 < t_\phi$, and $x = t_\phi + \varepsilon < t_p$, optimal algorithm keeps renting the ski and Algorithm 4 rents till $(t_\phi - 1)$ days and buys on the next day.

$$\frac{ALG}{OPT} \leq \frac{rent(1 \ldots t_\phi) + p}{rent(1 \ldots t_\phi + \varepsilon)} \leq \frac{(\phi - 1)p + p}{(\phi - 1)p} \leq 1 + \frac{1}{(\phi - 1)} = \phi + 1.$$

This leads us to a competitive ratio $\max\{\frac{1}{\phi - 1 - r\eta/p}, \phi + 1\}$. □

Example 3. *Let the actual daily rent of the ski be $r = 1$, and the purchasing price of the ski be $p = 10$ where $p/r = 10$. The discount is 2% of the purchasing price till the 5th day, and 1% then onward for each day. Hence, $t_p = 11$, i.e. $rent(1 \ldots 10) < p$ and $rent(1 \ldots 11) \geq p$. Also, $t_\phi = 7$, i.e. $rent(1 \ldots 6) < (\phi - 1)p$ and $rent(1 \ldots 7) \geq (\phi - 1)p$, where ϕ is the golden ratio. Let y_1 and y_2 be the two predictions by two ML oracles.*

If either y_1 and y_2 is correct where $y_1 \geq y_2$, Algorithm 3 is ϕ-competitive (following by Lemma 3). If both of the predictions are erroneous, where y is the better prediction between y_1 and y_2 as $|y - x| = \min\{|y_1 - x|, |y_2 - x|\}$. If $\lfloor 0.5x \rfloor \leq y \leq \lfloor 2x \rfloor$, we observe for the previous example that Algorithm 4 is 2.58-competitive. For $y_2 = 4$, $y_1 > 11$, and $x = 7$, $OPT = 6.61$ and $ALG = 6.61 + 10$. Hence, $ALG/OPT = 2.513$. For $y_2 = 7$, $y_1 > 11$, and $x = 4$, $OPT = 3.88$ and $ALG = 10$. Hence, $ALG/OPT = 2.577$.

3.6 Experimental evaluation

We evaluate the performance of the algorithms discussed in Section 3.3 by two numerical simulation-based experiments. In the first experiment, we vary the training epoch of the ML oracle and observe the competitive ratio. In the second experiment, we vary the error in prediction from the ML oracle and observe the competitive ratio. We evaluate the average performance of the best deterministic algorithm and juxtapose that with the performance of the algorithm 1, 2, and 4.

Setup. In this section, we explain the experimental details. For the numerical experiments, we design the problem instances with the purchasing price, $b = 10000$, and initial rent, $r = 100$. We design a non-increasing function $rent(x)$ that denotes the rent of the ski on day x. We design a function $rent(x)$ so that within $t_b = 200$ days the cumulative rent of the ski goes over b. We generate x from a uniformly distribution of the set of integers from $[1, 5t_b]$, hence $x \in [1, 1000]$. For each experiment, we generate an input sequence of length 1000 and amortize the competitive ratio for the algorithms we evaluate.

In the first experiment, we use a long short-term memory (LSTM) based deep neural network (DNN) as the ML oracle, and we vary the training epochs up to 1000. In the second experiment, we assume the prediction of the number of days to ski is $y = x + \eta$, where the error in prediction, η, is generated from a normal distribution with a mean zero and standard deviation σ. We vary the standard deviation σ up to 500.

We evaluate the performance of four algorithms: the break-even algorithm (best deterministic algorithm), naive ML augmented algorithm (algorithm 1), ML augmented algorithm with the trust parameter (algorithm 2), and ML augmented algorithm with two advisers (algorithm 4). We fix the trust parameter, $\lambda = 0.5$ so that the ML-advised algorithm has a robustness of 3. $\lambda = 1$ signifies the case where we do not consider the prediction of the ML oracle and $\lambda = 0$ bespeaks the case where we completely trust the ML predictions.

Long Short-Term Memory (LSTM) networks. Long Short-Term Memory (LSTM) networks, a type of recurrent neural network (RNN), are designed to model temporal sequences and their long-range dependencies more effectively than traditional RNNs. Introduced by Hochreiter and Schmidhuber in 1997 [90], LSTMs address the vanishing gradient problem that plagues standard RNNs by incorporating a more sophisticated architecture with gating mechanisms. An LSTM cell consists of three primary gates: the input gate, the forget gate, and the output gate, which regulates the flow of information into, within, and out of the cell, respectively. These gates enable LSTMs to retain relevant information over long sequences while discarding non-essential data, making them particularly suitable for tasks such as speech recognition, language modeling, and time series forecasting.

The mathematical formulation of an LSTM cell can be described as follows. Let $\mathbf{x}_t$ represent the input vector at time step t, $\mathbf{h}_{t-1}$ the hidden state of the previous time step, and $\mathbf{c}_{t-1}$ the cell state of the previous time step. The forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, and output gate $\mathbf{o}_t$ are computed using the following equations:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f),$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i),$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o).$$

where σ denotes the sigmoid function, W and U are weight matrices, and b represents bias vectors. The cell state c_t is updated as follows:

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c),$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

Finally, the hidden state $\mathbf{h}_t$ is computed using the updated cell state:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).$$

The use of gating mechanisms allows LSTM networks to maintain and adjust information over long sequences, addressing the limitations of traditional RNNs. This makes LSTMs highly effective for sequential data tasks, where capturing long-term dependencies is crucial.

Results. Figure 3.3 shows that after 500 epochs, as the ML oracle gets reasonably trained and the prediction becomes sufficiently accurate, the competitive ratio becomes lesser than the conventional break-even algorithm.

Figure 3.3 shows that for small prediction errors the ML augmented algorithms perform better than the classical algorithms. Even for larger prediction errors, the ML-augmented algorithms have lesser prediction errors, especially for the algorithms that do not have an unbounded competitive ratio.

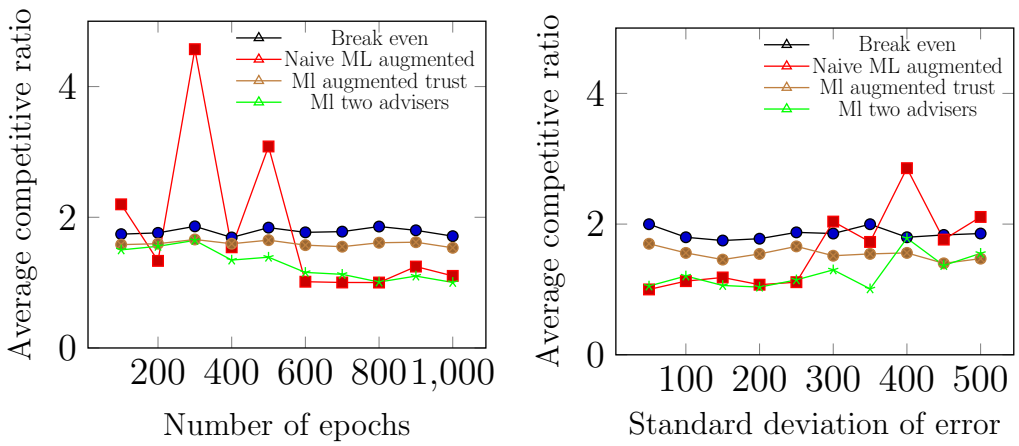


Figure 3.3: Average competitive ratio of the algorithms for (i) varying training epochs and (ii) varying the standard deviation of the prediction error.

Computational overhead from ML training. The ML-advised algorithms utilize predictions generated by ML models to guide decision-making, thereby enhancing average-case performance while maintaining robust worst-case guarantees as demonstrated in prior works [34, 35, 37]. However, a critical consideration in the practical deployment of ML-advised algorithms is the computational overhead associated with ML training. This overhead, which can be significant, is necessary for generating the high-quality advice that these algo-

rithms rely on. As a result, the computational complexity of the advisor must be taken into account when evaluating the overall efficiency of ML-advised algorithms. For the theoretical bounds on performance to hold, it is essential that the advisor operates in constant time, i.e., $O(1)$. If the computational cost of training and generating advice exceeds this, the practical benefits of using ML-advised algorithms may be undermined, thus invalidating the theoretical performance bounds that assume minimal advisor overhead.

3.7 Conclusion

We proposed the ski rental problem with a discount in a mathematical setting and designed online algorithms with single and multiple machine-learning advice. These algorithms use the ML advice to improve the performance on the average-case inputs than the traditional online algorithm. However, in order to do that, it does not become infinitely worse than the optimal offline algorithm in case of worst-case inputs. We showed the tradeoff between consistency and robustness in the context of the ski rental problem with a discount. The approach is very useful for beyond-worst-case analysis for several natural online problems. The next step of this approach is to investigate the performance gains for ML augmented algorithms in systems. It will also be exciting to explore the effect of several distributions of the prediction error of the ML advisers on the competitive ratio. Another interesting discussion should be how to prove the lower bound for the ML-advised algorithms for the ski rental problem with a discount. There are also several exciting directions regarding randomization. Choosing λ in a randomized manner and keeping the rest of the algorithm deterministic will also be an exciting approach.

Chapter 4

ML-Advised List Update Problem

In this chapter, we will evaluate whether the recipe we provide in Chapter 3 can be applied to design ML-advised algorithms for another challenging online computing problem, the list update problem. We are going to delve into the problem details in the next sections.

This work is done in collaboration with Dr. Helen Xu and Dr. Shahin Kamali [91].

Abstract. We study the list update problem under the ML-advised algorithm framework. We augment the traditional list update algorithm with advice from an advisor, which may or may not be from a Machine Learning (ML) oracle or advisor. Note that this advice might be arbitrarily poor. We first show that for lists of two items, correct predictions about the next request ensure optimality via a simple algorithm that only uses free exchanges. Next, we show that for lists with three items, knowing when the next time each item would be requested is not sufficient to make the optimal decision anymore. Finally, we propose an algorithm, namely the Penalty Distance Algorithm (PDA), which has access to predictions concerning the next two occurrences of the requested item in the request sequence. Our conjecture is that when the prediction is correct, using the proper potential function for the analysis, it can be shown that PDA's competitive ratio is ≤ 1.6.

4.1 Introduction

In the ***list update problem*** [4], the input is an online sequence of access requests to items in a linked list. To serve a request to an item at position i,

an online algorithm makes a linear scan of the list until it finds the item; this linear scan has an access cost of i. After accessing the item, the algorithm can move the item to a position closer to the front using a ***free exchange*** that has no cost. Moreover, it can make any number of ***paid exchanges***, each swapping two consecutive items at a unit cost.

The list update problem. Given an input request sequence, a list update algorithm's objective is to maintain the list in a way that minimizes the total cost (i.e., the sum of costs for all accesses and paid exchanges). A free exchange moves the item currently being accessed anywhere closer to the front (for instance, at the front) of the list with a cost of 0. On the other hand, a paid exchange swaps the positions of any two consecutive items in the list with a cost of 1. List update has a long history of study (Interested readers are welcome to check out the survey by Kamali et al. [92]), and was one of the two problems studied by Sleator and Tarjan in their seminal paper [4] that defines competitive analysis as a worst-case measure for the analysis of online algorithms.

Relationship with the caching problem. The list update problem is sometimes compared to the ***caching problem***, which asks for maintaining a fast but small memory (a cache) in a way to minimize the total number of requests to memory pages outside of the cache (number of faults). The online caching algorithm is given a sequence of page requests that arrive one at a time, and the algorithm has to serve the requests from a cache of fixed limited size k. If the requested page is already in the cache, the algorithm does not incur any cost to serve the request (***cache hit***). On the other hand, if the requested page is not in the cache (***cache miss***), the page must be read from a slow memory of unlimited size with a cost of 1. If the cache is full, a page from the cache has to be written back to the slow memory to make room for the requested page. The online algorithm must choose a page to be evicted in a way so that the number of cache misses is minimized for an input sequence. The performance of the online algorithm is compared with the optimal offline algorithm that knows the future page requests and always evicts the page that will be requested the furthest in the future. We refer to the maximum ratio of the total cost incurred by the online algorithm and the optimal offline algorithm as the ***competitive ratio*** of the online algorithm for every possible input sequence. The algorithm designers approach to minimize the competitive ratio relative to the optimal offline algorithm.

For the list update problem, ***Move To Front (MTF)***, which moves an accessed item to the front of the list (using a free exchange), is 2-competitive [4],

and it is the best that a deterministic algorithm can achieve [93]. As each request arrives, MTF serves it, then moves the item to the front of the list using a free exchange. Relatably, the best deterministic algorithm for caching is ***Least Recently Used (LRU)***, which evicts the least recently used page from the cache whenever eviction is needed and has a competitive ratio of k for a cache of size k [4]. In case of a cache miss, if the cache is full, the item requested furthest in the past is evicted. LRU is $O(k)$-competitive for a cache of size k [4]. Essentially, LRU and MTF both prefer to keep the recently used items handy.

4.2 List update algorithm with ML advisor

We investigate the list update problem in light of the recent development for the caching problem under a prediction model. In this line of research, [34, 68, 72] upon a request to a page x, a prediction is made available concerning the next time x is requested in the input sequence, that is, the index of the next request to x in the input sequence. We study similar predictions for the list update problem. Our starting observation is that, unlike the paging problem where correct predictions about the next request ensure optimality via a simple algorithm that evicts the page that is requested furthest, the list update problem is NP-hard [94], and we cannot hope for optimal solutions even if predictions are correct.

Nevertheless, we get hopeful from an initial success: For lists of two items, correct predictions about the next request ensure optimality via a simple algorithm that only uses free exchanges: for a list formed by items a and b, whenever there is a request to an item, say b, in the second position, move it to the front if and only if the next request to b appears earlier than the next request to a. Let the initial state of the list be $\{a, b\}$ and the requested item be b. The online list update algorithm has to decide whether to move b to the front with free exchange. If the next request is b, b should be moved to the front, whereas if the next request is a, b should not be moved. Hence, it is sufficient to know only the next request to decide optimally between either of the choices. We take a tumble with a list only with three items; knowing when the next time each item would be requested is not sufficient to make the optimal decision anymore. Simple predictions about the next request reach their limit for lists of length 3. In particular, when there is a request to the item at the third position in a list of length 3, an optimal algorithm sometimes needs to use a paid exchange to swap the first two items and then move the requested item between the first two using a free exchange [95, 96]. To make such decisions, however, the algorithm requires the next *two* requests for

each item. In other words, one can achieve optimality with correct predictions about the next two times each item is requested; but predictions about the next request is not sufficient.

When the list is $\{a, c, b\}$ and b is requested, the online algorithm has to decide the set of paid and free exchanges to perform. If the continuation of the request sequence is $\langle c, c, b, a, b, a, \cdots \rangle$, item c should be before b, and the relative ordering between a and b does not matter. In this case, the optimal algorithm would not perform any paid or free exchanges (cost = 0). However, if the online algorithm with prediction has access to only the next occurrence, the relative order of the items is $\langle c, b, a, \cdots \rangle$, i.e., $c < b$ and $a > b$. Hence, it will perform a paid exchange between a and c, and use the free exchange to make the list $\{c, b, a\}$ (cost = 1). However, we observe that having futuristic knowledge, even when the knowledge has no error, about the next occurrence of the requested page in the request sequence is not enough for the list update problem. Even in an easy scenario, where the length of the list is 2 and the initial state of the list is $\{a, b\}$, if the requested page is b, the online list update algorithm has to decide whether to bring b to the front or to let it remain where it is. If the next request is $\langle b, \cdots \rangle$ or $\langle a, b, \cdots \rangle$ is not enough to decide either of them. Note that if the future requests are $\langle a, b, a, b, \cdots \rangle$, the relative ordering between a and b does not matter. On the other hand, if the future requests are $\langle b, b, a, \cdots \rangle$, it is optimal to bring b to the front.

4.2.1 PDA algorithm

We design a prediction model where the list update algorithm has access to predictions concerning the next two occurrences of the requested item in the request sequence. We refer to this algorithm as ***Penalty Distance Algorithm (PDA)***. As each request is served, the list goes from one state to another through some paid exchanges and free exchanges. For that, PDA considers every possible **configuration** (ordering) of the list and computes two types of costs for each configuration, which we refer to as ***distance*** and ***penalty*** costs. PDA chooses the configuration that minimizes the sum of the penalty and the distance.

The idea behind penalty cost is to maintain an ideal configuration of the list that is aligned with an optimal ordering for lists of length 2 formed by the requested item and other items. Suppose b is the requested item, and let a be any item in the list. If the current ordering is (a, b), and the next three requests are predicted to be $\langle b, b, a, \cdots \rangle$, then a should be located after b in an ideal list. On the other hand, if the next three requests are predicted to be $\langle a, a, b, \cdots \rangle$, then a should be located before b in an ideal list. For any other continuation of the predicted input, the relative order of a and b does

not matter. Any violation of the ideal ordering in the above scheme translates to one unit of "penalty" in the PDA algorithm.

Upon accessing an item x in the list, PDA colors all other items based on their relative order with x. For each possible configuration C of the list, it computes the number of items whose ordering with x in C is not ideal; the number of such items will be the *penalty* associated with x. PDA also computes the **distance** of C from the current configuration, defined as the minimum number of free and paid exchanges to convert the current configuration of the list to C. The algorithm moves to a configuration with a minimum sum of penalty and distance. We are working on proof to show the following upper bound for the competitive ratio of PDA. If correct, the algorithm improves the best existing approximation algorithm with an approximation factor of $1.\bar{6}$ [95].

PDA uses the paid exchanges to change the configuration of the list and free exchanges to reduce the penalty to eventually reduce the sum of penalty and distance. First, we determine the relative color of the items before the requested item in the list. Our objective is to keep only green items before the requested item. We use paid exchanges to *bubble up* the green items before the red items as each request comes. The gray items do not matter. Finally, after making the paid exchanges, we use the free exchange to move the requested item. For a given initial and final state of a list and a requested element, let the number of green and red elements before the requested element be g_i and r_i respectively for the initial state of the list, and g_f and r_f for the final state of the list. We define penalty as $(\overline{r_f - g_f} - \overline{r_i - g_i})$. We define the "red-green factor" as the numerical difference between the number of green and red items before the requested item in the list. Next, we define penalty as the difference between the red-green factor in the final state and the same in the initial state of the list. We move the requested item toward the front in the final state, where the penalty is minimized.

Conjecture 1. *When the prediction is correct, using the proper potential function for the analysis, we show PDA's competitive ratio is* ≤ 1.6.

4.2.2 Pareto-efficiency

We study the trade-off between *consistency* (competitive ratio when predictions are all correct) and *robustness* (competitive ratio when predictions are adversarial) for list update algorithms. PDA has good consistency, but it fails if predictions are incorrect. We will use techniques like that of [97–99] to PDA with MTF (which has optimal robustness) to achieve Pareto-efficient algorithms with good consistency and robustness trade-off.

4.3 Conclusion

In conclusion, while ML-advised algorithms have shown promise in enhancing performance for various problems, such as the rent-or-buy scenarios discussed in the previous chapter, their efficacy is highly dependent on the nature of the advice and the specific characteristics of the problem at hand. In our study of the list update problem under the ML-advised framework, we observed that even with low prediction error, the learned advice does not necessarily guarantee superior performance.

For lists with two items, accurate predictions about the next request can ensure optimal performance using a simple algorithm that relies solely on free exchanges. However, for lists with three items, knowing when each item will be requested next is insufficient for making optimal decisions. To address this, we proposed the Penalty Distance Algorithm (PDA), which leverages predictions about the next two occurrences of the requested item. Although our conjecture suggests that PDA can achieve a competitive ratio of ≤ 1.6 when the predictions are correct, this highlights the nuanced and problem-specific nature of ML-advised algorithms. Thus, the utility of ML advice must be carefully evaluated within the context of each unique problem.

Chapter 5

ML-Advised Non-Clairvoyant Scheduling

This chapter studies a job scheduling problem and develops ML-advised online solutions for the problem. We will study the problem definition for the non-clairvoyant scheduling problem as well as see the empirical benefits of the ML-advised algorithmic solutions on some popularly-used benchmarks.

This study is done in collaboration with Dr. Rathish Das.

Abstract. The theory has taken an interest in studying the job scheduling problem when the scheduler remains in the dark about certain job characteristics, such as the processing time and arrival pattern. Such non-clairvoyance is very common in operating systems, where the job length is known only when the job finishes. The problem has been studied under traditional competitive analyses.

We study the non-clairvoyant non-preemptive online job scheduling problem with machine learning (ML) advice to traditional online algorithms. An ML oracle (adviser) in the system estimates the processing time of each job. However, this advice may be erroneous. We employ an error measure ζ that quantifies the prediction error that leads to changing the relative order of execution of any pair of jobs from the optimal offline algorithm. We designed an online algorithm MLA-RJF for the dynamic non-clairvoyant job scheduling problem, where each job has a release time and cannot be executed prior to its release. The jobs lose progress if preempted during execution. We show that MLA-RJF is $\min\{1 + \frac{\zeta}{OPT}, k\}$-competitive, where k is the ratio of the longest and the shortest job, OPT is the cost of the optimal offline algorithm.

We present an empirical study to show the performance advantage of using MLA-RJF on two datasets of real traces from production servers: Google and Alibaba. For both realistic traces, MLA-RJF performs within $2\times$ of the

optimal offline algorithm and 3× better than the best online algorithm without ML advice.

5.1 Introduction

Before Motwani et al.'s seminal work [100], scheduling theory considered only ***clairvoyant scheduling***, i.e., when the scheduler knows the specific characteristics of each job apriori [101], such as execution time, release time, interdependence of the jobs. However, the scheduler may not have such knowledge about the jobs' processing times for many real-world applications, such as scheduling jobs on time-sharing operating systems [102] and on shared compute clusters [103, 104].

Motwani et al. [100, 105] introduced the ***non-clairvoyant job scheduling*** problem to study online scheduling strategies for single- and multi-processor settings when the scheduler does not know the job's processing time until they finish. Note that it is distinct from the problem where only the release times of the future jobs are unknown, but as soon as a job is released, the total processing time (or resource requirement) is known to the scheduler [106].

Traditionally, ***competitive analysis*** [4] is used to study the performance of an online algorithm by comparing its performance with the optimal clairvoyant (offline) algorithm that is visionary in the sense that the algorithm knows the complete input from the future. It provides a worst-case performance guarantee for the algorithms for every possible problem instance. In light of competitive analysis, an online algorithm is ρ-competitive if $ALG(I) \leq \rho \cdot OPT(I)$ $\forall$ all problem instance I, where $ALG(\bullet)$ is the cost of the online algorithm and $OPT(\bullet)$ is the cost of the optimal offline algorithm.

This chapter focuses on the ***non-preemptive non-clairvoyant job scheduling***; the problem instance is given with a set of jobs with unknown processing times coming online at its release time; if a job is preempted the entire progress on that job is lost. Because in our model, each job must be performed in an uninterrupted manner until it finishes, we also refer to this model as ***kill-and-restart*** model [107]; if the scheduler kills a job midway, it has to run the job from the start. This is relatable to several realistic scheduling systems, for example, in the case of the query scheduling problem on clusters, in most cases, the progress on a query cannot be saved if it is preempted [108, 109]. Furthermore, non-preemptive schedulers are easier to implement and are more efficient because there is no overhead associated with preemptions. Hence, the theory has taken interest in studying the non-preemptive scheduling problems [110–112]. We refer to the set of jobs already released but not yet finished as ***active jobs***. In this model, a non-clairvoyant scheduler assigns jobs sequentially to a

single ***unit-speed processor***, a processor that can run one unit of job in unit time. The problem has two variants: ***static*** and ***dynamic***. For the static problem, all the jobs arrive at the beginning of the problem instance, i.e., for all the jobs, the ***release time*** or ***arrival time*** is 0. We focus on the dynamic version of the problem, where the jobs can have non-zero release times. [1]

First, we revisit the rich piece of research on static and preemptive non-clairvoyant scheduling and investigate which part we can use in the context of dynamic non-preemptive non-clairvoyant settings. For the static non-clairvoyant job scheduling problem, the ***Shortest Job First (SJF)*** algorithm is the optimal offline strategy. SJF is clairvoyant and schedules the jobs in the non-decreasing order of their processing time [2] non-preemptively, i.e., a shorter job is scheduled before a longer job. However, this requires clairvoyance, i.e., precise knowledge about the length of each job. For the static preemptive non-clairvoyant scheduling, where the length of the jobs is not known apriori, the best deterministic algorithm, the Round Robin algorithm (evenly distributes the computing power available among all the resources), is $(2 - 2/(n + 1))$-competitive in terms of the total completion time [100, 105] , where n is the number of jobs. Randomization could bring little improvement, only in lower-order terms.

The initial success with Round Robin bred hope that designing an algorithm for dynamic non-clairvoyant scheduling might not be challenging. In fact, Round Robin is probably already constant-competitive compared with the optimal offline algorithm that schedules the job with the shortest remaining length, i.e., ***Shortest Remaining Time First (SRTF)***. However, Motwani et al. [100, 105] showed that Round Robin is $\Omega(n/\log n)$-competitive. Furthermore, they also showed that any deterministic strategy can only be $\Omega(n^{1/3})$-competitive. We can only achieve up to $\Omega(\log n)$ even with randomization [100, 105]. In fact, any deterministic or randomized strategy that does not intentionally keep the processor idle/inactive is $O(n)$-competitive [100, 105].

At this crossroad, people bifurcated into two major directions to study dynamic non-clairvoyant scheduling. One is to have bounded job sizes [100, 105]; if the longest and shortest jobs have the ratio k, Round Robin is $O(k)$-competitive. Another line of research [113–116] explored the shortcomings of the competitive analysis framework. In this approach, named ***resource augmentation***, the online algorithm is given slightly more resources than the optimal offline algorithm, and their performance is compared on the same

[1] In this chapter, we refer to the non-clairvoyant job scheduling with a single unit-speed processor as the static and dynamic scheduling problems, depending on the release time of the jobs.

[2] We refer to the processing time of a job as ***job length***, as we have only a single unit-speed processor.

problem instance. Kalyansundaram and Pruhs [115, 116] showed that even with resource augmentation Round Robin is not constant-competitive. On the other hand, Shortest Executed Time First (SETF) [115, 116] is $(1/\varepsilon)$-competitive with $(1+\varepsilon)$-speed augmentation. This means, for example, if the optimal offline algorithm is given a unit-speed processor and the SETF algorithm is given a 1.1-speed processor (10% faster than what the offline algorithm gets), SETF is 10-competitive. However, SETF is a preemptive strategy. Moreover, the concern with resource augmentation is that the comparison is "apples vs. oranges" [5]—it introduces uncertainty about whether the optimal offline loses its edge over the online algorithm as we artificially restrict its resource.

The assumption of non-clairvoyance is not always accurate in real-life scenarios [104, 117]: the exact processing time might be unknown; however, predictions might be available to the scheduler in the form of insight or advice. Much later, recently, with the advent of the novel approach of designing online algorithms with Machine Learning (ML) augmentation [34–36] , Purohit et al. [71] explored the potential of ***ML advice (MLA)*** and designed an ML-advised algorithm for static preemptive non-clairvoyant scheduling with a competitive ratio of $min\{\frac{1}{\lambda}(1+\frac{2\eta}{n}), \frac{2}{1-\lambda}\}$, where λ is a hyperparameter, and η is the total prediction error in the ML oracle. While Im et al. [118] furthered this study by coming up with a better measure for the prediction error, Lindermayr and Megow [119] developed a different prediction model with a new error definition, i.e., the permutation prediction model. We use an unweighted version of Lindermayr and Megow's prediction error measure [119]; we refer to this error as the ***total prediction-ordering error*** (ζ). The total prediction-ordering error has an additive term only when the relative order of execution of a pair of jobs is different than the optimal offline schedule. We revisit the non-clairvoyant job scheduling problem in the ML-augmented setup: an ML oracle predicts the processing time of the job. We design online algorithms with learned advice for dynamic setups when the scheduler is in the dark, where the prediction from the oracle can be arbitrarily erroneous (untrusted advice [120]).

Our results. We designed an online algorithm MLA-RJF for the dynamic non-clairvoyant non-preemptive scheduling, where each job has a release time and cannot be executed prior to its release. We show MLA-RJF is $\min\{\frac{1}{\lambda}(1+\frac{\zeta}{OPT}), \frac{k}{1-\lambda}\}$-competitive, where λ is a hyperparameter, and k is the ratio of the longest and the shortest job and OPT is the cost of the optimal offline algorithm. For example, with $\lambda = \phi - 1$, where ϕ is the golden ratio (approximately 1.618), even when the total prediction-ordering

error reaches OPT, MLA-RJF is $2 \cdot \phi$-competitive, whereas the traditional online algorithms are not constant-competitive. MLA-RJF is not worse than $k \cdot (\phi + 1)$-competitive for arbitrarily poor prediction.

The ML-advised non-clairvoyant scheduling algorithm MLA-RJF is particularly helpful for query scheduling on clusters [104, 108, 121]. In most realistic scenarios, the queries come to the cluster in an online manner. There can be some insight available about the processing time of the query but the insight might be erroneous [104]. Furthermore, queries may not save the progress if they are preempted [108]. In that case, the scheduler needs to employ an algorithm that does not preempt unnecessarily. Finally, we corroborate this idea by performing an empirical evaluation of MLA-RJF. MLA-RJF's performance is compared with the optimal offline algorithm for two real data traces from ***Google*** [122] and ***Alibaba*** [123]. We vary the error in prediction by a normal distribution. We find that when the job length prediction is good, MLA-RJF is within a factor of 2 of the optimal offline algorithm. Even when the prediction is poor, MLA-RJF performs within 3× of the optimal offline cost. Interestingly, MLA-RJF performs up to 3× better than the best non-preemptive online algorithm without ML advice.

5.2 Problem formulation

Each problem is initiated with a set of n independent jobs denoted as $\mathcal{J} = \{J_1, \cdots, J_n\}$. Each job J_i has a release time r_i and processing time p_i, i.e., job J_i is first available to be executed only after time r_i, and if scheduled immediately after the release on a processor and executed without any preemption, it will take p_i time to finish the job. [3] Note that for the static non-clairvoyant problem, all the release times are zero, i.e., $r_i = 0 \; \forall \; i \in \{1, 2, \ldots, n\}$, as all the n jobs are released at the beginning. A summary of the scheduling algorithms for the static case is given in Appendix 5.3. The ***flow time*** (or ***completion time*** in the static case) of a job i is defined as the time $c_i - r_i$, given that job i is done executing at timestamp c_i for a certain schedule. Total flow time is the sum of flow times for all the jobs: $C = \sum_i (c_i - r_i)$.

A schedule $\mathcal{S}$ is an assignment of each job to the processor. If any job is preempted (or interrupted) during its execution, the entire progress on that job is lost; we refer to this model as "kill-and-restart" model [107]. This is the difference between the preemptive and the non-preemptive settings of the non-clairvoyant scheduling problem. In the preemptive setting, a job can be broken into infinite subjobs and these subjobs can be executed separately. In

[3] There is no setup time for a job.

that case, for each job J_i, the sum of the lengths of all the subjobs executed has to be p_i. We focus on solving the non-preemptive non-clairvoyant scheduling problem with release times (dynamic). In order to do that, we revisit some strategies for the static setting (with no release times) as well as some preemptive scheduling strategies. We assume, at one unit time, the processor can only run 1 unit of a job. We can also assume without loss of generality that the processing times of the jobs are integral, i.e., $p_i \in \mathcal{N}^+$ where $\mathcal{N}^+$ is the set of positive integers, i.e., $p_i \geq 1 \ \forall \ i \in \{1, 2, \ldots, n\}$. Furthermore, context switching is not allowed within a unit time, i.e., each processor can execute 1 unit of any particular job scheduled in a unit time. ALG is the total flow time for schedule $\mathcal{S}_{ALG}$ using algorithm ALG. OPT is the total flow time for schedule $\mathcal{S}_{OPT}$ using the optimal offline algorithm. The competitive ratio (CR) can be expressed as follows.

$$CR = \sup_{\mathcal{J}:|\mathcal{J}|=n} \frac{ALG(\mathcal{J})}{OPT(\mathcal{J})}.$$

In this chapter, we shall how to design an algorithm that minimizes the competitive ratio of total flow time for dynamic non-clairvoyant non-preemptive scheduling.

5.3 Preemptive non-clairvoyant scheduling

The ***optimal offline strategy*** is clairvoyant: it has the knowledge of the processing times for all the jobs, i.e., $p_i \ \forall \ i \in \{1, 2, \ldots, n\}$. It schedules the jobs following the ***Shortest Job First (SJF)*** [124] strategy. At $t = 0$, all n jobs are added to the queue of active jobs. The queue is sorted in an ascending order based on the processing time of the jobs (p_i). The jobs are scheduled in a non-preemptive manner following the queue from the top of the queue. The total completion time of SJF (SJF)[4] can be expressed by the following expression.

$$SJF = \sum_{i=1}^{n} p_i + \sum_{1 \leq i < j \leq n} \min(p_i, p_j). \tag{5.1}$$

The objective is to minimize the pairwise delay for each pair of jobs. The completion time of each job is the sum of the total delay caused by other jobs and the actual processing time of the job (***actual job length***). As for any

[4]Given that a non-clairvoyant scheduling algorithm's abbreviation is A, we express the total flow time of that algorithm as A. In the static case, this is the same as the total completion time as all the jobs' release times are 0.

pair of jobs (J_i, J_j), if job J_i is shorter, i.e. $p_i < p_j$, only J_j is delayed by J_i. This is how SJF minimizes the second term in the equation 5.1. Hence, for $p_1 \leq p_2 \leq \cdots \leq p_n$, SJF can be expressed as follows.

$$SJF = \sum_{i=1}^{n} p_i + \sum_{i=1}^{n} (n-i)p_i = \sum_{i=1}^{n} (n-i+1)p_i. \tag{5.2}$$

Round Robin (RR) algorithm. The processor distributes the computing resources equally among the active jobs. Hence, the algorithm ensures that all the active jobs receive an equal amount of processing time. Purohit et al. [71, 118] considered that RR cycles through the incomplete jobs and assigns 1 unit of processing time for each job [5]. For the static preemptive non-clairvoyant job scheduling on a single processor, RR is $2-2/(n+1)$-competitive [100, 105].

Randomized Selection (RAND) algorithm. With probability $1 - 2/(n+3)$, the Randomized Selection algorithm distributes the processing power equally among all the active jobs (Round Robin). With probability $2/(n+3)$, RAND schedules in a uniformly random manner and runs each job non-preemptively following that order; we refer to this algorithm as ***Random Job First (RJF)***[6]. RAND is $2 - 4/(n+3)$-competitive for the static non-clairvoyant scheduling problem [100, 105].

Dynamic preemptive non-clairvoyant job scheduling. For dynamic non-clairvoyant scheduling, there are arbitrary release times for each job. The optimal offline schedule, ***Shortest Remaining Time First (SRTF)***, remains similar to the optimal offline algorithm for the static case, Shortest Job First (SJF). SRTF allocates the processor to the job with the least remaining processing time. Note that, SRTF is a preemptive strategy, whereas SJF is non-preemptive. Motwani et al. showed that RR is $\Omega(n/\log n)$-competitive in the dynamic setting. In fact, any deterministic algorithm is $\Omega(n^{1/3})$-competitive, and any randomized algorithm is $\Omega(\log n)$-competitive [100, 105]. RJF, on its own, is a non-preemptive scheduling strategy. Hence, in the non-preemptive problem formulation as the optimal offline strategy is worse than SRTF, RJF's relative performance is no worse than the bound in Lemma 5.

Lemma 5. *For the dynamic preemptive non-clairvoyant job scheduling on a single processor, the randomized algorithm RJF is $\Theta(k)$-competitive, where the*

[5] RR is also referred to as EQUI.

[6] Motwani et al. [100, 105] referred to RJF as Run-to-completion (RTC) algorithm, and Cohen et al. referred to this as RANDOM.

ratio of the smallest and the largest job is at most k.

Proof. Note that both SRTF (the optimal offline algorithm) and RJF do not spend idle time when there are active jobs in the system. RJF can have at most k times as many jobs as the SRTF has received but is yet to finish. As the total flow time is the sum of the total execution times of the jobs and the total waiting time (sum of waiting time for all n jobs, when a job is released and not finished but not being executed as the job is waiting on some other job), only the waiting time of RJF can be at most k times that of SRTF. □

Machine Learning advice (MLA). Machine Learning (ML) advice [34–39] is an algorithm-design framework to improve the performance of traditional online algorithms. In the real world, online algorithms often have some insight into the input. ML-advised algorithms take advantage of such predictions and achieve a better average-case performance than their traditional online counterparts while retaining the worst-case performance guarantees.

Preferential Round Robin (PRR) algorithm. Purohit et al. [71] took the first step in designing an ML-advised algorithm for the static non-clairvoyant job scheduling problem; they refer to this algorithm as the Preferential Round Robin algorithm. Note that the algorithm requires all the jobs to have a release time of 0. PRR lets the ML oracle and the Round Robin algorithm share the computing resource. The ML oracle predicts the job lengths and schedules the job in the order of shorter predicted job length. If the oracle gets λ fraction of the computing resource and the rest goes to the Round Robin algorithm, PRR achieves a resultant competitive ratio of $\min\{\frac{1}{\lambda}(1+\frac{2\eta}{n}), \frac{2}{1-\lambda}\}$, where η is the error in prediction of processing time of the jobs by the ML oracle.

5.4 ML-advised non-clairvoyant scheduling

Previous systems research [102, 125, 126] suggests that it is possible to estimate the job length as they arrive from the historical data and job characteristics. However, ML-based solutions for clusters and operating systems can have unbounded errors in the prediction.

We designed an algorithm that takes advantage of the job length predictions. When the ***predicted job length*** is close to the actual job length, the algorithm performs closely to the optimal offline algorithm. Even when the prediction is poor, ML advice is helpful for job scheduling. The algorithm

distributes the processing power in two segments: a λ ($0 \leq \lambda \leq 1$) fraction goes to the ML oracle, and the rest $1 - \lambda$ goes to a traditional randomized online algorithm. The ML oracle predicts the job lengths and when the machine becomes available schedules the job with the shortest predicted length non-preemptively; we refer to this strategy as ***Predicted Shortest Time First (PSTF)***. The oracle's error in prediction can be unbounded. However, the traditional algorithm has a worst-case bounded performance. We take the best of both worlds; the resultant algorithm has a better average-case performance and a bounded worst-case performance.

For static non-clairvoyant scheduling, Purohit et al. [71] showed that PSTF schedules in the increasing order of job lengths and analyzed the performance using a ***linear cumulative error*** measure (l_1 distance between the predicted and actual job lengths). If the length of job J_i is predicted to be $\hat{p_i}$ by the ML oracle, and the prediction error is denoted by η_i, then η_i is the absolute linear distance between the actual length of the job and the predicted length of the job, i.e., $\eta_i = |p_i - \hat{p_i}|$. Following this error measure, PSTF is $(1 + \frac{2\eta}{n+1})$-competitive for the static non-clairvoyant job scheduling (Lemma 6).

Lemma 6. *For the static non-clairvoyant job scheduling problem, the Predicted Shortest Time First algorithm is* $(1 + \frac{2\eta}{n+1})$*-competitive [71], where we calculate the prediction error of the oracle as the sum of the absolute value of the* l_1 *distance between the predicted job length and the actual job length.*

Proof. Note that the pairwise delay for each pair of jobs (J_i, J_j) in case of the optimal offline is $\min\{p_i, p_j\}$. Let's say due to misprediction, there is an alteration in the relative order of execution for the pair (J_i, J_j). If $p_i < p_j$ is the relation between thee actual job lengths, the prediction says $\hat{p_i} > \hat{p_j}$. In

this case, the pairwise delay for PSTF becomes $\max\{p_i, p_j\}$.

$$\begin{aligned}
PSTF &= \sum_i p_i + \sum_{(i,j):p_i<p_j \& \hat{p_i}<\hat{p_j}} p_i + \sum_{(i,j):p_i<p_j \& \hat{p_i}>\hat{p_j}} p_j \\
&= \sum_i p_i + \sum_{i,j:p_i<p_j} p_i + \sum_{(i,j):p_i<p_j \& \hat{p_i}>\hat{p_j}} (p_j - p_i) \\
&\leq \sum_i p_i + \sum_{i,j:p_i<p_j} p_i + \sum_{(i,j):p_i<p_j \& \hat{p_i}>\hat{p_j}} (p_j - \hat{p_j}) - (p_i - \hat{p_i}) \\
&= \sum_i p_i + \sum_{i,j:p_i<p_j} p_i + \sum_{(i,j):p_i<p_j \& \hat{p_i}>\hat{p_j}} (\eta_i + \eta_j) \\
&\leq OPT + (n-1)\eta
\end{aligned}$$

As the minimum length of a job is $1, OPT \geq \dfrac{n \cdot (n+1)}{2}$

$$\frac{PSTF}{OPT} \leq 1 + \frac{(n-1)\eta}{OPT} = 1 + \frac{2(n-1)\eta}{n(n+1)} \leq 1 + \frac{2\eta}{n+1} \tag{5.3}$$

□

Using PSTF, Purohit et al. [71] designed the Preferential Round Robin algorithm that achieves a competitive ratio of $\min\{\frac{1}{\lambda}(1 + \frac{2\eta}{n+1}), \frac{2}{1-\lambda}\}$. Later, Im et al. [118] observed that, for the analysis of the performance of the PRR algorithm, the l_1 distance between the actual job length and the predicted job length is not a good error measure. Lindermayr and Megow [119] developed a different prediction model with a new error definition, i.e., the permutation prediction model. We use an unweighted version of their prediction error measure to capture the effect of misprediction more effectively.

Error measure in our study. For each pair of jobs (J_i, J_j) (assuming $p_i < p_j$), the pairwise delay for the optimal offline is $\min\{p_i, p_j\}$, i.e., p_i. In case of a misprediction, the l_1 distance is not a proper error measure because, even if the predicted job lengths are too far from the actual job lengths, as long as $\hat{p_i} < \hat{p_j}$, oracle still schedules J_i before J_j. On the other hand, if $\hat{p_i} > \hat{p_j}$, in that case the pairwise delay is $\max\{p_i, p_j\}$, i.e., p_j. The extra additive term in the pairwise delay for the pair of jobs (J_i, J_j) can be expressed as $\mathcal{U}[\hat{p_i} - \hat{p_j}] \cdot (p_j - p_i)$, where $\mathcal{U}[x]$ is the ***unit-step function*** or ***Heaviside step function***, i.e., it takes a value 1 when $x \geq 0$, and 0 otherwise. The overall error in prediction, ζ, is defined as the sum of the extra additive terms of all the pairwise delays. We refer to this error measure as the total prediction-

ordering error.

$$\zeta = \sum_{(i,j):p_i<p_j \& \hat{p_i}>\hat{p_j}} \mathcal{U}[\hat{p}_i - \hat{p}_j] \cdot (p_j - p_i). \tag{5.4}$$

With the total prediction-ordering error measure ζ, we show that PSTF is $(1 + \frac{\zeta}{OPT})$-competitive (Theorem 1) for dynamic preemptive non-clairvoyant scheduling. As PSTF is non-preemptive itself and the optimal offline SRTF is preemptive, in the non-preemptive setting, the optimal offline cannot be better than SRTF. Hence, the performance bound of PSTF for the preemptive setting holds in the case of the non-preemptive setting as well.

Theorem 1. *For the dynamic preemptive non-clairvoyant problem, the Predicted Shortest Time First algorithm is $(1 + \frac{\zeta}{OPT})$-competitive where ζ is the total prediction-ordering error, and OPT is the total flow time of the optimal offline algorithm.*

Proof. We observe that with $\zeta = 0$, i.e., when the prediction has no error, ML oracle runs the optimal offline algorithm Shortest Remaining Time First (SRTF). For any misprediction that causes the order of execution of any two pairs of jobs to alter, PSTF has an extra additive term in the total waiting time, leading to increased total flow time. Hence, the total prediction-ordering error, ζ, can be bounded by the following expression. $\zeta \leq \sum_{(i,j):p_i<p_j} \mathcal{U}[\hat{p}_i - \hat{p}_j] \cdot (p_j - p_i) = \sum_{(i,j):p_i<p_j \& \hat{p_i}>\hat{p_j}} (p_j - p_i)$. Hence, the cost of PSTF ($PSTF$) can be expressed as $OPT + \zeta$, i.e., PSTF is $(1 + \frac{\zeta}{OPT})$-competitive in the preemptive setting. □

Next, we show that if two non-preemptive dynamic non-clairvoyant scheduling algorithms, one learned-oracle based (ORACLE) and the other conventional randomized algorithm (ALG), share the processor, the competitive ratio of the resulting algorithm can be expressed by their share of processor and their individual competitive ratios (Theorem 2).

The resultant scheduling algorithm runs the two schedules given by ORACLE and ALG in a non-preemptive manner. ORACLE maintains its schedule from the predicted job lengths and ALG maintains its schedule by choosing an arbitrary order among the available jobs. Note that any job appears twice, once in each schedule. Once a job is already performed from one of the schedules, when its turn comes in the other schedule the job is skipped. The ORACLE gets a share λ of the computing resource (processor) and the rest $(1 - \lambda)$ goes to ALG. The sharing works in the following manner. ORACLE runs a job non-preemptively until it finishes, say the job length is x. Next, ALG has a "budget" of $x \cdot (1 - \lambda)/\lambda$. ALG runs a number of jobs until it

runs out of the budget without preemption. After that, the resultant scheduling algorithm then switches back to ORACLE and so on. Note that x does not have to be a predefined constant (and hence the system's performance is independent of x); it can be be data-dependent because the schedulers are non-preemptive it runs a job to the end and calculates the budget for the next scheduler thereafter.

Theorem 2. *Given two non-preemptive dynamic non-clairvoyant scheduling algorithms, a learning-based algorithm ORACLE that is α-competitive, and a randomized algorithm ALG that is β-competitive, if they share the processor and get a fraction λ and $1-\lambda$ respectively, the overall competitive ratio becomes* $\min\{\frac{\alpha}{\lambda}, \frac{\beta}{1-\lambda}\}$.

Proof. Given that instead of getting a share 1 of the computing resource, ORACLE gets λ, the flow time for job i increases by a factor $1/\lambda$. Note that, the jobs are run in a non-preemptive manner; any preemption causes losing the entire progress on that job. Similarly for ALG, the flow time of every job increases by $1/(1-\lambda)$. As both the bounds hold simultaneously, even if the other algorithm makes a little progress, the resultant bound remains the better one between the two, i.e., $\min\{\frac{\alpha}{\lambda}, \frac{\beta}{1-\lambda}\}$. □

For the static non-clairvoyant problem, Purohit et al. [71] designed the PRR algorithm that lets the PSTF and traditional RR algorithm share the processor. RR has a major disadvantage: if a job has a length x, it preempts the job $O(x)$ times. Hence, PRR cannot be applied in a non-preemptive setting.

This chapter proposes MLA-RJF, a simple modification to PRR, where MLA-RJF lets PSTF and RJF share the processor and is applicable in a non-preemptive setting. MLA-RJF provides PSTF λ share of the processor, and the rest $(1-\lambda)$ goes to the RJF. MLA-RJF achieves a bounded total flow time within $\min\{\frac{1}{\lambda}(1+\frac{\zeta}{OPT}), \frac{k}{1-\lambda}\}$ times of the total flow time of the optimal offline algorithm (OPT) (see Theorem 3), where ζ is the total prediction-ordering error and k is the ratio of the longest and the shortest job length.

Theorem 3. *With the dynamic non-clairvoyant scheduling problem, MLA-RJF is* $\min\{\frac{1}{\lambda}(1+\frac{\zeta}{OPT}), \frac{k}{1-\lambda}\}$*-competitive, when ζ denotes the total prediction-ordering error in the ML oracle and k is the ratio of the largest and smallest job length.*

Proof. For MLA-RJF, we get two schedules from the ML oracle (PSTF) and the RJF algorithm simultaneously. We allocate a λ factor of the processor to run the schedule obtained from the PSTF. PSTF is $(1+\frac{\zeta}{OPT})$-competitive (see Theorem 1). RJF is $O(k)$-competitive for dynamic non-clairvoyant job

scheduling on a single unit-speed processor when the ratio of the shortest and the longest job is bounded by k (see Lemma 5). Theorem 2 shows that MLA-RJF is bounded by the minimum of the two performance bounds, weighted with λ and $1 - \lambda$. □

5.5 Empirical evaluation

This section presents an empirical evaluation of three strategies for dynamic non-clairvoyant scheduling and compares their performances with the optimal offline algorithm. The three algorithms are as follows: FAO (the scheduler follows the arrival order to schedule the jobs)[7], RJF (the scheduler schedules a job chosen in a uniformly random manner from the queue of active jobs), and MLA-RJF (the learning-based PSTF and RJF share the processor). We also include PSTF (blindly trusting the oracle predictions) as a baseline in the study (see Table 5.1). Note that, we exclude Round Robin (RR) [100, 105], Preferential Round Robin [71, 118], and TwoStage [127] from our empirical evaluation, as they require preemption of jobs without any cost, and this chapter deals with the kill-and-restart model (refer to Section 5.2).

5.5.1 Dataset preparation

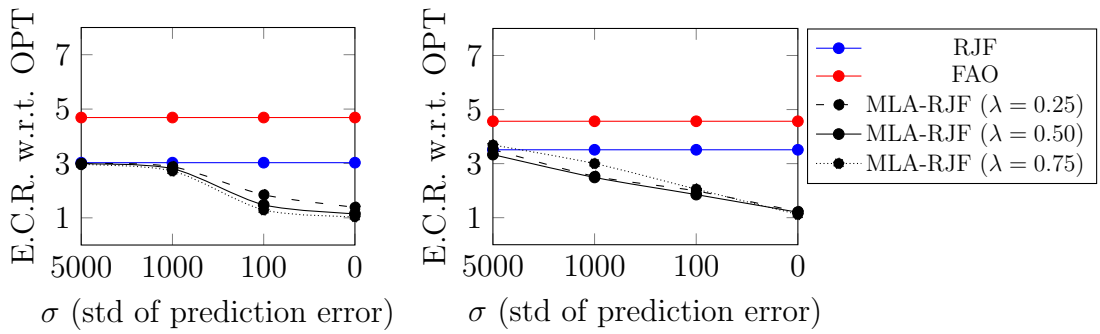
This work compares the performance of the non-clairvoyant scheduling algorithm MLA-RJF against the optimal offline algorithm (clairvoyant) on two real-life datasets: Google trace data [122] and Alibaba trace data [123].

Google trace data. Google trace data [122] consists of traces from parts of the Google cluster management systems. The 2011 dataset provides traces from a cluster taken over a 7 hour period. The workload consists of a set of tasks, where each task runs on a single machine. Tasks consume memory and one or more cores (in fractional units). Each task belongs to a single job; a job may have multiple tasks. For our experiments, we simplify the data by using each job as an inseparable task and by assuming that the nature of the job is not preemption-friendly. The source of uncertainty is two-fold: the arrival pattern of the jobs is unknown to the scheduler and the actual length of a job is not revealed to the scheduler until a job is finished. The trace contains a snapshot of the system every 5 seconds.

Alibaba trace data. Alibaba trace [123] provides trace data from a production cluster over a 24 hour period. The data includes part of the machines and

[7]FAO is also referred to as FIFO (First-in-first-out) in the literature.

the workload of the whole cluster. As the datacenter grows in scale, large-scale online service and batch jobs co-allocation are used to increase the datacenter's efficiency. The co-allocation challenges existing cluster management systems, particularly the services and jobs scheduler, which must work together to increase cluster utilization and efficiency. We simplify the data by taking the related tasks to construct a single job. We derive the execution time of a job by summing the execution times of the component tasks. We do not take the directed acyclic graph (DAG) relationship of the tasks into consideration. Note that, as we consider only one machine in our problem, we overlook the resource demand and resource availability in this work and leave it for future investigation.



(a) Plot for Google trace (b) Plot for Alibaba trace

Figure 5.1: Empirical Competitive Ratio (E.C.R.) of the non-clairvoyant algorithms studied in our work as we vary the standard deviation (σ) of the normal distribution in the prediction error. RJF is the best-known online algorithm for non-preemptive non-clairvoyant scheduling without learned advice. The mean of the prediction error is 0 and the standard deviation is varied from 0 to 5000. Note that $\sigma = 0$ denotes that the advice has no error. PSTF gets λ fraction of the processing resource and the rest goes to RJF.

5.5.2 Algorithm implementation

We tested the algorithm MLA-RJF for the dynamic non-clairvoyant non-preemptive setting. For most of the real-life experiments, it is difficult and somewhat unrealistic to assume that either all the jobs are released at the beginning or the job release pattern is known to the scheduler. Both the datasets we used necessarily pose the same constraint, hence, the jobs arrive at an arbitrary pattern. Note that in the experimentation phase, we do not synthetically

introduce a randomized job arrival pattern. The job arrival timestamp is unknown to the scheduler and follows a realistic pattern in the datasets we used.

We compare the performance of MLA-RJF with a clairvoyant algorithm optimal offline (OPT) under the kill-and-restart model with total flow time as the objective function. We employ the ***empirical competitive ratio (E.C.R.)*** [119] as our performance metric, hence, the competitive ratio of MLA-RJF for a given data trace is the ratio of the total flow time incurred by MLA-RJF to the total flow time incurred by OPT (Table 5.1). Note that, under the kill-and-restart model, the Shortest Remaining Time First (the optimal preemptive strategy for dynamic scheduling) cannot be directly plugged in as the optimal solution; SRTF needs a little modification to be optimal. OPT weighs in two possibilities: either to preempt the job and lose all the progress on the job or not to preempt and execute the shorter job at some later point in time. For example, if a job of length x starts at $t = 0$ and at $t = \delta$ another job of length y arrives and $y < x - \delta$. SRTF attempts to preempt the earlier job to execute the latter one. However, in the kill-and-restart setting, OPT does not interrupt the former job if $y + \delta < x < y + 2\delta$. Additionally, we include non-interruptive SRTF (N-SRTF) in our evaluation. N-SRTF maintains a queue of jobs that are released but not executed yet, and runs the shortest available job from the queue when the machine becomes available. Essentially, the key difference between the N-SRTF and the OPT is that when the scheduler runs a job and a shorter job arrives, N-SRTF never preempts the job.

Furthermore, we implemented the Random Job First (RJF) algorithm as a baseline non-preemptive online algorithm. RJF introduces a job when it arrives in a global queue at a position chosen in a uniformly random manner. When the machine becomes available, it runs the jobs from the global queue in order of the queue. Note that RJF is the best-known online algorithm without learned advice. As we discussed in section 5.2, RJF is within a factor of $O(k)$ of the OPT, where k is the ratio of the longest job length to the shortest job length (Lemma 5). For example, for the Google data trace, the longest job length is 380 seconds and the shortest job length is 5 seconds, resulting in $k = 78$.

Finally, we implement MLA-RJF: MLA-RJF lets two algorithms, learning-based Predicted Shortest Job First (PSTF) and Random Job First (RJF), share the processor, in a non-preemptive manner (as explained in Theorem 2). There is a hyperparameter λ that is used to determine how much processing resource goes to PSTF and how much to RJF. The oracle predicts the execution time of a job when it arrives. The prediction is untrusted in that it can be erroneous and the error may not be bounded by the actual job size. This ensures that even for short jobs, the predicted job length may be large, leading

to a large additive term in the total prediction-ordering error (ζ).

5.5.3 Experimental details

In this subsection, we explain our experimental model and the findings. We observe that in order to test the empirical advantage of MLA-RJF, there are two axes of freedom. First, we vary the error in prediction from the ML oracle. Second, we vary the hyperparameter λ in the setting of the algorithm to determine the share of the processing resource between PSTF and RJF. Along these two axes, we evaluate the performance of MLA-RJF.

The total prediction-ordering error (ζ) is varied based on a probability distribution, and the performance of MLA-RJF is observed. We follow Lindermayr and Megow [119] and choose a normal distribution for the prediction error for each job as they arrive. For each job, as it arrives, the predicted job length is the sum of the true job length and the prediction error. Hence, the predicted job length for job i can be expressed as $\mathcal{N}(p_i, \sigma^2)$, i.e., the mean of the prediction error is 0 and the standard deviation of the prediction error is σ. The standard deviation σ is varied from 0 to 5000. The reason we increase the standard deviation up to a large value compared to the longest job size (380 seconds for Google data trace) is that this will approach simulating an unbounded prediction error for poor ML oracles that only studies the historical pattern of job sizes rather than job characteristics as features [8]. This choice of prediction error refrains from favoring MLA-RJF with predictions of good quality, instead, develops a robust experimental milieu. Note that, when the total prediction-ordering error is 0, it denotes that the ML oracle correctly predicts the execution time of all the jobs.

Finally, the parameter λ can be "tuned" adaptively in practice. For example, when the predictor is working poorly, λ (the share of the processor for the learned-advice-based algorithm PSTF) can be reduced adaptively. On the other hand, when the predictor's performance is good, by increasing λ greater share of the processor can be given to the PSTF. We investigate MLA-RJF's performance for three values of λ: 0.25, 0.5, and 0.75.

Table 5.1 summarizes our experimental findings and Figure 5.1 shows how with the improvement of prediction, MLA-RJF performs increasingly better than RJF, the best-known online non-preemptive algorithm. MLA-RJF performs around 3× better than RJF when there is no prediction error. When we introduce a prediction error with standard deviation $\sigma = 100$, MLA-RJF still performs 2.3× and 1.7× better than RJF for the Google trace and the Alibaba

[8] The large value for the standard deviation also helps to simulate a large "distortion factor" [128], i.e., the ratio between the actual job length to the predicted job length.

trace respectively. Even when the standard deviation of the prediction error increases up to 5000, MLA-RJF performs comparably to RJF, within 5% of each other. We observe that as we allocate more computing resources on PSTF, i.e., increase λ for the Alibaba trace, the performance declines with poor prediction more rapidly compared to the decline for the Google trace. The reason for this is the variance of the actual job sizes is higher in the case of Alibaba trace (i.e., 216344) than that in the Google trace (i.e., 20534). Hence, the misprediction cost MLA-RJF more in the case of the Alibaba trace. Another observation is that when we have blind trust in the ML oracle's prediction and run PSTF, the performance can become very poor when the prediction becomes poor. On the other hand, even with large mispredictions, MLA-RJF's performance does not become as worse as PSTF.

Computational overhead from ML training. For the non-clairvoyant job scheduling problem with non-preemptive settings, ML-advised algorithms leverage predictions generated by ML models to guide decision-making. This approach enhances average-case performance while maintaining robust worst-case guarantees, as demonstrated in prior works [71, 72]. However, a crucial consideration for the practical deployment of ML-advised solutions is the computational overhead associated with ML training. This overhead, which can be significant, is necessary to generate the high-quality advice that these algorithms depend on. For example, in order to generate an advice with a prediction error with a low standard deviation requires a strong and efficient ML model to be trained as an advisor. Consequently, the computational complexity of the advisor must be factored into the overall efficiency evaluation of ML-advised algorithms. For the theoretical performance bounds to be valid, it is essential that the advisor operates in constant time, i.e., $O(1)$. If the computational cost of training and generating advice exceeds this, the practical benefits of using ML-advised algorithms may be compromised, thus invalidating the theoretical performance bounds that assume minimal advisor overhead.

5.6 Conclusion

In this chapter, we showed how to design algorithms for dynamic instances of job scheduling problems where the job length is not known apriori. We corroborate that instead of the linear distance between the actual job length and predicted job length, the relative ordering of execution of the jobs matters the most in calculating the effect of misprediction. Importantly, the learning-advised scheduling solution MLA-RJF is easily scalable to more-than-one-machine settings. One potential way to do that is to maintain two queues

for jobs that are released but not executed yet, one for the ML-advised PSTF algorithm and the other for RJF, as well as distribute an equal budget to PSTF and RJF for each machine. For example, if there are two machines, whenever any machine becomes available, PSTF runs a job from its queue. After that, based on the hyperparameter λ (explained in Section 5.4), RJF gets a budget to run one or more unfinished jobs from its queue non-preemptively on that specific machine.

We extend our theoretical understanding with an empirical evaluation. We test the scheduling strategies on two real data traces. This takes care by introducing a realistic distribution of the job lengths as well as a realistic job arrival pattern. We introduced prediction error following the normal distribution of high standard deviation in the experiments. Studying some other distributions for misprediction of job lengths will be a part of future studies. Furthermore, with different types of job arrival rates and patterns, it will be interesting to find out what should be the learning strategy of an ML oracle.

Finally, in most common cluster scheduling environments, a job can be scheduled to more than one machine or core. Also, the jobs are accumulations of smaller tasks that are related to each other by directed acyclic graphs. We plan to address these in our future work.

Algorithm	*E.C.R. for Google trace*	*E.C.R. for Alibaba trace*
OPT	1.0	1.0
N-SRTF	1.0	1.005
Follow Arrival Order (FAO)	4.689	4.567
Random Job First (RJF)	3.029	3.512
PSTF ($\sigma = 0$)	1.0	1.0
PSTF ($\sigma = 100$)	1.200	1.987
PSTF ($\sigma = 1000$)	2.056	3.901
PSTF ($\sigma = 5000$)	5.250	9.176
($\lambda = 0.25$)		
MLA-RJF ($\sigma = 0$)	1.391	1.225
MLA-RJF ($\sigma = 100$)	1.848	1.987
MLA-RJF ($\sigma = 1000$)	2.882	2.532
MLA-RJF ($\sigma = 5000$)	3.000	3.523
($\lambda = 0.5$)		
MLA-RJF ($\sigma = 0$)	1.152	1.200
MLA-RJF ($\sigma = 100$)	1.478	1.854
MLA-RJF ($\sigma = 1000$)	2.809	2.489
MLA-RJF ($\sigma = 5000$)	2.985	3.334
($\lambda = 0.75$)		
MLA-RJF ($\sigma = 0$)	1.041	1.119
MLA-RJF ($\sigma = 100$)	1.291	2.056
MLA-RJF ($\sigma = 1000$)	2.723	3.002
MLA-RJF ($\sigma = 5000$)	2.959	3.689

Table 5.1: Empirical Competitive Ratio (E.C.R.) of the following non-clairvoyant scheduling strategies studied in this work under the kill-and-restart model. Random Job First (RJF) is the best-known online algorithm without learned advice. ML-advised RJF has a hyperparameter λ. We investigate the performance for three values of λ: 0.25, 0.5, and 0.75. The predicted job length has an error of normal distribution, the mean is the actual job length, and the standard deviation (σ) is varied with the following values: 0, 100, 1000, and 5000. These are absolute values in terms of seconds, whereas the job lengths range from 5–380 seconds in the Google trace and 1–2000 in the Alibaba trace. Hence, k, the ratio of the longest job length to the shortest job length, is 78 and 2000 respectively.

Chapter 6

Conclusion

This thesis has explored two distinct approaches to move beyond worst-case analysis in algorithm design: cache adaptivity and machine learning advice. Both offer valuable insights into how algorithms can be tailored to perform well in realistic scenarios, rather than solely focusing on theoretical worst-case bounds. For systems designers, these insights become helpful to choose algorithms as they design efficient systems with good theoretical performance bounds. By considering factors like memory fluctuations and predictive capabilities in modern systems, we can design algorithms that are more robust, efficient, and better suited to real-world applications.

Cache-adaptive algorithms for modern systems with resource-sharing. Our investigation into cache adaptivity revealed the importance of considering memory fluctuations in multi-core, multi-threaded, shared-memory, and cloud systems. By developing an empirical framework, we were able to assess the cache adaptivity of various algorithms under a range of memory conditions. The results highlighted the robustness of a certain class of cache-oblivious algorithms with a divide-and-conquer structure, demonstrating their ability to maintain performance even in the presence of memory fluctuations. This finding has practical implications for algorithm design for parallel systems, suggesting that cache-obliviousness can serve as a valuable heuristic for achieving cache adaptivity. Future work could explore more sophisticated memory fluctuations and investigate the potential for dynamic adaptation to memory fluctuations, potentially leading to algorithms that can adjust their behavior in shared-resource environments, which are common in modern systems.

Machine Learning advice: performance bounds with better empirical performance. The exploration of ML-augmented online algorithms demonstrated the potential for leveraging machine learning to enhance algo-

rithmic performance. By incorporating predictions into traditional online algorithms, we can achieve provable performance gains when the predictions are accurate. Crucially, even with imperfect predictions, these algorithms maintain robustness, preventing catastrophic performance degradation. The study of the ski rental problem with a discount and the list update problem showcased the nuances of designing effective ML advice. It is not enough to simply have accurate predictions; the advice must be carefully integrated into the algorithm's decision-making process. This highlights the need for a deeper understanding of the interplay between prediction accuracy, advice design, and algorithmic performance. Future research could focus on developing more sophisticated techniques for generating and incorporating ML advice, potentially leading to algorithms that can learn and adapt to changing environments.

Towards a more nuanced view of algorithmic performance. In conclusion, this thesis has contributed to a growing body of work that seeks to move beyond worst-case analysis in algorithm design. By considering cache adaptivity and machine learning advice, we have demonstrated the potential for designing algorithms that are more robust, efficient, and better suited to real-world applications. These approaches offer valuable tools for navigating the complexities of modern computing environments, where factors like memory fluctuations can significantly affect performance and available predictive capabilities can significantly improve performance. As we continue to push the boundaries of algorithm design, it is essential to embrace a more nuanced view of algorithmic performance, one that considers not only theoretical worst-case bounds but also the practical considerations for real-world applications. This will lead to the development of modern systems that are not only theoretically sound but also practically effective, ultimately driving progress in various fields that rely on efficient and robust computation.

Bibliography

[1] Anna R. Karlin, Mark S. Manasse, L. A. McGeoch, and S. Owicki. Competitive randomized algorithms for nonuniform problems. *Algorithmica*, 11(6):542–571, 1994.

[2] Anna R. Karlin, Mark S. Manasse, Larry Rudolph, and Daniel Dominic Sleator. Competitive snoopy caching. *Algorithmica*, 3:77–119, 1988.

[3] Allan Borodin and Ran El-Yaniv. *Online Computation and Competitive Analysis*. Cambridge University Press, 1998.

[4] Daniel D. Sleator and Robert E. Tarjan. Amortized efficiency of list update and paging rules. *Communications of the ACM*, 28:202–208, 1985. doi: 10.1145/2786.2793.

[5] Tim Roughgarden. *Beyond the Worst-Case Analysis of Algorithms (Chapter 4, Resource Augmentation)*. Cambridge University Press, 2020. ISBN 9781108637435. doi: https://doi.org/10.1017/9781108637435.

[6] Michael Mitzenmacher and Sergei Vassilvitskii. Algorithms with predictions. *Communications of the ACM*, 65:33 – 35, 2020. URL `https://api.semanticscholar.org/CorpusID:219708471`.

[7] Asit Dan and Donald F. Towsley. An approximate analysis of the LRU and FIFO buffer replacement schemes. In Gary J. Nutt, editor, *Proceedings of the 1990 ACM SIGMETRICS conference on Measurement and modeling of computer systems, University of Colorado, Boulder, Colorado, USA, May 22-25, 1990*, pages 143–152. ACM, 1990. doi: 10.1145/98457.98525.

[8] Tim Roughgarden. Beyond worst-case analysis. *Commun. ACM*, 62(3): 88–96, 2019. URL `https://doi.org/10.1145/3232535`.

[9] Predrag R. Jelenković and Ana Radovanović. Least-recently-used caching with dependent requests. *Theoretical Computer Science*, 326

(1):293–327, 2004. ISSN 0304-3975. doi: https://doi.org/10.1016/j.tcs.2004.07.029. URL https://www.sciencedirect.com/science/article/pii/S030439750400475X.

[10] Rajeev Motwani and Prabhakar Raghavan. *Randomized Algorithms*. Cambridge University Press, 1995.

[11] Amos Fiat, Richard M. Karp, Michael Luby, Lyle A. McGeoch, Daniel D. Sleator, and Neal E. Young. Competitive paging algorithms. *Journal of Algorithms*, page 685–699, 1991.

[12] Daniel A. Spielman and Shang-Hua Teng. Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time. *J. ACM*, 51(3):385–463, may 2004. ISSN 0004-5411. doi: 10.1145/990308.990310.

[13] Daniel A. Spielman and Shang-Hua Teng. Smoothed analysis: An attempt to explain the behavior of algorithms in practice. *Communications of the ACM*, 52(10):76–84, 2009.

[14] Daniel A. Spielman and Shang-Hua Teng. Smoothed analysis of algorithms: why the simplex algorithm usually takes polynomial time. In Jeffrey Scott Vitter, Paul G. Spirakis, and Mihalis Yannakakis, editors, *Proceedings on 33rd Annual ACM Symposium on Theory of Computing, July 6-8, 2001, Heraklion, Crete, Greece*, pages 296–305. ACM, 2001. URL https://doi.org/10.1145/380752.380813.

[15] Jon Louis Bentley and M. Douglas McIlroy. Engineering a sort function. *Software: Practice and Experience*, 23, 1993. URL https://api.semanticscholar.org/CorpusID:8822797.

[16] Alok Aggarwal and Jeffrey S. Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, September 1988.

[17] Rezaul Alam Chowdhury. *Cache-efficient Algorithms and Data Structures: Theory and Experimental Evaluation*. PhD thesis, The University of Texas at Austin, 2007.

[18] R. Stanley Williams. What's next? [the end of moore's law]. *Computing in Science & Engineering*, 19(2):7–13, 2017. doi: 10.1109/MCSE.2017.31.

[19] Thomas N. Theis and H.-S. Philip Wong. The end of moore's law: A new beginning for information technology. *Computing in Science & Engineering*, 19(2):41–50, 2017. doi: 10.1109/MCSE.2017.29.

[20] Dave Dice, Virendra J. Marathe, and Nir Shavit. Brief announcement: Persistent unfairness arising from cache residency imbalance. In *Proc. of the 26th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 82–83, Prague, Czech Republic, 2014. ACM.

[21] Michael A. Bender, Roozbeh Ebrahimi, Jeremy T. Fineman, Golnaz Ghasemiesfeh, Rob Johnson, and Samuel McCauley. Cache-adaptive algorithms. In *Proc. 25th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 958–971, Portland, Oregon, 2014. ACM-SIAM.

[22] Michael A. Bender, Erik D. Demaine, Roozbeh Ebrahimi, Jeremy T. Fineman, Rob Johnson, Andrea Lincoln, Jayson Lynch, and Samuel McCauley. Cache-adaptive analysis. In *Proc. 28rd Annual ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 135–144, Asilomar State Beach, CA, 2016. ACM.

[23] HweeHwa Pang, Michael J. Carey, and Miron Livny. Memory-adaptive external sorting. In *Proc. 19th International Conference on Very Large Data Bases (VLDB)*, pages 618–629, Dublin, Ireland, 1993. Morgan Kaufmann.

[24] HweeHwa Pang, Michael J Carey, and Miron Livny. Partially preemptible hash joins. In *Proc. 5th ACM SIGMOD International Conference on Management of Data (COMAD)*, pages 59–68, Washington, DC, 1993. ACM.

[25] Weiye Zhang and Per-Äke Larson. A memory-adaptive sort (MASORT) for database systems. In *Proc. 6th International Conference of the Centre for Advanced Studies on Collaborative research (CASCON)*, pages 41–54, Toronto, Ontario, Canada, 1996. IBM Press.

[26] Weiye Zhang and Per-Äke Larson. Dynamic memory adjustment for external mergesort. In *Proc. 23rd International Conference on Very Large Data Bases (VLDB)*, pages 376–385, Athens, Greece, 1997. Morgan Kaufmann.

[27] Richard Tran Mills. *Dynamic adaptation to CPU and memory load in scientific applications*. PhD thesis, The College of William and Mary, 2004.

[28] Richard T Mills, Andreas Stathopoulos, and Dimitrios S Nikolopoulos. Adapting to memory pressure from within scientific applications on multiprogrammed cows. In *Proc. 8th International Parallel and Distributed Processing Symposium (IPDPS)*, pages 71–88, Santa Fe, New Mexico, 2004. IEEE.

[29] Richard T Mills, Chuan Yue, Andreas Stathopoulos, and Dimitrios S Nikolopoulos. Runtime and programming support for memory adaptation in scientific applications via local disk and remote memory. *Journal of Grid Computing*, 5:213—-234, 2007.

[30] Rakesh D. Barve. *Algorithmic Techniques To Overcome The I/O Bottleneck*. PhD thesis, Duke University, 1998.

[31] Rakesh D. Barve and Jeffrey Scott Vitter. A theoretical framework for memory-adaptive algorithms. In *Proc. 40th Annual Symposium on the Foundations of Computer Science (FOCS)*, pages 273–284, New York City, NY, 1999. IEEE.

[32] Gerth Stølting Brodal and Rolf Fagerberg. Cache oblivious distribution sweeping. In *Proc. 29th International Colloquium on Automata, Languages and Programming (ICALP)*, pages 426–438, Malaga, Spain, 2002. Springer-Verlag.

[33] Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. Cache-oblivious algorithms. In *Proc. of the 40th Annual Symposium on the Foundations of Computer Science (FOCS)*, pages 285–298, New York City, NY, 1999. IEEE.

[34] Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80, pages 3296–3305. PMLR, 2018.

[35] Michael Mitzenmacher. A model for learned bloom filters, and optimizing by sandwiching. In *32nd Conference on Neural Information Processing Systems (NeurIPS)*, 2018.

[36] Andres Muñoz Medina and Sergei Vassilvitskii. Revenue optimization with approximate bid predictions. In *31st Conference on Neural Information Processing Systems (NeurIPS)*, 2017.

[37] Tim Kraska, Alex Beutel, Ed H. Chi, Jeff Dean, and Neoklis Polyzotis. The case for learned index structures. In *SIGMOD/PODS International Conference on Management of data*, Houston, TX, USA, 2018. ACM.

[38] Sreenivas Gollapudi and Debmalya Panigrahi. Online algorithms for rent-or-buy with expert advice. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pages 2319–2327. ACM, 2019.

[39] Thodoris Lykouris and Sergei Vassilvitskii. Competitive caching with machine learned advice. *Journal of the ACM*, 68(24):1—25, 2021. doi: 10.1145/3447579.

[40] Arghya Bhattacharya, Abiyaz Chowdhury, Helen Xu, Rathish Das, Rezaul A. Chowdhury, Rob Johnson, Rishab Nithyanand, and Michael A. Bender. When Are Cache-Oblivious Algorithms Cache Adaptive? A Case Study of Matrix Multiplication and Sorting. In *30th Annual European Symposium on Algorithms (ESA 2022)*, volume 244 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 16:1–16:17, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. ISBN 978-3-95977-247-1. doi: 10.4230/LIPIcs.ESA.2022.16.

[41] Peter J. Denning. Thrashing: its causes and prevention. In *Proceedings of the December 9-11, 1968, fall joint computer conference, part I*, AFIPS '68, pages 915–922, San Francisco, CA, 1968. AFIPS.

[42] Peter J. Denning. Working sets past and present. *IEEE Transactions on Software Engineering*, 6(1):64–84, 1980.

[43] J. Ousterhout. Scheduling techniques for concurrent systems. In *Proc. of the 3rd IEEE International Conference on Distributed Computing Systems (ICDCS)*, pages 22–30, Miami, Florida, 1982. IEEE.

[44] Matteo Frigo, Charles E. Leiserson, Harald Prokop, and Sridhar Ramachandran. Cache-oblivious algorithms. *ACM Transactions on Algorithms*, 8(1):4, 2012.

[45] H. Prokop. Cache oblivious algorithms. Master's thesis, Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, June 1999.

[46] Masaru Kitsuregawa, Hidehiko Tanaka, and Tohru Moto-Oka. Application of hash to data base machine and its architecture. *New Generation Computing*, 1:63–74, 1983.

[47] Kurt P Brown, Michael James Carey, and Miron Livny. Managing memory to meet multiclass workload response time goals. In *Proc. of the 19th International Conference on Very Large Data Bases (VLDB)*, pages 328–328, Dublin, Ireland, 1993. IEEE.

[48] Michael A. Bender, Gerth Stølting Brodal, Rolf Fagerberg, Dongdong Ge, Simai He, Haodong Hu, John Iacono, and Alejandro López-Ortiz. The cost of cache-oblivious searching. In *Proceedings of the 44th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 271–280, Cambridge, MA, 2003. IEEE.

[49] Michael A. Bender, Gerth Stølting Brodal, Rolf Fagerberg, Dongdong Ge, Simai He, Haodong Hu, John Iacono, and Alejandro López-Ortiz. The cost of cache-oblivious searching. *Algorithmica*, 61(2):463–505, 2011.

[50] Michael A. Bender, Rezaul A. Chowdhury, Rathish Das, Rob Johnson, William Kuszmaul, Andrea Lincoln, Quanquan C Liu, Jayson Lynch, and Helen Xu. Closing the gap between cache-oblivious and cache-adaptive analysis. In *Proc. 19th Annual ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 63–73, virtual event, USA, 2020. ACM.

[51] Rezaul Alam Chowdhury and Vijaya Ramachandran. The cache-oblivious gaussian elimination paradigm: theoretical framework, parallelization and experimental evaluation. *Theory of Computing Systems*, 47(4):878–919, 2010.

[52] Andreas Björklund, Rasmus Pagh, Virginia Vassilevska Williams, and Uri Zwick. Listing triangles. In *Proc. of 41st International Colloquium on Automata, Languages, and Programming*, pages 223–234, Copenhagen, Denmark, 2014. Springer.

[53] Virginia Vassilevska Williams and Ryan Williams. Subcubic equivalences between path, matrix and triangle problems. In *Proc. of the 51st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 645–654, Las Vegas, NV, USA, 2010. IEEE.

[54] Rezaul A. Chowdhury, Pramod Ganapathi, Stephen Tschudi, Jesmin Jahan Tithi, Charles Bachmeier, Charles E. Leiserson, Armando Solar-Lezama, Bradley C. Kuszmaul, and Yuan Tang. Autogen: Automatic discovery of efficient recursive divide-&-conquer algorithms for solving dynamic programming problems. *ACM Transactions on Parallel Computing*, 4(1):1–12, 2017.

[55] Rezaul A. Chowdhury, Pramod Ganapathi, Jesmin Jahan Tithi, Yuan Tang, Charles A. Bachmeier, Bradley C Kuszmaul, Charles E. Leiserson, and Armando Solar Lezama. Autogen: Automatic discovery of efficient recursive divide-&-conquer algorithms for solving dynamic programming problems. In *Proc. of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 1–12, Barcelona, Spain, 2016. ACM.

[56] Gerth Stølting Brodal, Rolf Fagerberg, and Kristoffer Vinther. Engineering a cache-oblivious sorting algorithm. *ACM Journal of Experimental Algorithmics*, 12:1–23, 2007.

[57] Jesper Holm Olsen, Søren Skov, and Frederik Rønn. Cpp implementations of a cache-oblivious sorting method. Private communication, June 2003.

[58] Transaction Processing Performance Council. Tpc-c benchmark. Technical Report 5.10.1, TPC, 2009. URL `http://www.tpc.org/tpcc/`.

[59] Arghya Bhattacharya and Rathish Das. Machine learning advised algorithms for the ski rental problem with a discount. *Theoretical Computer Science*, 938:39–49, 2022. ISSN 0304-3975. doi: https://doi.org/10.1016/j.tcs.2022.10.006. URL `https://www.sciencedirect.com/science/article/pii/S0304397522005862`.

[60] Arghya Bhattacharya and Rathish Das. Machine learning advised ski rental problem with a discount. In P. Mutzel, M.S. Rahman, and S. Slamin, editors, *Proceedings of the 16th International Conference and Workshops on Algorithms and Computation (WALCOM)*, volume 13174 of *Lecture Notes in Computer Science (LNCS)*, pages 213–224. Springer, Cham, 2022. doi: 10.1007/978-3-030-96731-4_18.

[61] Adam Kalai and Santosh Vempala. Efficient algorithms for online decision problems. *Journal of Computer and System Sciences*, 71:291–307, 2005. doi: 10.1016/j.jcss.2004.10.016.

[62] Alexander Rakhlin and Karthik Sridharan. Online learning with predictable sequences. In *Proceedings of the 26th Annual Conference on Learning Theory (COLT)*, pages 1–27, 2013.

[63] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Proceedings of the 28th International Con- ference on Neural*

Information Processing Systems (NIPS), pages 2503 – 2511. MIT Press, 2015.

[64] Hiroshi Fujiwara and Kazuo Iwama. Average-case competitive analyses for ski-rental problems. *Algorithmica*, 42:95–107, 2005.

[65] Yinfeng Xu, Weijun Xu, and Hongyi Li. On the on-line rent-or-buy problem in probabilistic environments. *Journal of Global Optimization*, 38:1–20, 2007.

[66] Dhruv Rohatgi. Near-optimal bounds for online caching with machine learned advice. In *Proceedings of the 14th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1834–1845, Salt Lake City, Utah, USA, January, 2020. ISBN 978-1-61197-599-4. doi: 10.1137/1.9781611975994.112.

[67] Alexander Wei. Better and simpler learning-augmented online caching. In Jarosław Byrka and Raghu Meka, editors, *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM)*, volume 17, pages 1–60. Dagstuhl Publishing, Germany, 2020.

[68] Jakub Chledowski, Adam Polak, Bartosz Szabucki, and Konrad Zolna. Robust learning-augmented caching: An experimental study. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139, pages 1920–1930. PMLR, 2021.

[69] Zhihao Jiang, Debmalya Panigrahi, and Kevin Sun. Online algorithms for weighted paging with predictions. In *Proceedings of 47th International Colloquium on Automata, Languages, and Programming (ICALP)*, page 69:1–69:18. LIPICS, 2020.

[70] Antonios Antoniadis, Christian Coester, Marek Elia, Adam Polak, and Bertrand Simon. Online metric algorithms with untrusted predictions. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119, pages 345–355, Virtual, 2020. PMLR.

[71] Manish Purohit, Zoya Svitkina, and Ravi Kumar. Improving online algorithms via ml predictions. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Proceedings of 32nd Conference on Neural Information Processing Systems (NeurIPS)*, volume 31. Curran Associates, Inc., 2018.

[72] Silvio Lattanzi, Thomas Lavastida, Benjamin Moseley, and Sergei Vassilvitskii. Online scheduling via learned weights. In *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1859–1877, 2020.

[73] Michael Mitzenmacher. Scheduling with predictions and the price of misprediction. In *Proceedings of Innovations in Theoretical Computer Science (ITCS)*, 2020.

[74] Sungjin Im, Ravi Kumar, Mahshid Montazer Qaem, and Manish Purohit. Non-clairvoyant scheduling with predictions. In *Proceedings of the 33rd ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 285–294, Virtual, USA, 2021. doi: 10.1145/3409964.3461790.

[75] Spyros Angelopoulos and Shahin Kamali. Contract scheduling with predictions. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI)*, volume 35, 2021.

[76] Spyros Angelopoulos. Online search with a hint. In *Proceedings of the 12th Innovations in Theoretical Computer Science Conference (ITCS)*, page 51:1–51:16. LIPICS, 2021.

[77] Spyros Angelopoulos, Shahin Kamali, and Kimia Shadkami. Online bin packing with predictions, 2021.

[78] Spyros Angelopoulos, Christoph Dürr, Shendan Jin, Shahin Kamali, and Marc Renault. Online computation with untrusted advice. In *Proceedings of 11th Innovations in Theoretical Computer Science Conference (ICTS)*, pages 52:1—-52:15. LIPICS, 2020.

[79] Paolo Ferragina, Fabrizio Lillo, and Giorgio Vinciguerra. On the performance of learned data structures. *Theoretical Computer Science*, 871: 107–120, 2021.

[80] Keerti Anand, Rong Ge, and Debmalya Panigrahi. Customizing ML predictions for online algorithms. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 303–313. PMLR, 13–18 Jul 2020.

[81] Sandy Irani and Dinesh Ramanathan. The problem of renting versus buying. Private communication, August 1998.

[82] Adam Meyerson. The parking permit problem. In *46th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 274–284, 2005.

[83] Ali Khanafer, Murali Kodialam, and Krishna P. N. Puttaswamy. The constrained ski-rental problem and its application to online cloud cost optimization. In *Proc. of IEEE INFOCOM*, INFOCOM, pages 1492–1500, Turin, Italy, 2013. IEEE.

[84] Zvi Lotker, Boaz Patt-Shamir, and Dror Rawitz. Rent, lease or buy: Randomized algorithms for multislope ski rental. In *Proc. of 25th Annual Symposium on the Theoretical Aspects of Computer Science*, STACS, pages 503–514, Bordeaux, France, 2008.

[85] Zvi Lotker, Boaz Patt-Shamir, and Dror Rawitz. Ski rental with two general options. *Information Processing Letters*, 108:365–368, 2008.

[86] Guiqing Zhang, Chung Keung Poon, and Yinfeng Xu. The ski-rental problem with multiple discount options. *Information Processing Letters*, 111(18):903–906, 2011.

[87] R. El-Yaniv, R. Kaniel, and N. Linial. Competitive optimal on-line leasing. *Algorithmica*, 25:116—-140, 1999.

[88] Baruch Awerbuch and Yossi Azar. Buy-at-bulk network design. *Symposium on Foundations of Computer Science*, 1997.

[89] Anna R. Karlin, Claire Kenyon, and Dana Randall. Dynamic tcp acknowledgement and other stories about $e/(e-1)$. *Algorithmica*, 36(3): 209–224, 2003.

[90] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, nov 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL `https://doi.org/10.1162/neco.1997.9.8.1735`.

[91] Arghya Bhattacharya, Shahin Kamali, and Helen Xu. Online list update with predictions. Poster presented at the Learning-augmented Algorithms: Theory and Applications, June 2023. Hosted at ACM SIGMETRICS 2023 at Orlando, FL, USA.

[92] Shahin Kamali and Alejandro López-Ortiz. *A Survey of Algorithms and Models for List Update*, volume 8066 of *Lecture Notes in Comput. Sci.* Springer, Berlin, Heidelberg, 2013.

[93] Sandy Irani. Two results on the list update problem. *Information Processing Letters*, 38(6):301–306, 1991. ISSN 0020-0190.

[94] Christoph Ambühl. Offline list update is np-hard. In Mike S. Paterson, editor, *Algorithms - ESA 2000*, pages 42–51, Berlin, Heidelberg, 2000. Springer. ISBN 978-3-540-45253-9.

[95] Joan Boyar, Shahin Kamali, Kim S. Larsen, and Alejandro López-Ortiz. On the list update problem with advice. *Information and Computation*, 253:411–423, 2017. ISSN 0890-5401. LATA 2014.

[96] Susanne Albers. A competitive analysis of the list update problem with lookahead. *Theoretical Computer Science*, 197(1):95–109, 1998. ISSN 0304-3975.

[97] Spyros Angelopoulos, Christoph Dürr, Shendan Jin, Shahin Kamali, and Marc P. Renault. Online computation with untrusted advice. In Thomas Vidick, editor, *Proceedings of the* 11th *Innovations in Theoretical Computer Science Conference (ITCS)*, volume 151 of *LIPICS*, pages 52:1–52:15, Seattle, WA, USA, 2020. Dagstuhl Publishing, Germany.

[98] Antonios Antoniadis, Christian Coester, Marek Eliás, Adam Polak, and Bertrand Simon. Online metric algorithms with untrusted predictions. In *Proceedings of the* 37th *International Conference on Machine Learning (ICML)*, volume 119, pages 345–355, Virtual, 2020. PMLR.

[99] Antonios Antoniadis, Christian Coester, Marek Eliáš, Adam Polak, and Bertrand Simon. Online metric algorithms with untrusted predictions. *ACM Trans. Algorithms*, 19(2), apr 2023. ISSN 1549-6325.

[100] Rajeev Motwani, Steven J. Phillips, and Eric Torng. Non-clairvoyant scheduling. In Vijaya Ramachandran, editor, *Proceedings of the fourth annual symposium on Discrete algorithms (SODA)*, pages 422–431, Austin, TX, USA, 1993. ACM-SIAM.

[101] Eugene L. Lawler, Jan Karel Lenstra, Alexander H.G. Rinnooy Kan, and David B. Shmoys. Chapter 9 sequencing and scheduling: Algorithms and complexity. In *Logistics of Production and Inventory*, volume 4 of *Handbooks in Operations Research and Management Science*, pages 445–522. Elsevier, 1993. doi: https://doi.org/10.1016/S0927-0507(05)80189-6. URL `https://www.sciencedirect.com/science/article/pii/S0927050705801896`.

[102] Andrew S. Tanenbaum and Robbert Van Renesse. Distributed operating systems. *ACM Comput. Surv.*, 17(4):419–470, dec 1985. ISSN 0360-0300. doi: 10.1145/6041.6074. URL `https://doi.org/10.1145/6041.6074`.

[103] Shanka Subhra Mondal, Nikhil Sheoran, and Subrata Mitra. Scheduling of time-varying workloads using reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(10):9000–9008, May 2021. doi: 10.1609/aaai.v35i10.17088. URL `https://ojs.aaai.org/index.php/AAAI/article/view/17088`.

[104] Jun Woo Park, Alexey Tumanov, Angela Jiang, Michael A. Kozuch, and Gregory R. Ganger. 3sigma: Distribution-based cluster scheduling for runtime uncertainty. In *Proceedings of the Thirteenth EuroSys Conference*, EuroSys '18, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450355841. doi: 10.1145/3190508.3190515. URL `https://doi.org/10.1145/3190508.3190515`.

[105] Rajeev Motwani, Steven J. Phillips, and Eric Torng. Non-clairvoyant scheduling. *theoretical Computer Science*, 130(1):17–47, 1994.

[106] S. Rao Kosaraju, Mike Fellows, Avi Wigderson, and John A. Ellis, editors. *New Algorithms for an Ancient Scheduling Problem*, volume 51, Victoria, British Columbia, CA, 1992. ACM. doi: 10.1145/129712.129718.

[107] D.B. Shmoys, J. Wein, and D.P. Williamson. Scheduling parallel machines on-line. In *Proceedings 32nd Annual Symposium of Foundations of Computer Science*, pages 131–140, 1991.

[108] Yun Chi, Hakan Hacígümüş, Wang-Pin Hsiung, and Jeffrey F. Naughton. Distribution-based query scheduling. *Proc. VLDB Endow.*, 6(9):673–684, jul 2013. ISSN 2150-8097. doi: 10.14778/2536360.2536367. URL `https://doi.org/10.14778/2536360.2536367`.

[109] Kay Ousterhout, Patrick Wendell, Matei Zaharia, and Ion Stoica. Sparrow: Distributed, low latency scheduling. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, SOSP '13, pages 69–84, New York, NY, USA, 2013. Association for Computing Machinery. ISBN 9781450323888. doi: 10.1145/2517349.2522716. URL `https://doi.org/10.1145/2517349.2522716`.

[110] Leah Epstein and Lene M. Favrholdt. Optimal non-preemptive semi-online scheduling on two related machines. In *Mathematical Founda-*

tions of Computer Science (MFCS'02), volume 2420 of *Lecture Notes in Computer Science*, pages 245–256, Berlin, Heidelberg, 2002. Springer.

[111] Rodney R. Howell and Muralidhar K. Venkatrao. On non-preemptive scheduling of recurring tasks using inserted idle times. *Information and Computation*, pages 50–62, 1995.

[112] Kevin Jeffay, Donald F. Stanat, and Charles U. Martel. On non-preemptive scheduling of periodic and sporadic tasks. In *Proceedings of the Twelfth IEEE Real-Time Systems Symposium*, pages 129–139, San Antonio, TX, USA, 1991. IEEE Computer Society Press.

[113] Jeff Edmonds. Scheduling in the dark. In *Proceedings of the* 31st *Annual ACM Symposium on Theory of Computing*, STOC '99, page 179–188, New York, NY, USA, 1999. Association for Computing Machinery. ISBN 1581130678. doi: 10.1145/301250.301299. URL `https://doi.org/10.1145/301250.301299`.

[114] Jeff Edmonds. Scheduling in the dark. *Theoretical Computer Science*, 235(1):109–141, 2000. ISSN 0304-3975. doi: https://doi.org/10.1016/S0304-3975(99)00186-3. URL `https://www.sciencedirect.com/science/article/pii/S0304397599001863`.

[115] Bala Kalyanasundaram and Kirk Pruhs. Speed is as powerful as clairvoyance [scheduling problems]. In *Proceedings of IEEE 36th Annual Foundations of Computer Science*, pages 214–221, Milwaukee, WI, USA, 1995. IEEE. doi: 10.1109/SFCS.1995.492478.

[116] Bala Kalyanasundaram and Kirk Pruhs. Speed is as powerful as clairvoyance. *J. ACM*, 47(4):617–643, jul 2000. ISSN 0004-5411. doi: 10.1145/347476.347479. URL `https://doi.org/10.1145/347476.347479`.

[117] Alexey Tumanov, Timothy Zhu, Jun Woo Park, Michael A. Kozuch, Mor Harchol-Balter, and Gregory R. Ganger. Tetrisched: Global rescheduling with adaptive plan-ahead in dynamic heterogeneous clusters. In *Proceedings of the* 11th *European Conference on Computer Systems*, EuroSys '16, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450342407. doi: 10.1145/2901318.2901355. URL `https://doi.org/10.1145/2901318.2901355`.

[118] Sungjin Im, Ravi Kumar, Mahshid Montazer Qaem, and Manish Purohit. Non-clairvoyant scheduling with predictions. In *Proceedings of the 33rd ACM Symposium on Parallelism in Algorithms and Architectures*,

SPAA'21, pages 285–294, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450380706. doi: 10.1145/3409964.3461790. URL https://doi.org/10.1145/3409964.3461790.

[119] Alexander Lindermayr and Nicole Megow. Permutation predictions for non-clairvoyant scheduling. In *Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA '22, page 357–368, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450391467. doi: 10.1145/3490148.3538579. URL https://doi.org/10.1145/3490148.3538579.

[120] Spyros Angelopoulos, Christoph Dürr, Shendan Jin, Shahin Kamali, and Marc Renault. Online computation with untrusted advice. In Thomas Vidick, editor, 11th *Innovations in Theoretical Computer Science Conference (ITCS 2020)*, pages 52:1–52:15. LIPICS, 2020. doi: 10.4230/LIPIcs.ITCS.2020.52.

[121] Alexey Tumanov, Angela Jiang, Jun Woo Park, Michael A. Kozuch, and Gregory R. Ganger. *JamaisVu: Robust Scheduling with Auto-Estimated Job Runtimes*. Carnegie Mellon University, 2016.

[122] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proc. of the* 3rd *ACM Symposium on Cloud Computing (SOCC'12)*. ACM, 2012.

[123] Congfeng Jiang, Guangjie Han, Jiangbin Lin, Gangyong Jia, Weisong Shi, and Jian Wan. Characteristics of co-allocated online services and batch jobs in internet data centers: A case study from alibaba cloud. *IEEE Access*, 7:22495–22508, 2019. doi: 10.1109/ACCESS.2019.2897898.

[124] Richard W. Conway, William L. Maxwell, and Louis W. Miller. *Theory of Scheduling*. Addison-Wesley, Reading, MA, USA, 1967.

[125] Leonard Kleinrock. *Queueing Systems*, volume 1. John Wiley & Sons, Inc., New York, NY, USA, 1975.

[126] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Prentice-Hall, Englewood Cliffs, 4 edition, 1992.

[127] Alexander Wei and Fred Zhang. Optimal robustness-consistency tradeoffs for learning-augmented online algorithms. In *Proc. of 34th Con-*

ference on Neural Information Processing Systems (NeurIPS), NIPS'20, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.

[128] Yossi Azar, Stefano Leonardi, and Noam Touitou. Flow time scheduling with uncertain processing time. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2021, pages 1070–1080, New York, NY, USA, 2021. Association for Computing Machinery.